



**SÃO PAULO**  
GOVERNO DO ESTADO  
SÃO PAULO SÃO TODOS

## Faculdade de Tecnologia de Rio Preto

### Prova #2: CPU 4 Bits & 8 Operações

*Central Processing Unit (Unidade Central de Processamento) 4 Bits & 8 Operações*

**Aluno:** Breno Ferracini de Oliveira – **RA:** 1210482522030

**Aluno:** Luiz Francisco Neves Mendes – **RA:** 1210482522040

**Turma:** Tarde

**Disciplina:** Arquitetura e Organização de Computador

**Curso:** Tecnologia em Análise e Desenvolvimento de Sistemas

**Professor:** Me José A. A. Viana

São José do Rio Preto, 04 de Dezembro de 2025.

# Sumário

|                                                                     |    |
|---------------------------------------------------------------------|----|
| INTRODUÇÃO.....                                                     | 4  |
| OBJETIVO.....                                                       | 5  |
| GLOSSÁRIO.....                                                      | 6  |
| Valores Digitais.....                                               | 6  |
| Termos Estruturais / Conceituais.....                               | 8  |
| Termos Lógicos / Matemáticos Gerais.....                            | 9  |
| 1 — FUNDAMENTOS.....                                                | 12 |
| 1.1 — Glossário dos Fundamentos.....                                | 12 |
| 1.1.1 — Termos de Circuitos.....                                    | 12 |
| 1.1.2 — Termos sobre as Operações.....                              | 15 |
| 1.1.3 — Termos relacionados à Simulação.....                        | 17 |
| 1.2 — Cálculos e Conversões.....                                    | 20 |
| 1.2.1 — Dígitos binários.....                                       | 20 |
| 1.2.2 — Agrupamento em Nibbles.....                                 | 20 |
| 1.2.3 — Por que multiplicar por 16.....                             | 20 |
| 1.2.4 — Conversões no Sistema Hexadecimal.....                      | 21 |
| 1.3 — Unidade Lógica Aritmética (ULA).....                          | 22 |
| 1.4 — Fundamentos dos Circuitos Lógicos Combinacionais Digital..... | 23 |
| 1.4.1 — Circuito Integrados (CIs).....                              | 23 |
| 1.4.2 — Portas lógicas.....                                         | 23 |
| 1.4.3 — Resumo dos símbolos em expressões.....                      | 32 |
| 1.5 — Ferramenta de Simulação: Logisim.....                         | 33 |
| 1.5.1 — Uso do Logisim.....                                         | 35 |
| 2 — MÉTODO DE DESENVOLVIMENTO.....                                  | 36 |
| 2.1 — Escolha da organização e metodologia de construção.....       | 36 |
| 2.2 — Agrupamento por categorias.....                               | 36 |
| 2.3 — Estratégia para barramentos e Tri-States.....                 | 36 |
| 2.4 — Módulos de memória e sincronização (clock).....               | 36 |
| 2.5 — Expansão da ULA e reutilização de estruturas.....             | 37 |
| 2.6 — Decodificação e multiplexação de controle.....                | 37 |
| 2.7 — Decisões estéticas: fios multibit e macros customizados.....  | 37 |
| 2.8 — Handlers (Tratadores) & Helpers (Ajudantes).....              | 37 |
| 2.9 — Testes, validação incremental e documentação.....             | 37 |
| 3 — DESENVOLVIMENTO DOS MÓDULOS.....                                | 38 |
| 3.1 — Módulos Básicos.....                                          | 38 |
| 3.1.1 — Explicações básicas.....                                    | 38 |
| 3.1.2 — Decodificadores construídos.....                            | 39 |
| 3.1.3 — MUX 1 bit construídos.....                                  | 44 |
| 3.1.4 — MUX 4 bit construídos (Encadeamento).....                   | 50 |
| 3.1.5 — Shifter (Pós ULA).....                                      | 56 |
| 3.1.6 — Helper 4op to 3op.....                                      | 58 |
| 3.2 — Módulos de Controle (Tri-States).....                         | 60 |

|                                                                               |     |
|-------------------------------------------------------------------------------|-----|
| 3.2.1 — Transmission Gate.....                                                | 61  |
| 3.2.2 — High & Low Enable.....                                                | 62  |
| 3.2.3 — Buffers Tri-States.....                                               | 63  |
| 3.2.4 — Inverters Tri-States.....                                             | 67  |
| 3.2.5 — Macros Tri-States.....                                                | 71  |
| 3.3 — Módulos de Memória.....                                                 | 72  |
| 3.3.1 — Latches Transparentes.....                                            | 72  |
| 3.3.1.1 — Latches Transparentes.....                                          | 72  |
| 3.3.1.2 — Latches Master.....                                                 | 73  |
| 3.3.1.3 — Latches Slave.....                                                  | 75  |
| 3.3.2 — Flip Flop D.....                                                      | 77  |
| 3.3.2.1 — Implementações construídas.....                                     | 77  |
| 3.3.2.2 — Observações importantes.....                                        | 77  |
| 3.3.2.3 — Uso dos Flip-Flops nativos da ferramenta.....                       | 77  |
| 3.3.2.4 — Flip Flop D 1 bit.....                                              | 79  |
| 3.3.2.5 — Flip Flop D 4 bit.....                                              | 80  |
| 3.3.3 — Latch AB (Latch A e Latch B).....                                     | 81  |
| 3.3.4 — MAR (Memory Address Register).....                                    | 82  |
| 3.3.5 — MBR (Memory Buffer Register).....                                     | 84  |
| 3.3.5.1 — Pure MBR (MBR Puro).....                                            | 85  |
| 3.3.5.2 — MBR.....                                                            | 86  |
| 3.3.5.3 — Handler MBR.....                                                    | 87  |
| 3.3.6 — Registradores (Registers).....                                        | 88  |
| 3.3.6.1 — Register.....                                                       | 89  |
| 3.3.6.2 — Handlers Register.....                                              | 90  |
| 3.3.7 — Banco de Registradores (Registers Bank).....                          | 91  |
| 4 — DESENVOLVIMENTO DA ULA.....                                               | 93  |
| 4.1 — NAND.....                                                               | 93  |
| 4.2 — NOR.....                                                                | 94  |
| 4.3 — XOR.....                                                                | 95  |
| 4.4 — XNOR.....                                                               | 96  |
| 4.5 — Somador (Adder).....                                                    | 97  |
| 4.5.1 — Meio Somador (Adder Half).....                                        | 98  |
| 4.5.2 — Somador Inteiro ou 1 bit (Full Adder).....                            | 99  |
| 4.5.3 — Somador 4 bits usando Ripple-Carry (Adder).....                       | 101 |
| 4.6 — Subtrator (Subtractor).....                                             | 103 |
| 4.6.1 — Subtrator 4 bits usando Complemento (Subtractor).....                 | 103 |
| 4.7 — Multiplicador (Multiplier).....                                         | 105 |
| 4.7.1 — Multiplicador Combinacional 2 bits (Multiplier 2 bits).....           | 105 |
| 4.7.2 — Multiplicador 4 bits usando o Multiplicador de 2 bits.....            | 108 |
| 4.7.3 — Multiplicador 4 bits usando Array Multiplier (Multiplier 4 bits)..... | 109 |
| 4.8 — Divisor (Divider).....                                                  | 112 |
| 4.8.1 — Método 1: Divisor teórico.....                                        | 112 |
| 4.8.2 — Método 2: Divisor com Flip-Flops / circuitos sequencial.....          | 112 |

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| 4.8.3 — Método 3: Divisor do Logisim (utilizado na ULA).....        | 112 |
| 4.9 — Junção na ULA.....                                            | 114 |
| 5 — DESENVOLVIMENTO DOS MÓDULOS PARA A CPU.....                     | 116 |
| 5.1 — MIR (Microinstruction Register).....                          | 116 |
| 5.1.1 — Lógica de organização e ordem dos bits.....                 | 116 |
| 5.1.2 — Ordem dos Bits.....                                         | 116 |
| 5.2 — Banco MIR (Microinstruction ROM).....                         | 119 |
| 5.1.1 — Mapeamento.....                                             | 121 |
| 5.1.1.1 — Mascara Hex (Hex Mask).....                               | 121 |
| 5.1.1.2 — Palavras de endereço na formatação (ROM WordAddress)..... | 121 |
| 5.1.2 — Instruções Configuradas.....                                | 122 |
| 5.1.3 — Aviso de teste.....                                         | 123 |
| 5.3 — RxTx (Receiver/Transcriber).....                              | 124 |
| 5.3 — Clock ou Sub-ciclos.....                                      | 125 |
| 5.3.1 — Configuração dos Clocks no Logisim.....                     | 125 |
| 5.3 — Input e Output de Dados.....                                  | 127 |
| 5.3 — Entrada de Instruções.....                                    | 128 |
| 6 — RESULTADO DA CPU.....                                           | 129 |
| CONCLUSÃO.....                                                      | 131 |
| Referências Bibliográficas.....                                     | 132 |

## INTRODUÇÃO

O desenvolvimento de uma CPU representa um passo essencial na construção de sistemas digitais capazes de executar tarefas de forma organizada, previsível e controlada. Enquanto a ULA fornece os mecanismos básicos de cálculo e operação lógica, é a CPU que coordena, direciona e integra esses processos, estabelecendo o fluxo de dados e o sequenciamento das instruções.

Neste trabalho, parte-se da ULA de 4 bits já construída previamente — um módulo funcional que serviu como base estrutural. Essa ULA será aprimorada e expandida, permitindo a implementação de um conjunto maior de operações e possibilitando sua integração ao restante da arquitetura. A partir dela, será projetada a CPU completa, incluindo controle, caminhos de dados e os componentes necessários para simular o comportamento de um processador simples, porém funcional.

## **OBJETIVO**

O objetivo deste trabalho é desenvolver e simular uma Unidade Central de Processamento (CPU) simples, utilizando um software capaz de representar o comportamento lógico de seus componentes internos. A CPU será construída de forma modular, contemplando blocos como unidade de controle, caminhos de dados, registradores, contador de programa, memória de instruções e demais elementos essenciais para o fluxo de execução.

A ULA de 4 bits previamente desenvolvida servirá como base para esta implementação, sendo aprimorada e expandida para oferecer um conjunto maior de operações, de modo a atender às necessidades da arquitetura proposta. Esses aprimoramentos incluem a reorganização interna da ULA, a integração com sistemas de controle e a incorporação de novas operações que permitam maior flexibilidade e capacidade de processamento.

O projeto também busca aprofundar a compreensão do funcionamento lógico por trás dos circuitos de uma CPU, explorando técnicas de organização, eficiência e clareza estrutural. Aspectos como geração e interpretação de flags, manipulação de carry e overflow, e o sequenciamento de instruções serão analisados para garantir uma implementação coerente e expansível.

Com isso, pretende-se construir uma CPU funcional, simples, mas organizada de maneira a permitir futuras expansões, seja para aumento do conjunto de instruções, melhoria da ULA ou evolução da arquitetura como um todo.

## GLOSSÁRIO

Este trabalho possui um glossário dos termos que serão citados durante o desenvolvimento. Esses serão citados durante o desenvolvimento, então se espera o conhecimento prévio desses.

Dito isso, muitos significados de uma vez, podem ser excessivos, por isso, em caso do pouco conhecimento sobre circuitos digitais, recomenda-se entender os tópicos: [Valores Digitais](#), [Termos Estruturais / Conceituais](#) e [Termos Lógicos / Matemáticos Gerais](#).

Em seguida para: [Glossário dos Fundamentos](#), em [FUNDAMENTOS](#).

Para a tratar com valores: [Cálculos e Conversões](#), e continuar a leitura a partir do tópico: [Unidade Lógica Aritmética \(ULA\)](#).

### Valores Digitais

**Representação Digital:** Em circuitos digitais, as informações são representadas por dois estados possíveis, denominados níveis lógicos: 0 — nível lógico baixo e 1 — nível lógico alto. Estes estados correspondem a níveis de tensão, definidas pela tecnologia do circuito. Em suma, 1 representa a passagem de energia, enquanto 0, não.

**Junção dos Bits:** Se refere à junção de bits, combinando seus valores para permitir valores maiores, onde, os mais à direita têm pesos menores do que os da esquerda, peso é a quantidade de valor, que aquele bit vale, de acordo com seus deslocamento na linha. Por exemplo, sempre da **direita** para a **esquerda**, o primeiro tem o peso mais leve, enquanto, mais distante do primeiro, maior seu peso. Essa regra é a mais usada, e permite diferentes interpretações dos diferentes sistemas para leitura e interpretação de bits.

**Agrupamento de Bits:** Agrupamento de Bits, é o ato de agrupar os bits, eles também aumentam os valores dos bits de acordo com o deslocamento, porém, permitem uma melhor leitura dos dados, ao menos para o humano. Muitos desses, recebem “apelidos”, 1 bit e 2 bits, não têm nomes próprios, mas 4 bits formam um Nibble, e 8 bits um Byte.

**Sistema binário:** Esse é o sistema mais conhecido para a representação de números em ambientes digitais. Ele utiliza apenas os dígitos 0 e 1, que representam valores lógicos distintos. A interpretação de um número binário depende da quantidade de bits no agrupamento, pois cada posição possui um **peso**, determinado pela potência de 2 correspondente à sua posição.

Os pesos são atribuídos da direita para a esquerda, iniciando em  $2^0$ . Assim, a primeira casa vale 1, a segunda vale 2, a terceira 4, a quarta 8, e assim por diante.

É importante destacar que, ao apresentar valores numéricos em qualquer base — seja em textos, diagramas, imagens ou circuitos — deve-se indicar o sufixo da base, principalmente quando pode haver ambiguidades. Exemplos:

- $1011_{10}$  (= mil e onze em decimal);
- $1011_2$  (= 11 em binário);
- $13_8$  (= 11 em octal);
- $B_{16}$  (= 11 em hexadecimal).

**Pesos em Bits:** O peso é o valor atribuído a cada bit de acordo com sua posição dentro de um número. Em sistemas de numeração posicional, cada casa possui um valor proporcional à sua base elevada ao índice da posição.

No sistema binário, os pesos crescem de acordo com as potências de 2, sempre da direita para a esquerda. Em agrupamentos de quatro bits, por exemplo, os pesos são **8, 4, 2 e 1**. Assim, cada bit contribui de forma diferente para o valor total do número, dependendo de sua posição.

Esse conceito será melhor explicado na seção “**1.1 — Cálculos e Conversões com dígitos binários**”, onde os pesos serão aplicados em operações práticas.

**Sistema hexadecimal:** O sistema hexadecimal utiliza 16 símbolos para representar valores numéricos:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E e F.

Os símbolos de A a F correspondem aos valores decimais de 10 a 15.

Este sistema tem uma correspondência direta com o sistema binário:

- 4 bits formam um nibble,
- e cada nibble equivale exatamente a um dígito hexadecimal.

Isso torna o hexadecimal extremamente útil para leitura, depuração e documentação de circuitos digitais, reduz longas sequências binárias a representações mais compactas e legíveis. Inclusive, é uma famosa forma de representar cores em computadores, como “FF0000”, que, em RGB: 255, 0, 0 — equivalente ao vermelho.

Exemplos de uso:

- $1011_2 = B_{16} = 11_{10}$
- $1111_2 = F_{16} = 15_{10}$
- $1100\ 1010\ 1111\ 1110_2 = \text{CAFE}_{16} = 51966_{10}$

Assim como no binário, recomenda-se sempre indicar explicitamente a base — usando subscrito ou sufixo — para evitar interpretações ambíguas em diagramas e textos técnicos.



## Termos Estruturais / Conceituais

**Bit (Binary Digit):** Refere-se à menor unidade de informação em sistemas digitais. Capaz de assumir dois valores: 0 ou 1, que representam estados lógicos de um circuito digital. Importante citar que — bits são diferentes de valores — quando falamos na fórmula binária de ver os bits, eles sempre estarão representados por 0 e 1, porém, o valor que eles representam, variam de acordo com suas combinações, um único bit, pode assumir os valores de 0 e 1.

**Nibble (4 bits):** Conjunto formado por quatro bits agrupados, que também equivale a um dígito em hexadecimal. É utilizado para representar valores, que podem variar entre 0 à 15. Os circuitos desenvolvidos, serão para montar uma ULA de 4 bits, e em diversos momentos será possível visualizar sequências de quatros 0 ou 1.

**Byte (8 bits):** Conjunto formado por oito bits agrupados, equivalente à 2 nibbles ou 2 dígitos hexadecimais. É utilizado para representar valores, que podem variar entre 0 e 255, dependendo das combinações realizadas

**MSB (Most Significant Bit):** Refere-se ao bit mais significativo dentro de uma sequência binária. É o bit de maior peso, normalmente localizado à esquerda do agrupamento. Em representações binárias, o MSB é o responsável por indicar a maior parcela do valor numérico daquele conjunto de bits.

**LSB (Least Significant Bit):** É o bit menos significativo da sequência binária, localizado na posição mais à direita do agrupamento. Possui o menor peso dentro da representação numérica. O LSB é frequentemente utilizado em operações como incremento, verificação de paridade e manipulações de menor granularidade em circuitos digitais.

**Input (Entrada):** Corresponde a qualquer valor lógico, dado ou sinal fornecido a um circuito digital. As entradas definem as condições iniciais que permitirão ao circuito produzir um resultado. Em diagramas digitais, são representadas como pinos ou portas que recebem sinais vindos de outros componentes. Durante o desenvolvimento, ele serão apresentados de diversas formas, os pinos de entrada podem aparecer como “A”, “B” e outras, e serão explicados quando mostrados.

**Output (Saída):** É o resultado gerado por um circuito digital após o processamento de suas entradas. Representa o valor lógico final produzido pela operação realizada. As saídas podem ser visualizadas como pinos de saída, LEDs, displays ou ligações com outros módulos. Assim como as entradas possuem suas representações em letras, as saídas podem ser apresentadas como “Q” vindo de “Quit” (Sair) ou como “S” em alguns momentos, vindo de “Saída”.

**Barramento:** É um conjunto organizado de múltiplas linhas de sinal que transportam vários bits simultaneamente. Um barramento permite o tráfego paralelo de dados entre diferentes partes do circuito, facilitando a conexão de somadores,

multiplexadores, entre outros dispositivos. Em diagramas, é representado como uma linha mais espessa indicando múltiplos bits agrupados.

Em muitos dos circuitos desenvolvidos os barramentos serão vistos como um único pino de input, os quais, podem transportar diferentes tamanhos de bits, desde apenas um único bit, até um nibble. Caso necessário, os bits desses nibbles, serão divididos, usando dispositivos oferecidos pelas ferramentas, os quais permitem dividir um input de 4 bits, em 4 fios de 1 bit cada, mantendo seus valores sincronizados com a entrada. Essa escolha, é puramente estética e, é uma forma de mostrar o que a ferramenta utilizada pode fazer, mas ainda sim, serão mantidos os modelos recomendados, para melhor didática e entendimento.

**Sinal lógico / Nível lógico (0 e 1):** É a representação elétrica do estado de um circuito digital. O nível lógico 0 (ou baixo) e o nível lógico 1 (ou alto) correspondem a faixas específicas de tensão dependendo da tecnologia do circuito. Em ferramentas, esses valores são tratados apenas como 0 e 1, determinando o comportamento das portas lógicas e dos componentes digitais.

**Propagação de sinal:** Refere-se ao percurso que um valor lógico realiza ao atravessar um circuito, passando por fios, portas lógicas e componentes até atingir a saída. A propagação está relacionada ao tempo que o circuito leva para transferir e processar um sinal, influenciando o comportamento temporal e a estabilidade das operações digitais.

**Delay:** É o intervalo de tempo necessário para que uma alteração em uma entrada produza o efeito correspondente nas saídas do circuito. O delay ocorre devido às características físicas das portas lógicas, fios e elementos internos. Em simulações digitais simplificadas, pode ser visualizado quando uma mudança de valor não é refletida instantaneamente no resultado, que podem ou não serem percebidas, dependendo de diversos fatores de desempenho.

## **Termos Lógicos / Matemáticos Gerais**

### **Expressão Booleana / Expressão lógica**

É uma forma simbólica de representar o comportamento de um circuito digital usando variáveis (A, B, X...), operadores lógicos (AND, OR, NOT...) e constantes (0 e 1).

Uma expressão booleana descreve exatamente como uma saída depende das entradas. Por exemplo, a expressão:  **$X = A \text{ AND } B$** , representa um circuito onde a saída **X** só é 1 quando **A** e **B** são 1. Expressões booleanas servem como “linguagem matemática” para modelar circuitos lógicos, por isso, podem ter diferentes notações em suas representações.

### **Função lógica**

É a relação completa entre entradas e saídas de um circuito digital, podendo ser descrita por uma expressão booleana, tabela verdade ou circuitos esquemáticos. Uma função lógica determina qual saída é produzida para cada combinação possível de entradas. Exemplo são a função AND, a função OR, função XOR, etc. A “função” é o comportamento, não o circuito em si.

### Operação lógica

É a ação individual realizada por portas lógicas básicas, como AND, OR, NOT, NAND, NOR, XOR, XNOR. Cada operação define regras específicas de como combinar bits de entrada para produzir um bit de saída. Operações lógicas são os “blocos fundamentais” usados para construir expressões booleanas e funções lógicas.

### Tabela Verdade

É uma tabela que enumera todas as combinações possíveis das entradas e mostra o valor correspondente da saída. Muitas das vezes, as combinações podem gerar tabelas gigantes — como multiplexadores —, então é possível apresentar as tabelas em um resumo funcional de suas funcionalidades, essas geralmente têm “i” que indica para cada bit, além de outros indicadores que serão apresentados como guia.

É essencial para analisar ou validar o comportamento de uma função lógica. Por exemplo, para duas entradas (A e B), a tabela verdade terá 4 linhas (00, 01, 10, 11).

### LSB (Least Significant Bit)

É o **bit menos significativo** de uma sequência binária, é o bit mais à direita. Ele representa o **menor peso** na representação do número ( $2^0 = 1$ ). Ele é importante para: operações de soma, manipulação de deslocamentos (shifts) e interpretação de valores binários.

### MSB (Most Significant Bit)

Refere-se ao **bit mais significativo** dentro de uma sequência binária, é o bit mais à esquerda. Representa o **maior peso** (maior potência de 2). Ele determina a maior parcela do valor numérico representado e, em sistemas com sinal, pode ser usado como o bit que indica se o número é positivo ou negativo.

### MinTermos (MinTerm)

São expressões que representam **uma única combinação específica** das entradas que torna a função igual a **1**. Cada mintermo corresponde a uma linha da tabela verdade onde a saída é verdadeira.

Esse método faz parte da **forma canônica Soma-de-Produtos (SOP)**, derivada dos estudos de **George Boole**, e sistematizada posteriormente no trabalho de **Claude Shannon**, que aplicou álgebra booleana diretamente a circuitos elétricos.

## MaxTermos (MaxTerm)

Também são expressões que representam **uma única combinação específica** das entradas, porém, que torna a função igual a **0**. Cada maxtermo corresponde a uma linha da tabela verdade onde a saída é falsa.

O método segue a lógica complementar aos mintermos, integrante da **forma canônica Produto-de-Somas (POS)**, também fundamentada nos trabalhos iniciais de Boole e posteriormente organizada e aplicada à engenharia por **Shannon**.

## Simplificação lógica

É o processo de reduzir uma expressão, função ou circuito lógico para uma forma equivalente **com menos termos, menos portas ou menor custo**. Muito utilizadas em MinTermos ou MaxTermos, utilizando técnicas como:

- Álgebra Booleana
- Mapa de Karnaugh

O objetivo é tornar o circuito **mais eficiente** — principalmente no mundo físico — mas mantendo o mesmo comportamento lógico.

## Equivalência lógica

Duas expressões são logicamente equivalentes quando **produzem a mesma saída para todas as combinações de entrada**, mesmo que tenham formas diferentes.

Serve para validar simplificação, substituição de portas e otimizações. Por exemplo:

$$A \text{ OR } ( A \text{ AND } B )$$

que é equivalente à apenas **A**.

# 1 — FUNDAMENTOS

## 1.1 — Glossário dos Fundamentos

### 1.1.1 — Termos de Circuitos

#### Circuito combinacional

É um tipo de circuito digital cuja saída depende **exclusivamente dos valores atuais das entradas**, sem qualquer forma de armazenamento interno. Exemplos comuns incluem somadores, decodificadores, multiplexadores e portas lógicas. Sua operação é inteiramente determinada por expressões booleanas, portanto, para um mesmo conjunto de entradas, o circuito sempre produz a mesma saída.

#### Circuito sequencial

Um circuito sequencial possui **memória interna**, o que significa que sua saída depende tanto das entradas atuais quanto do estado anterior, geralmente esses também dependem de clock. Esse estado é armazenado em elementos como flip-flops, shift registers (shifters) e registradores.

#### Hierarquia de circuitos

É a organização dos componentes digitais em diferentes **níveis de abstração**, possibilitando compreender e projetar sistemas complexos de forma estruturada. Em uma hierarquia, circuitos simples formam módulos intermediários, que por sua vez compõem sistemas maiores.

#### Bloco macro (Macro-Block ou apenas Macro)

Refere-se a um agrupamento funcional que **encapsula** um circuito mais complexo em um bloco simplificado. Um macro-block é uma forma de representação abstrata, permitindo que circuitos grandes sejam tratados como unidades únicas — como uma ULA, um multiplexador expandido ou qualquer função construída a partir de subcircuitos menores. Esses blocos possuem uma aparência característica, adotada como um consenso para a representação gráfica de macros em circuitos digitais.

#### Modularidade de circuitos

Conceito que envolve dividir um circuito em partes independentes e reutilizáveis, chamadas **módulos**. Cada módulo executa uma função específica e pode ser projetado, testado e modificado de forma isolada.

#### Componente / Subcircuito

Um subcircuito é um circuito menor encapsulado que realiza uma função específica e pode ser reutilizado dentro de um projeto maior. São peças menores, que tem uma função unitária.

## Componentes, Módulos e Macros

Componentes são blocos básicos do circuito, como portas lógicas, somadores de 1 bit, multiplexadores, displays, clocks e outros.

Já os módulos, são blocos funcionais formados por componentes conectados — como somador de 4 bits — que pode ser feito utilizando um método de conexão de 4 somadores de 1 bit, um dos mais conhecidos, é o método do **Ripple Carry**.

Os macros, são representações gráficas dos circuitos, mas que também servem como uma forma compactada e encapsulada de módulos e componentes, como somadores de 4 bits, para ser utilizado em uma ULA e nomeado como “Adder4”.

## Tri-States, Buffers e Inversores

Entre os diversos componentes utilizados na construção de módulos lógicos mais complexos, os circuitos Tri-State desempenham um papel fundamental no controle do fluxo de dados. Diferente de uma porta lógica comum — que só pode apresentar nível lógico alto ou baixo — o Tri-State possui um terceiro estado possível: o estado de alta impedância, também conhecido como **High-Z**. Nesse estado, o componente se comporta como se estivesse “desconectado” da linha, não interferindo nem puxando o sinal para nenhum dos níveis lógicos.

A partir desse princípio, surgem dois usos muito comuns: **Buffers** e **Inverters** com controle de habilitação.

**Buffers Tri-State** permitem que apenas um módulo por vez coloque dados em um barramento compartilhado. Quando habilitado, o buffer replica o valor de entrada para a saída. Quando desabilitado, coloca sua saída em High-Z, deixando o barramento livre. Isso evita conflitos elétricos — por exemplo, dois módulos tentando forçar valores opostos ao mesmo tempo — e é essencial em arquiteturas de CPU que dependem de barramentos para movimentar dados entre registradores, memória e unidades funcionais.

**Inversores Tri-State** funcionam da mesma forma, mas aplicam uma inversão lógica ao sinal quando habilitados. Ao serem combinados a outros módulos, permitem que a mesma linha transporte versões diferentes de um dado (direta ou invertida), dependendo do controle da CPU, e também entram em High-Z quando não estão ativos.

O estado **High-Z (alta impedância)** é o mecanismo que permite a coexistência de múltiplas fontes de dados em um único barramento físico. Ele impede que os sinais “disputem” a mesma linha, garantindo que apenas o módulo habilitado interfira no valor presente naquele instante.

Além disso, é comum que esses elementos utilizem **transistores em paralelo**, formando o chamado **Transmission Gate** (TG). Nesse arranjo, transistores complementares (geralmente um NMOS/N-Type e um PMOS/P-Type) trabalham em conjunto para permitir ou bloquear totalmente a passagem de um sinal. Esse método assegura uma condução mais eficiente e simétrica, possibilitando que o fluxo de dados seja controlado diretamente por outro sinal lógico. Dessa forma, o Transmission Gate atua como um “interruptor eletrônico” ideal, essencial para multiplexação, controle de barramentos e implementação de buffers e tri-states em níveis mais baixos de abstração.

## 1.1.2 — Termos sobre as Operações

### 1.1.2.1 — Carry-in (Cin)

Valor de entrada que representa o “vai-um” **recebido** por um somador, normalmente proveniente de uma operação anterior ou de um bit menos significativo. Ele afeta diretamente o resultado da soma, principalmente em operações em cadeia, como no ripple carry.

### 1.1.2.2 — Carry-out (Cout)

Valor de saída que indica o “vai-um” **gerado** pela operação de soma do bit mais significativo daquele bloco. Em um somador de múltiplos bits, o Cout é propagado para o próximo estágio. Em operações aritméticas, também pode representar um estouro da capacidade daquele conjunto de bits.

### 1.1.2.3 — Overflow

Condição que ocorre quando o resultado de uma operação aritmética ultrapassa o limite de representação possível para aquele número de bits, especialmente em operações com números assinados. Ocorre quando o bit de sinal é calculado incorretamente devido ao resultado exceder o intervalo permitido.

### 1.1.2.4 — Borrow

Equivalente ao “empréstimo” utilizado em subtrações binárias. É um valor que representa que o bit menos significativo precisou “pedir” 1 ao bit seguinte para realizar a operação. Assim como o carry em somadores, o borrow se propaga entre bits em operações de subtração, e também há a existência do **borrow in** (Bin) e do **borrow out** (Bout), e seguem a mesma idéia de receber o empréstimo (Bin), e o emprestar (Bout).

### 1.1.2.5 — Ripple Carry

É o método de implementação de somadores em que o carry gerado por um bit é propagado sequencialmente ao próximo. O nome “ripple” descreve essa propagação gradual, que introduz atraso cumulativo, mas não caracteriza um circuito sequencial.

Também existe o **Ripple Borrow**, para circuitos de subtração, porém, da forma em que o subtrator será construído neste projeto, ele não será visto, apenas citado.

### 1.1.2.6 — Operação aritmética e lógica

Operações aritméticas abrangem cálculos como soma, subtração, subtração, divisão, até paridade (Par ou Ímpar) e primalidade (Números Primos), enquanto as operações lógicas envolvem manipulação bit a bit, como AND, OR, XOR e NOT. Ambas compartilham a mesma estrutura geral, mas têm finalidades distintas.



### 1.1.2.7 — Modo de Operação

Refere-se à configuração que define qual operação específica a ULA deve executar, geralmente selecionada por bits de controle. Cada modo habilita caminhos internos distintos, ativando a operação correspondente (ex.: modo soma, modo AND, modo OR).

### 1.1.2.8 — Circuitos “Full” e “Half”

Designações usadas para diferenciar se o circuito considera ou não um carry/borrow externo.

- Um **Half-Adder** (meio somador) calcula a soma de dois bits, sem carry-in.
- Um **Full-Adder** (somador completo) considera o carry-in, permitindo sua utilização em cadeias.

O mesmo raciocínio vale para subtratores, porém, não será usado por conta do método utilizado para construí-lo.

### 1.1.2.9 — Flags

Sinais auxiliares emitidos pela ULA que indicam condições específicas do resultado, como negativo, carry, overflow e outros. São utilizados por sistemas maiores para decidir ações posteriores. Neste projeto, os macros terão suporte para expansão, consequentemente para sinais, eles serão tratados, explicados e demonstrados, mas não serão utilizados na CPU principal.

### **1.1.3 — Termos relacionados à Simulação**

#### **1.1.3.1 — Simulação digital**

Processo de modelar o comportamento de circuitos digitais em software, permitindo observar seu funcionamento sem necessidade de montar fisicamente o circuito.

Ferramentas como o Logisim reproduzem níveis lógicos, propagação de sinais e atrasos básicos. E o mesmo, será o software usado para desenvolver.

#### **1.1.3.2 — Wire**

Representa o fio que conecta componentes dentro do circuito. No simulador, é responsável por conduzir os valores lógicos entre entradas, saídas e blocos internos.

#### **1.1.3.3 — Pin**

Elemento de entrada ou saída usado para interagir com o circuito. Permite definir valores externos (entradas) ou observar resultados (saídas) durante a simulação.

A ferramenta utilizada permite que pins possam transmitir e receber diversos tamanhos de dados, desde apenas 1 bit, até maiores. Neste projeto, será apresentado uso de até 4 pins com 1 bit cada, até 1 pin com 4 bits, que podem ser divididos usando splitters, se necessário.

Eles não mudam a forma em que os dados são transmitidos, e permitem o uso de componentes já prontos para melhor estética e apresentação, como display hexadecimal.

#### **1.1.3.4 — Splitter**

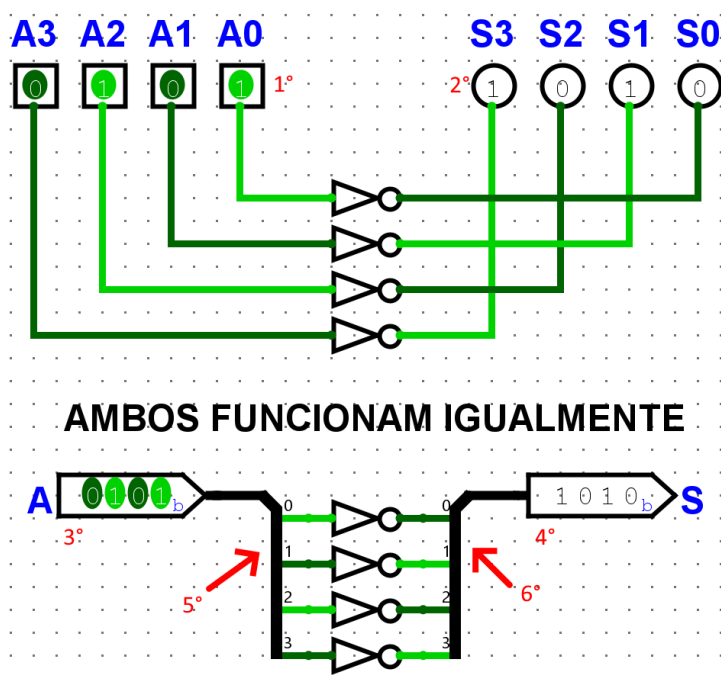
Componente que divide ou agrupa conjuntos de bits, permitindo manipular barramentos mais largos ou extrair partes específicas de uma sequência binária.

Ele consegue preservar a linha de dados de uma entrada e saída de dados, como exemplo: um pin de 4 bits de dados, consegue ser dividido em quatro linhas com um bit cada, e, em exemplo: caso apenas o quarto bit desse pin esteja em 1, apenas a quarta linha do splitter vai transmitir dados.

Para deixar mais claro, a Figura 1 apresenta a mesma lógica, de um circuito de uma porta lógica de 4 bits, construída usando quatro portas lógicas de 1 bit.

Ambos possuem o mesmo funcionamento, porém, a mais acima com 4 entradas de 1 bit precisam receber e enviar quatro linhas de dados diferentes, enquanto a mais abaixo, com apenas uma entrada de 4 bits, internamente consegue usar o splitter para dividir esses 4 bits em quatro linhas de dados diferentes, e depois, consegue fazer a junção dessas quatro linhas em uma saída de 4 bits.

Figura 1 — Exemplo de um circuito sem o uso e com o uso de pin e splitter



- 1: Pino de 1 bit de entrada com aparência clássica do Logisim;
- 2: Pino de 1 bit de saída com aparência clássica do Logisim;
- 3: Pino de 4 bits de entrada com aparência "Arrow";
- 4: Pino de 4 bits de saída com aparência "Arrow";
- 5: Splitter que consegue receber dados pela porta mais inclinada para esquerda, e dividir para outras quatro portas na parte vertical reta;
- 6: Splitter que consegue receber dados pelas portas na parte vertical, e juntá-los para uma única saída na porta mais inclinada para direita.

Percebe-se que, os splitters, no programa usado, não possuem diferenciadores muito visuais, além da direção e aparência, como "Right-Handed, East" e "Left-Handed, West". Eles buscam respeitar os seus fios conectores, ele tenta "entender" de qual direção vem os dados, como pin de entrada e saída.

### 1.1.3.5 — Display Hex

Elemento visual que exibe um valor binário como um dígito ou grupo de dígitos em hexadecimal. Facilita a leitura de resultados em operações de diferentes bits.

### 1.1.3.6 — Clock

Sinal periódico usado para sincronizar circuitos sequenciais. Marca os instantes em que as operações devem mudar de estado. Em simuladores, pode ser configurado para pulsar automaticamente ou manualmente.

Para o projeto da ULA, esse não se mostra necessário, porém, será apresentado em outro trabalho.

### 1.1.3.7 — Label

Texto ou nome usado para identificar fios, blocos, entradas e saídas. Serve exclusivamente para organização, leitura e documentação do circuito.

### 1.1.3.8 — Tunnel

Recurso que permite conectar partes do circuito sem utilizar fios longos. Dois túneis com o mesmo nome funcionam como se estivessem diretamente ligados, facilitando a organização visual.

Eles serão utilizados apenas em momentos para representar algo de forma mais clara, pois, o seu uso intenso, apesar de funcional para o programa, pode causar confusão para o entendimento dos circuitos.

### 1.1.3.9 — Constantes, Power e Ground

O Logisim disponibiliza componentes de constante lógica (**Constant**), que funcionam como pinos de saída fixa, sempre apresentando o valor definido pelo usuário — normalmente 0 ou 1. Uma constante configurada como 0 tem o mesmo comportamento funcional do componente **Ground**, enquanto uma constante configurada como 1 equivale ao componente **Power**. Na prática, todos esses elementos fornecem níveis lógicos estáveis para o circuito, diferindo apenas na forma de representação visual oferecida pela ferramenta.

## 1.2 — Cálculos e Conversões

### 1.2.1 — Dígitos binários

Para determinar o valor representado por uma sequência binária, utiliza-se uma abordagem baseada nos pesos de cada posição. Essa técnica, bastante adequada para cálculos manuais, consiste em multiplicar cada dígito (bit) pelo peso correspondente à sua posição.

Como exemplo o número  $1011_2$ , temos:

$$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$$

Nessa representação, o multiplicando é o dígito binário (0 ou 1), enquanto o multiplicador corresponde ao peso daquela casa, baseado nas potências de 2. A soma de todos os produtos resulta no valor decimal equivalente.

### 1.2.2 — Agrupamento em Nibbles

Para números binários maiores, é possível facilitar os cálculos utilizando agrupamentos de 4 bits (nibbles).

Por exemplo, a sequência  $1011\ 1011_2$  pode ser dividida em dois grupos:

- primeiro nibble:  $1011_2$ ;
- segundo nibble:  $1011_2$

$$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$$

No entanto, como cada nibble à esquerda representa valores 16 vezes maiores (pois ocupa as posições de  $2^4$  até  $2^7$ ), basta multiplicar o resultado do nibble mais significativo por 16:

$$16 \times 11 = 176$$

Somando com o valor do nibble menos significativo:

$$176 + 11 = 187$$

Ou seja:

$$1011\ 1011_2 = 187_{10}$$

### 1.2.3 — Por que multiplicar por 16

Um nibble possui valores possíveis de 0 a 15.

O próximo valor, 16, só pode ser representado com 5 bits — ou seja, um novo nibble inicia. Esse limite é o que chamamos, de forma informal, de “**teto + 1**”:

$$15 + 1 = 16$$

Por isso, cada nibble à esquerda desloca o valor em potências de 16, exatamente da mesma forma que, no sistema decimal, um dígito à esquerda representa dezenas, centenas, milhares etc.

Utilizando o número  $0001\ 1111\ 1111_2$ , temos três nibbles:

- $0001_2 = 1$
- $1111_2 = 15$
- $1111_2 = 15$

Os pesos para cada nibble são:

- $16^2 = 256$  para o primeiro
- $16^1 = 16$  para o segundo
- $16^0 = 1$  para o terceiro

Logo:

$$(256 \times 1) + (16 \times 15) + (1 \times 15) = 511$$

Essa simplificação, evita escrever uma soma extensa, como esta>

$$0001\ 1111\ 1111_2 = (0 \times 2^{11}) + (0 \times 2^{10}) + \dots + (1 \times 2^1) + (1 \times 2^0) = 511_{10}$$

#### 1.2.4 — Conversões no Sistema Hexadecimal

O processo de conversão no sistema hexadecimal segue a mesma lógica, porém utilizando potências de 16, e não de 2. Um número como  $FF_{16}$  representa:

$$F = 15$$

$$(16 \times 15) + 15 = 255$$

É comum agrupar representações hexadecimais em pares de dígitos, pois dois dígitos hexadecimais equivalem a um byte (8 bits).

Também é prática recomendada deixar explícita a base utilizada, utilizando subscritos ou sufixos como:

- $FF_{16}$
- $0xFF$
- $FFh$

### 1.3 — Unidade Lógica Aritmética (ULA)

A Unidade Lógica e Aritmética (ULA) é um dos principais componentes de uma Unidade Central de Processamento (CPU), responsável por executar operações aritméticas e lógicas de forma direta e controlada. Dentro de um processador, a ULA atua como o bloco funcional onde cálculos são efetuados e decisões lógicas são tomadas, servindo como o ponto central de manipulação dos dados.

Em seu funcionamento, a ULA opera seguindo uma ordem definida de operações, que depende da forma como seu circuito interno é estruturado e do sinal de controle recebido. Essa ordem que a ALU executa as operações, depende da forma em que foi projetada, geralmente, essa ordem busca seguir um padrão, indo pela ordem alfabética dos nomes das operações em inglês, neste caso, a ordem:

- **Primeira operação:** Adder (Somador);
- **Segunda operação:** Divider (Divisor);
- **Terceira operação:** Multiplier (Multiplicador);
- **Quarta operação:** Nand (Não E);
- **Quinta operação:** Nor (Não Ou);
- **Sexta operação:** Subtractor (Subtrator);
- **Sétima operação:** Xnor (Não Ou Exclusivo);
- **Oitava operação:** Xor (Ou Exclusivo).

A ULA construída possui duas entradas, InputA e InputB (ou simplesmente A e B), ambas com a mesma largura de bits da ULA. Nas operações aritméticas, cada entrada pode assumir valores entre 0 e 15. O resultado exibido também fica limitado a esse intervalo. Na subtração, o circuito não suporta situações em que B é maior que A, pois não há tratamento para resultados negativos nem indicação de sinal.

Cada operação é implementada como um circuito específico, e a integração desses circuitos é feita por meio de multiplexadores, que selecionam qual resultado será disponibilizado ao restante da CPU.

Essa organização modular permite que a ULA seja expandida com novas operações, otimizando o desempenho e possibilitando sua adaptação a diferentes arquiteturas de processadores, desde sistemas simples até unidades mais complexas.

## 1.4 — Fundamentos dos Circuitos Lógicos Combinacionais Digital

### 1.4.1 — Circuito Integrados (CIs)

Em aplicações reais, as portas lógicas não existem isoladamente como representações abstratas, mas sim integradas em circuitos integrados (CIs) específicos. Cada CI possui um conjunto definido de portas, número de entradas, características elétricas e limitações próprias. Essas informações variam conforme o fabricante e o modelo do componente.

Esses circuitos são identificados por códigos padronizados — como a família **74HC**, **74LS**, **CD4000**, entre outras — que determinam tanto o tipo de porta contida quanto suas propriedades tecnológicas. Por exemplo, o CI **MM74HC32M** contém quatro portas OR de duas entradas.

O funcionamento elétrico, o número de portas disponíveis e a pinagem são especificados em seus respectivos *datasheets*, fornecidos pelos fabricantes. Esses *datasheets* são a fonte oficial de consulta e descrevem características como tensões de operação, tempos de comutação, diagramas internos e tabelas verdade.

*Datasheets* disponíveis em: <https://www.alldatasheet.com>

### 1.4.2 — Portas lógicas

Os circuitos lógicos combinacionais são a base da computação digital, sendo responsáveis por produzir saídas que dependem exclusivamente das entradas atuais, sem armazenamento de estado.

Esses circuitos são formados pela combinação de portas lógicas fundamentais, que realizam operações elementares sobre sinais binários.

Em simulações digitais e em expressões simplificadas, uma porta lógica pode ter inúmeras entradas, mas, fisicamente elas são limitadas à forma com que são apresentadas. É comum ver portas lógicas de quatro entradas quando — por exemplo: a uma representação de um MinTermo — pois se trata de um circuito digital, mas, com componentes físicos, é raro encontrar, sendo necessário fazer conexões entre portas iguais, para resultar em uma saída.



## Buffer

Embora o buffer não seja classificado como uma porta lógica fundamental, ele é amplamente utilizado em circuitos digitais, incluindo ULAs. Mas, seu principal uso está em circuitos físicos, onde há perda de força na carga elétrica, e outras funções do, como por exemplo:

- **reforço** (aumenta a capacidade de corrente para dirigir cargas maiores);
- **direcionar** energia para direções e específicas;
- **isolamento** entre estágios do circuito;
- **redução de ruído**;
- **proteção** contra interferências;
- e **controle da direção de sinais**, no caso de buffers *tri-state*.

Digitalmente, o buffer é um dispositivo cuja função básica é **replicar o sinal de entrada na saída**, sem alterá-lo. Em termos lógicos: **Saída = Entrada**.

Figura 2 — Símbolo da porta Buffer



Tabela 1 — Variações do buffer com entradas A (1 entrada)

| A | S = A |
|---|-------|
| 0 | 0     |
| 1 | 1     |

## Porta NOT

A porta NOT realiza a operação de negação lógica. Ela possui apenas uma entrada e inverte seu valor: 0 torna-se 1, e 1 torna-se 0. É fundamental para operações de complemento e geração de sinais inversos.

Essa porta também é conhecida como **inversor**, nome comum em circuitos integrados e documentação técnica. Em muitos CIs, o termo “inverter” é usado diretamente para se referir ao componente que realiza a operação NOT, já que sua função principal é exatamente inverter o nível lógico da entrada.

Figura 3 — Símbolo da porta lógica NOT

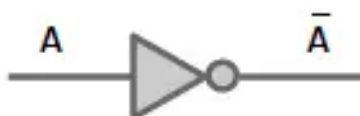


Tabela 2 — Variações da porta lógica NOT com entradas A (1 entrada)

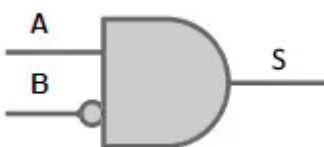
| A | $\bar{A}$ |
|---|-----------|
| 0 | 1         |
| 1 | 0         |

Em raros casos, NOTs podem ser incluídas antes da conexão de uma entrada em portas lógicas.

Na Figura 4, é um exemplo de uma portá AND, porém, na entrada B, uma negação em seu sinal. Isso é o equivalente a inverter o sinal usando uma NOT, antes de conectar na entrada de portas lógicas.

Apesar de poder ser encontrado em diagramas, eles podem causar confusão para leitura, pois em em circuitos reais, são usados em casos onde, em um módulo só há a inversão de tal sinal, apenas uma vez, assim economizando na construção. Mas, digitalmente não faz tanta diferença, por conta disso, será evitado durante o projeto.

Figura 4 — Símbolo da porta lógica AND com uma NOT antes da conexão



## Porta AND

A porta AND (conjunção lógica) realiza uma operação em que a saída é igual a 1 somente quando **todas** as entradas também forem 1. Em qualquer outro caso, a saída será 0.

Usada quando é necessário garantir que duas condições ocorram simultaneamente.

Figura 5 — Símbolo da porta lógica AND

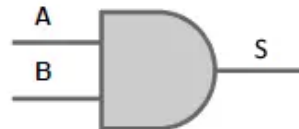


Tabela 3 — Variações da porta lógica AND com entradas A e B (2 entradas)

| A | B | $S = A \cdot B$ |
|---|---|-----------------|
| 0 | 0 | 0               |
| 0 | 1 | 0               |
| 1 | 0 | 0               |
| 1 | 1 | 1               |

## Porta OR

A porta OR (disjunção lógica) produz saída igual a 1 sempre que **ao menos uma** das entradas for 1. A saída será 0 apenas quando todas as entradas forem 0.

Usada para representar condições alternativas.

Figura 6 — Símbolo da porta lógica OR

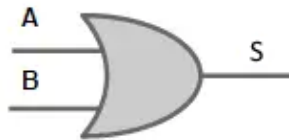


Tabela 4 — Variações da porta lógica OR com entradas A e B (2 entradas)

| A | B | $S = A + B$ |
|---|---|-------------|
| 0 | 0 | 0           |
| 0 | 1 | 1           |
| 1 | 0 | 1           |
| 1 | 1 | 1           |

## Porta XOR

A porta XOR (OU exclusivo) produz saída igual a 1 quando **as entradas são diferentes** entre si.

Pode ser referenciada como uma “**detectora de diferenças**”, onde, caso as entradas sejam diferentes, a saída é 1.

É amplamente usada em circuitos aritméticos, especialmente em somadores, por sua capacidade de detectar diferenças entre bits.

Figura 7 — Símbolo da porta lógica XOR



Tabela 5 — Variações da porta lógica XOR com entradas A e B (2 entradas)

| A | B | $S = A \oplus B$ |
|---|---|------------------|
| 0 | 0 | 0                |
| 0 | 1 | 1                |
| 1 | 0 | 1                |
| 1 | 1 | 0                |

## Porta NAND

A porta NAND é a negação da operação AND. Assim, a saída é 0 somente quando todas as entradas forem 1; em qualquer outra combinação, a saída será 1.

Muito conhecido por ser uma **porta universal**, permitindo construir qualquer circuito lógico apenas com NANDs.

Figura 8 — Símbolo da porta lógica NAND

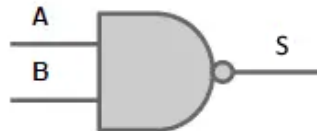


Tabela 6 — Variações da porta lógica NAND com entradas A e B (2 entradas)

| A | B | $S = \overline{A \cdot B}$ |
|---|---|----------------------------|
| 0 | 0 | 1                          |
| 0 | 1 | 1                          |
| 1 | 0 | 1                          |
| 1 | 1 | 0                          |

## Porta NOR

A porta NOR é a negação da operação OR. A saída será 1 apenas quando todas as entradas forem 0; nos demais casos, será 0.

Assim como a NAND, também é considerada uma porta universal, permitindo a implementação de qualquer outro tipo de porta.

Figura 9 — Símbolo da porta lógica NOR

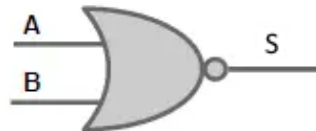


Tabela 7 — Variações da porta lógica NOR com entradas A e B (2 entradas)

| A | B | $S = \overline{A + B}$ |
|---|---|------------------------|
| 0 | 0 | 1                      |
| 0 | 1 | 0                      |
| 1 | 0 | 0                      |
| 1 | 1 | 0                      |

## Porta XNOR

A porta XNOR (OU exclusivo negado) é a negação da XOR. Sua saída é 1 quando **as entradas são iguais**.

Pode ser referenciada como uma “**detectora de igualdade**”, onde, caso as entradas sejam iguais, a saída é 1.

É útil em comparadores de igualdade e em determinadas técnicas de verificação de paridade.

Figura 10 — Símbolo da porta lógica XNOR



Tabela 8 — Variações da porta lógica XNOR com entradas A e B (2 entradas)

| A | B | $S = \overline{A \oplus B}$ |
|---|---|-----------------------------|
| 0 | 0 | 0                           |
| 0 | 1 | 1                           |
| 1 | 0 | 1                           |
| 1 | 1 | 1                           |



### 1.4.3 — Resumo dos símbolos em expressões

Tabela 9 — Diferentes representações dos símbolos das portas lógicas

| Operação | Matemático        | Álgebra                                       |
|----------|-------------------|-----------------------------------------------|
| Buffer   | $A$               | $S = A$                                       |
| NOT      | $\neg A$          | $\bar{A}$ <b>ou</b> $A'$                      |
| AND      | $\wedge$          | $A \cdot B$ <b>ou</b> $A . B$ <b>ou</b> $AB$  |
| OR       | $\vee$            | $A + B$                                       |
| XOR      | $\oplus$          | $A \oplus B$                                  |
| NAND     | $\uparrow$        | $\overline{A \cdot B}$                        |
| NOR      | $\downarrow$      | $\overline{A + B}$                            |
| XNOR     | $\leftrightarrow$ | $\overline{A \oplus B}$ <b>ou</b> $A \odot B$ |

Todas as imagens dos símbolo lógicos foram retiradas de:

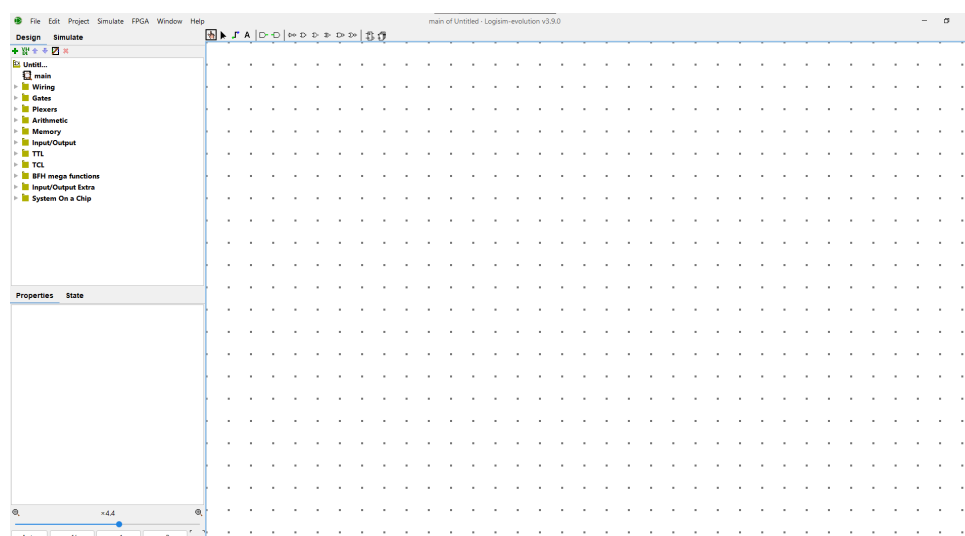
<https://embarcados.com.br/portas-logicas>

Elas foram recortadas, e outras, editadas trocando os símbolos de saída para S para melhor didática. A porta lógica XNOR foi criada com base na porta XOR existente, e a representação do Buffer também foi criada com base na porta NOT.

## 1.5 — Ferramenta de Simulação: Logisim

O Logisim-Evolution é um software open source para simulação de circuitos digitais, desenvolvido por colaboradores independentes e escrito em Java, o que garante compatibilidade com diferentes sistemas operacionais. Ele permite criar e testar circuitos de vários níveis de complexidade, desde portas lógicas básicas até ULAs e arquiteturas completas de processadores.

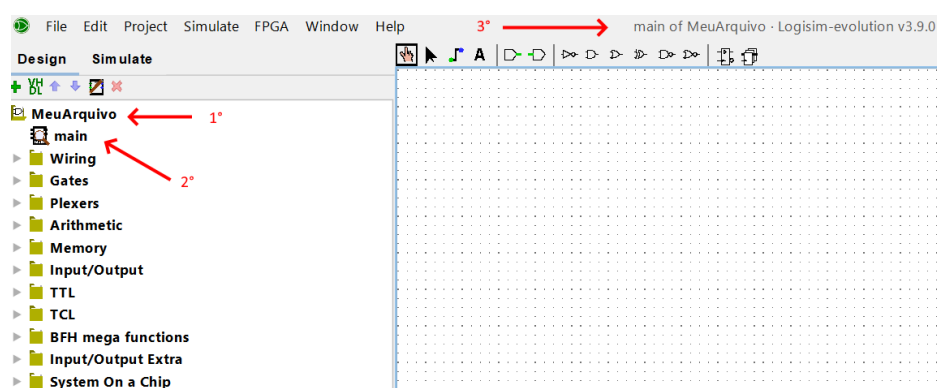
Figura 11 — Interface do Logisim-Evolution na versão v3.9.0



O programa já inclui diversas bibliotecas internas com portas lógicas, componentes aritméticos, multiplexadores, decodificadores e memórias. Esses elementos podem ter sua largura de bits ajustada nas configurações, permitindo que operem com vetores maiores sem a necessidade de replicar várias portas individuais de 1 bit, o que facilita a construção de circuitos modulares.

Cada projeto é salvo como um arquivo “.circ”, que contém tanto o circuito principal quanto seus subcircuitos. Esse formato funciona como uma biblioteca própria, podendo ser importado em outros projetos para reutilização de componentes já criados.

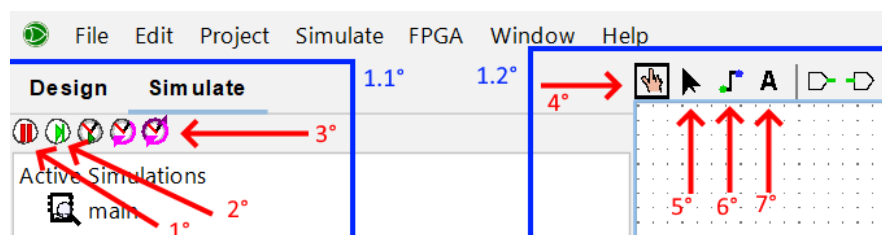
Figura 12 — Interface do Logisim-Evolution indicando o arquivo e seus circuitos



- 1º: Arquivo “.circ” aberto no programa, para editar.
- 2º: Primeiro item da lista de circuitos, presente em um único “.circ”.
  - Novos circuitos podem ser criados, indicado pelo “+” parte superior do menu, podem ser reorganizados nos utilizando os ícones de “↑” e “↓”, excluídos usando o “x”.
  - Importante citar que, no espaço pontilhado à direita, indica o circuito selecionado, mas, olhando para a lista, para saber quando um circuito está selecionado, em seu ícone na lista, uma lupa fica sobre.
- 3º: Neste ponto, é possível ver qual arquivo está selecionado de qual projeto, neste caso, arquivo “main” no projeto “MeuArquivo”.

O Logisim também trabalha em simulação contínua, mantendo o circuito sempre ativo e atualizando os resultados automaticamente conforme o usuário modifica valores ou componentes. É possível alternar entre o modo de edição e o modo de simulação para montar e testar o circuito de forma rápida e intuitiva.

Figura 13 — Interface do Logisim-Evolution indicando o menu de simulação e menu de ações rápidas



- 1.1: Menu de simulação.
- 1.2: Menu de ações rápidas (mostrando apenas as opções principais)
- 1: Botão para **parar** e **executar** a simulação.
- 2: Botão para executar um step da simulação, ou seja, quando clicado, ao invés de executar toda a lógica de uma vez, ele apenas faz uma parte.
- 3: Opções relacionadas a clock da simulação.
- 4: Troca o modo do cursor para “Modo de simulação”. Atalho: (Ctrl + 1).
- 5: Troca o modo do cursor para “Modo de edição”. Atalho: (Ctrl + 2).
- 6: Troca o modo do cursor para “Interagir com fios”. Atalho: (Ctrl + 3).
- 7: Troca o modo do cursor para “Interagir com o Labels”. Atalho: (Ctrl + 4).

A ferramenta, já consta com funções internas de análise lógica, que permitem — por exemplo — uma análise lógica do circuito, para gerar tabelas verdades, expressões booleanas e até mapas de otimização. Além de, quando necessário, o programa consegue gerar um circuito completo a partir das variáveis e tabela verdade, e até expressões booleanas. A geração automática, não será usada para o desenvolvimento, quando necessário, a função para transformar o circuito em tabela verdade e expressão booleana.

### **1.5.1 — Uso do Logisim**

Anteriormente, o CircuitMaker chegou a ser considerado para o desenvolvimento do projeto, mas acabou sendo deixado de lado, após uma análise das necessidades práticas. Optou-se pelo Logisim por oferecer um ambiente mais estável, atualizado com maior frequência e mais adequado para simulações acadêmicas.

Além disso, o Logisim inclui funcionalidades que tornam o trabalho mais simples no dia a dia, como salvamento automático, personalização de cores, uma biblioteca de componentes mais ampla, suporte a vários idiomas e uma interface mais acessível.

Outro ponto relevante é o suporte eficiente a atalhos de teclado — como Ctrl+Z, Ctrl+C e Ctrl+S — que agilizam o uso do programa. A interface mais moderna também ajuda na organização visual dos circuitos, facilitando a navegação e a compreensão do projeto.

## 2 — MÉTODO DE DESENVOLVIMENTO

### 2.1 — Escolha da organização e metodologia de construção

A organização do projeto foi pensada para facilitar entendimento, testes incrementais e futura expansão para uma CPU completa. A construção seguiu um fluxo modular: primeiro os blocos elementares, depois blocos funcionais, em seguida módulos de memória e controle, e por fim a integração dos componentes em um caminho de dados e unidade de controle que representam a CPU.

Nos circuitos e nos macros, foi priorizado a nomenclatura dos módulos em inglês, assim para seguir um padrão, ou seja, a “ULA”, aparecerá como “ALU”, “Q” serão visíveis ao invés de “S” e assim por diante.

### 2.2 — Agrupamento por categorias

- **Componentes básicos:** portas lógicas, multiplexadores de 2/4/8 operações (implementados tanto em 1 bit quanto em 4 bits), shifters elementares e flip-flops.
  - **Observação:** o AMUX (Mux de 4 bits e 2 operações) foi nomeado propositalmente por sua recorrência no projeto — será referenciado como AMUX nas demais seções por clareza.
- **Módulos funcionais:** combinando os componentes básicos para formar unidades como adder1, adder4 (ripple), shifter4, multiplicador parcial, e blocos lógicos 4-bit (NAND, NOR, XOR, XNOR).
- **Macros e encapsulamentos:** criação de macros customizados para padronizar aparência e comportamento dos módulos reutilizáveis (ex.: Adder4 macro, Register macro), mantendo visibilidade didática quando necessário e estética compacta quando integrado ao datapath.

### 2.3 — Estratégia para barramentos e Tri-States

Para ampliar o entendimento sobre os circuitos e seus usos reais, foi explorado e construído cada um dos tipos de controladores de sinais, como Buffers e Inversores Tri-States. Foi entendido o uso de Transmission Gates, construídos a partir de uma conexão em paralelo de transistores PMOS (P-Type) e NMOS (N-Type), com isso Buffers “High” e “Low” “Enable”, e o mesmo serve para os Inversores foram projetados e construídos.

### 2.4 — Módulos de memória e sincronização (clock)

A família de elementos de memória foi construída hierarquicamente: latches master-slave → flip-flop D → MAR/MBR → registradores individuais → banco de registradores → banco MIR. O projeto do clock e dos sinais de controle foi tratado com prioridade: borda/nível do clock, habilitações síncronas e condições de

setup/hold foram avaliadas conceitualmente para garantir a integridade dos dados durante transferências entre módulos, e para permitir simulação fiel no Logisim.

## **2.5 — Expansão da ULA e reutilização de estruturas**

A ULA pré-existente foi mantida como bloco base, porém foi ampliada para suportar mais operações sem duplicar lógica desnecessária: adder/subtractor (mesma estrutura com complemento de dois para subtração), lógica bitwise (NAND, NOR, XOR, XNOR) implementada tanto por portas diretas quanto reaproveitando somador/subtrator quando conveniente, e dois multiplicadores (2-bit e 4-bit) construídos por métodos distintos para explorar trade-offs de complexidade e desempenho. Uma das operações da ALU

## **2.6 — Decodificação e multiplexação de controle**

Foram implementados decodificadores 2x4 e multiplexadores 4x1/8x1 em versões 1-bit e 4-bit para acomodar sinais de controle e dados. O AMUX tem papel recorrente na seleção das operações da ULA e na roteação de fontes para o barramento principal.

## **2.7 — Decisões estéticas: fios multibit e macros customizados**

Por questões didáticas iniciais muitas instâncias foram apresentadas de forma expansível (1 bit por fio) para facilitar o acompanhamento do fluxo de bits. Contudo, ao avançar para integração da CPU foi priorizada a estética e organização: utilização de fios multibit nativos do Logisim e macros customizados feitos pela equipe para manter o circuito compacto e legível.

## **2.8 — Handlers (Tratadores) & Helpers (Ajudantes)**

Neste projeto, será apresentado um conceito nomeado como Handlers e Helpers, como o nome propõe são de uma forma ou de outra facilitadores. Os Handlers, são Macros criados quando se encontra um circuito repetido dentro de outro circuito, como em MBR's de 4 bits, que usam uma lógica que é repetida 4 vezes, por isso, essa lógica foi transformada em um macro, para deixar o circuito mais dinâmico. Os Helpers são macros criados com a intenção de prevenir possíveis erros, como um que transforma a entrada de 2 bits "11" em "00", em circuitos que só possuem três operações, para prevenir operações que não existem.

## **2.9 — Testes, validação incremental e documentação**

Cada componente foi testado isoladamente no Logisim antes de integrar ao nível seguinte. A estratégia seguiu iterações curtas: testar componentes → registrar casos de teste (vetores) → integrar → validar integração. A documentação das interfaces (entrada/saída, largura de bits, sinais de controle) foi mantida para reduzir erros durante a integração.

## **3 — DESENVOLVIMENTO DOS MÓDULOS**

### **3.1 — Módulos Básicos**

#### **3.1.1 — Explicações básicas**

##### **Decodificadores (Decoders)**

Os decodificadores são responsáveis por converter um conjunto reduzido de bits de seleção em múltiplas linhas de saída, nas quais exatamente uma é ativada por vez. Em outras palavras, eles “traduzem” um valor binário para uma única saída válida, garantindo que somente uma operação, caminho ou módulo seja habilitado naquele instante.

Esse mecanismo é essencial em unidades de controle, multiplexadores de larga escala e na escolha de operações da ULA, já que permite selecionar funções de forma simples e totalmente determinística.

##### **Multiplexadores (Mux)**

Os multiplexadores realizam a função inversa: entre diversas entradas possíveis, selecionam apenas uma e a encaminham para a saída. A escolha é feita por linhas de seleção, que determinam qual entrada está ativa em determinado momento.

Assim, permitindo que várias fontes de dados compartilhem uma única linha de saída sem conflito — um conceito indispensável para barramentos, rotações de dados e seleção de resultados dentro da CPU.

##### **Multiplexadores de 1 bit**

O multiplexador de 1 bit é a forma mais simples desse dispositivo. Ele manipula apenas um sinal por vez, selecionando entre duas, quatro ou mais entradas individuais. Todos os multiplexadores precisam de um decodificador feito para eles em quantidade de dados, ou seja, se for preciso escolher entre 2 entradas de dados, é necessário um decodificador para alternar entre as entradas.

##### **Multiplexadores de 4 bits (Mux 4b)**

Tal como outros módulos digitais modulares, os multiplexadores podem ser expandidos por replicação. Um multiplexador de 4 bits é construído conectando-se quatro multiplexadores de 1 bit em paralelo, todos compartilhando as mesmas linhas de seleção.

Cada multiplexador controla um único bit do grupo, e ao serem combinados formam um único módulo capaz de selecionar, de forma simultânea e consistente, todos os quatro bits de uma operação. O resultado é um componente multibit que mantém a

simplicidade do Mux 1b, mas com capacidade para manipular palavras inteiras — ideal para operações da ULA e para roteamento de dados entre registradores.

### 3.1.2 — Decodificadores construídos

#### Decoders 1x2

Possuí 1 bit de entrada e 2 saídas. Entrada 0 libera a saída 0 e a 1 a saída 1.

**Expressão:**

$$m0 = \bar{A}$$

$$m1 = A$$

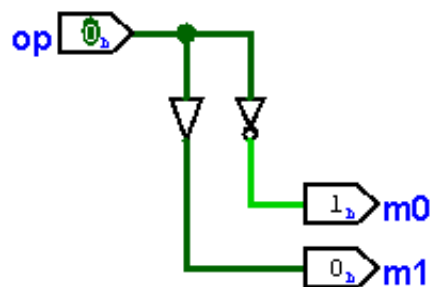
**Tabela:**

Tabela 10 — Decoders 1x2

| Op0 | m1 | m0 |
|-----|----|----|
| 0   | 0  | 1  |
| 1   | 1  | 0  |

**Circuito:**

Figura 14 — Circuito: Decoders 1x2



**Bloco Macro:**

Figura 15 — Macro: Decoders 1x2





## Decoders 2x4

Possuí 2 bit de entrada e 4 saídas. Cada valor de entrada libera uma saída diferente, sempre da saída 0 até a saída 3.

**Expressão:**

$$m0 = \bar{A} \cdot \bar{B}$$

$$m1 = A \cdot \bar{B}$$

$$m2 = \bar{A} \cdot B$$

$$m3 = A \cdot B$$

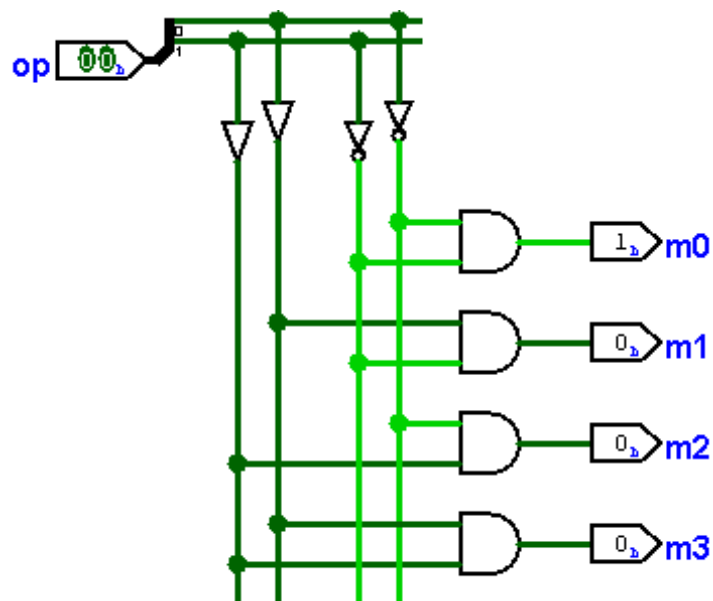
**Tabela:**

Tabela 11 — Decoders 2x4

| Op1 | Op0 | m3 | m2 | m1 | m0 |
|-----|-----|----|----|----|----|
| 0   | 0   | 0  | 0  | 0  | 1  |
| 0   | 1   | 0  | 0  | 1  | 0  |
| 1   | 0   | 0  | 1  | 0  | 0  |
| 1   | 1   | 1  | 0  | 0  | 0  |

**Circuito:**

Figura 16 — Circuito: Decoders 2x4



## Bloco Macro:

Figura 17 — Macro: Decoders 2x4



### 3.1.2.3 — Decoders 3x8

Possuí 3 bit de entrada e 8 saídas. Cada valor de entrada libera uma saída diferente, sempre da saída 0 até a saída 7.

**Expressão:**

$$m0 = \bar{A} \cdot \bar{B} \cdot \bar{C}$$

$$m1 = \bar{A} \cdot \bar{B} \cdot C$$

$$m2 = \bar{A} \cdot B \cdot \bar{C}$$

$$m3 = \bar{A} \cdot B \cdot C$$

$$m4 = A \cdot \bar{B} \cdot \bar{C}$$

$$m5 = A \cdot \bar{B} \cdot C$$

$$m6 = A \cdot B \cdot \bar{C}$$

$$m7 = A \cdot B \cdot C$$

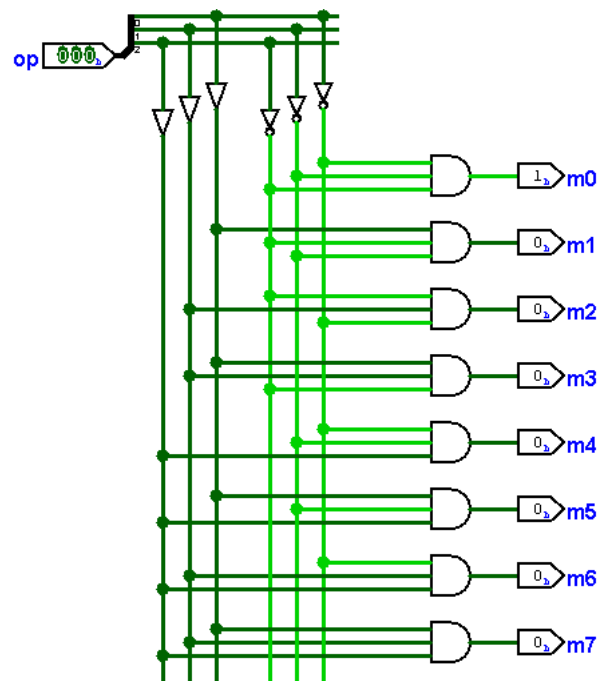
**Tabela:**

Tabela 12 — Decoders 3x8

| Op2 | Op1 | Op0 | m7 | m6 | m5 | m4 | m3 | m2 | m1 | m0 |
|-----|-----|-----|----|----|----|----|----|----|----|----|
| 0   | 0   | 0   | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  |
| 0   | 0   | 1   | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  |
| 0   | 1   | 0   | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  |
| 0   | 1   | 1   | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  |
| 1   | 0   | 0   | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  |
| 1   | 0   | 1   | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  |
| 1   | 1   | 0   | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1   | 1   | 1   | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |

## Circuito:

Figura 18 — Circuito: Decoders 3x8



## Bloco Macro:

Figura 19 — Macro: Decoders 3x8



### 3.1.3 — MUX 1 bit construídos

#### Multiplexador 2 Operações

Precisa de 1 bit de seleção, e alterna entre 2 entradas de dados.

**Expressão:**

$$i0 = \overline{Op0}$$

$$i1 = Op0$$

$$Q = i0 + i1$$

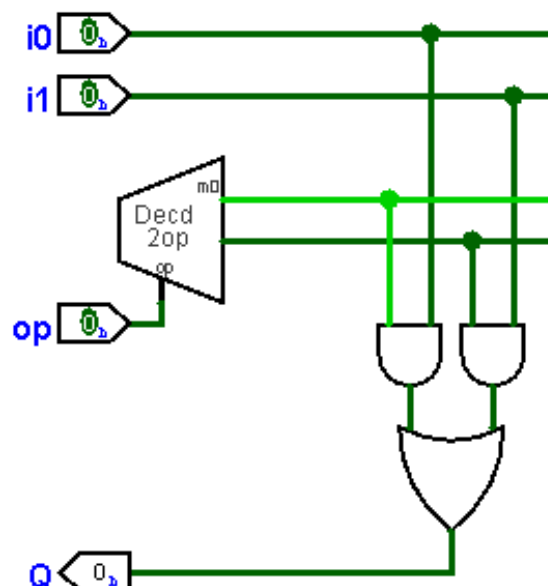
**Tabela:**

Tabela 13 — Multiplexador 2 operações

| Op0 | Q  |
|-----|----|
| 0   | i0 |
| 1   | i1 |

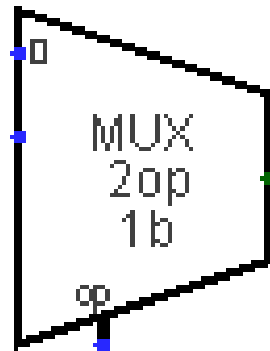
**Circuito:**

Figura 20 — Circuito: Multiplexador 2 operações



**Bloco Macro:**

Figura 21 — Macro: Multiplexador 2 operações



## Multiplexador 4 Operações

Precisa de 2 bits de seleção, e alterna entre 4 entradas de dados.

**Expressão:**

$$i0 = \overline{Op1} \cdot \overline{Op0}$$

$$i1 = Op1 \cdot \overline{Op0}$$

$$i2 = \overline{Op1} \cdot Op0$$

$$i3 = Op1 \cdot Op0$$

$$Q = i0 + i1 + i2 + i3$$

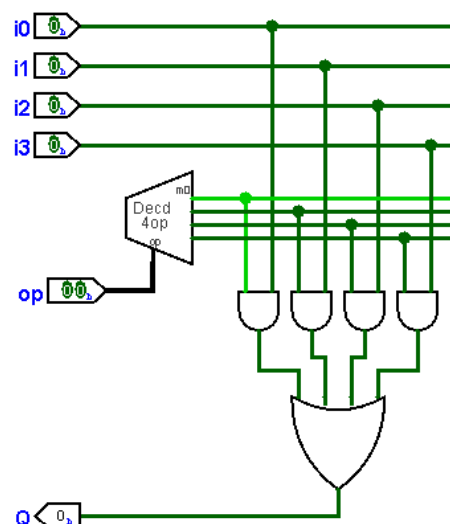
**Tabela:**

Tabela 14 — Multiplexador 4 operações

| Op1 | Op0 | Q  |
|-----|-----|----|
| 0   | 0   | i0 |
| 0   | 1   | i1 |
| 1   | 0   | i2 |
| 1   | 1   | i3 |

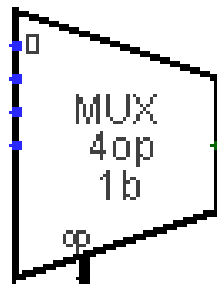
**Circuito:**

Figura 22 — Circuito: Multiplexador 4 operações



**Bloco Macro:**

Figura 23 — Macro: Multiplexador 4 operações





## Multiplexador 8 Operações

Possui 3 bit de entrada e 8 saídas. Cada valor de entrada libera uma saída diferente, sempre da saída 0 até a saída 7.

**Expressão:**

$$i0 = \overline{Op2} \cdot \overline{Op1} \cdot \overline{Op0}$$

$$i1 = \overline{Op2} \cdot \overline{Op1} \cdot Op0$$

$$i2 = \overline{Op2} \cdot Op1 \cdot \overline{Op0}$$

$$i3 = \overline{Op2} \cdot Op1 \cdot Op0$$

$$i4 = Op2 \cdot \overline{Op1} \cdot \overline{Op0}$$

$$i5 = Op2 \cdot \overline{Op1} \cdot Op0$$

$$i6 = Op2 \cdot Op0 \cdot \overline{Op1}$$

$$i7 = Op2 \cdot Op1 \cdot Op0$$

$$S = i0 + i1 + i2 + i3 + i4 + i5 + i6 + i7$$

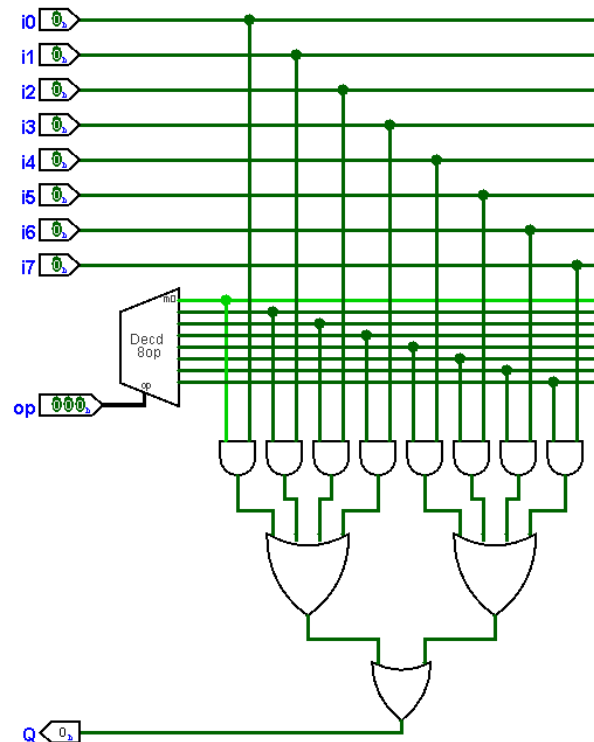
**Tabela:**

Tabela 15 — Multiplexador 8 operações

| Op2 | Op1 | Op0 | Q  |
|-----|-----|-----|----|
| 0   | 0   | 0   | i0 |
| 0   | 0   | 1   | i1 |
| 0   | 1   | 0   | i2 |
| 0   | 1   | 1   | i3 |
| 1   | 0   | 0   | i4 |
| 1   | 0   | 1   | i5 |
| 1   | 1   | 0   | i6 |
| 1   | 1   | 1   | i7 |

## Circuito:

Figura 24 — Circuito: Multiplexador 8 operações



## Bloco Macro:

Figura 25 — Macro: Multiplexador 8 operações



### 3.1.4 — MUX 4 bit construídos (Encadeamento)

#### Multiplexador 2 Operações

Precisa de 1 bit de seleção, e alterna entre 2 entradas de dados.

#### Expressão:

A expressão do multiplexador de 4 bits, segue a mesma base do Mux de 1 bit, porém, levando em conta que os dados transportados para a saída e recebidos são de 4 bits, com exceção do seletor das operações que continua 1 bit, e tratados de forma individual por cada multiplexador interno.

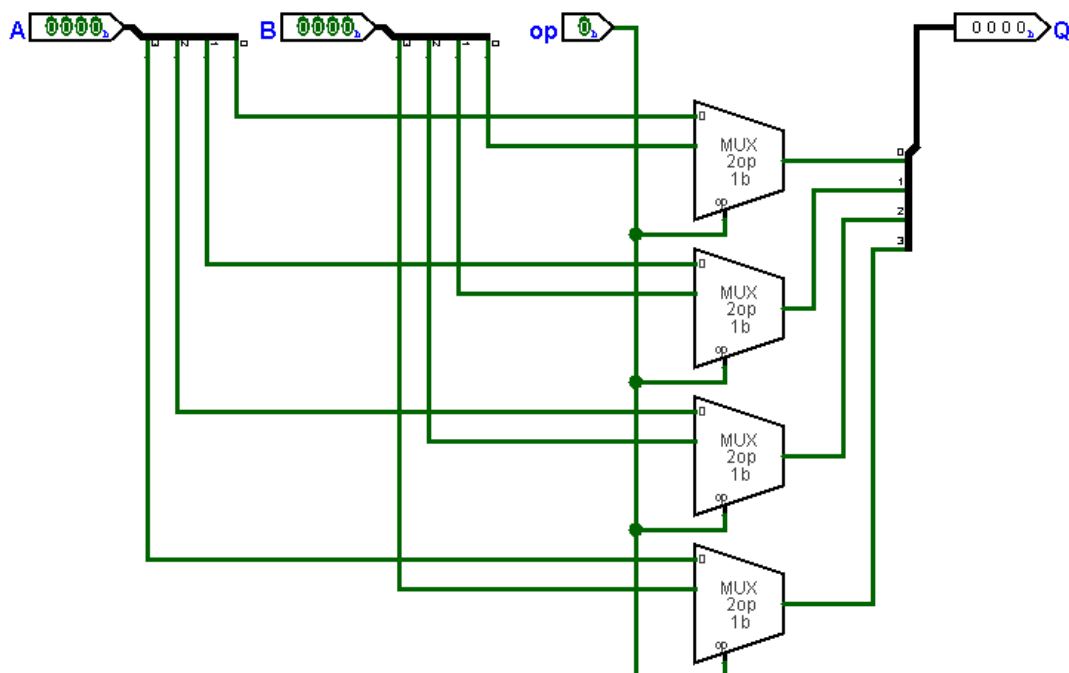
#### Tabela:

Tabela 16 — Multiplexador 2 operações 4 bits

| Op0 | Q      |
|-----|--------|
| 0   | InputA |
| 1   | Input  |

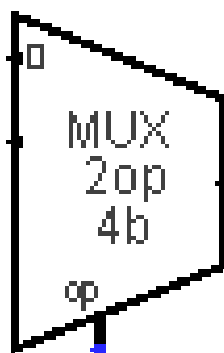
#### Circuito:

Figura 26 — Circuito: Multiplexador 2 operações 4 bits



**Bloco Macro:**

Figura 27— Macro: Multiplexador 2 operações 4 bits



## Multiplexador 4 Operações

Precisa de 2 bits de seleção, e alterna entre 4 entradas de dados.

### Expressão:

A expressão do multiplexador de 4 bits, segue a mesma base do Mux de 1 bit, e o seletor de operações 2 bits. A base do construção é a mesma do multiplexador de 2 operações.

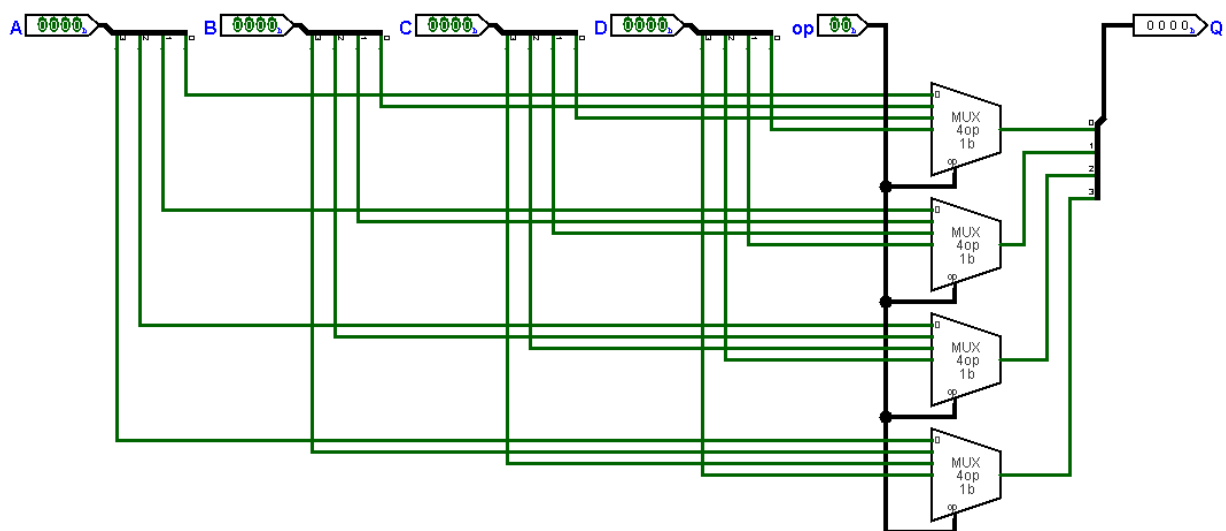
### Tabela:

Tabela 17 — Multiplexador 4 operações

| Op1 | Op0 | Q      |
|-----|-----|--------|
| 0   | 0   | InputA |
| 0   | 1   | InputB |
| 1   | 0   | InputC |
| 1   | 1   | InputD |

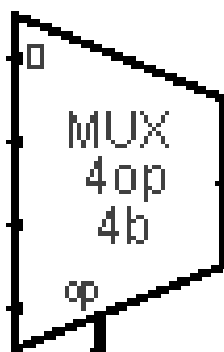
### Circuito:

Figura 28 — Circuito: Multiplexador 4 operações 4 bits



**Bloco Macro:**

Figura 29 — Macro: Multiplexador 4 operações 4 bits



## Multiplexador 8 Operações

Possui 3 bit de entrada e 8 saídas. Cada valor de entrada libera uma saída diferente, sempre da saída 0 até a saída 7.

### Expressão:

A expressão do multiplexador de 4 bits, segue a mesma base do Mux de 1 bit, e o seletor de operações 3 bits. A base da construção é a mesma do multiplexador de 4 operações.

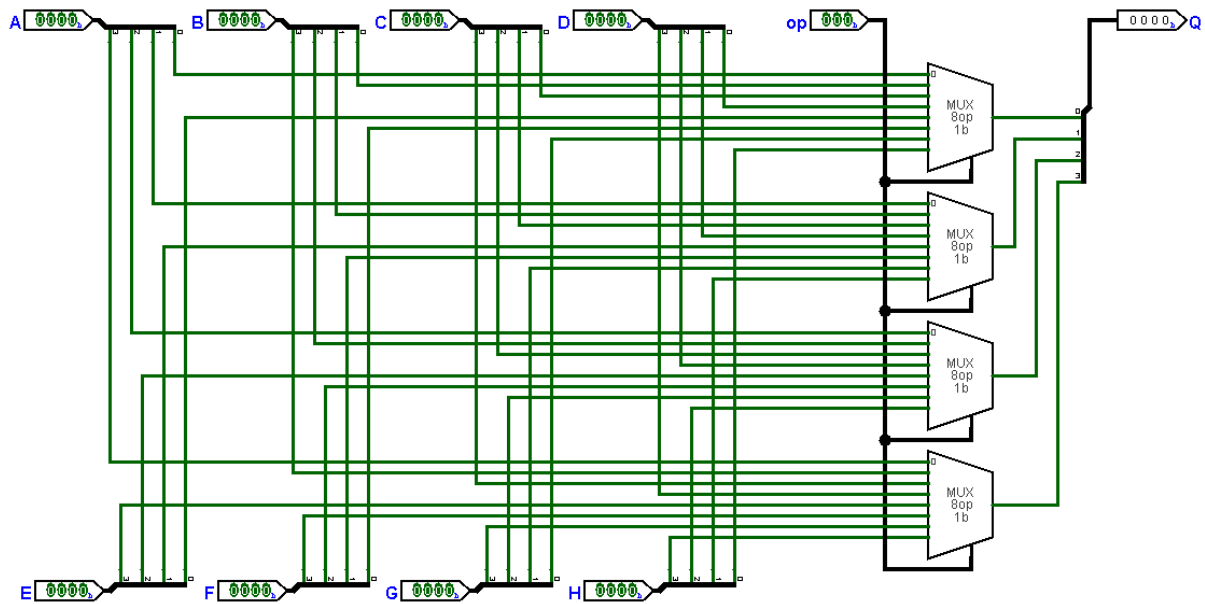
### Tabela:

Tabela 18 — Multiplexador 8 operações 4 bits

| Op2 | Op1 | Op0 | Q      |
|-----|-----|-----|--------|
| 0   | 0   | 0   | InputA |
| 0   | 0   | 1   | InputB |
| 0   | 1   | 0   | InputC |
| 0   | 1   | 1   | InputD |
| 1   | 0   | 0   | InputE |
| 1   | 0   | 1   | InputF |
| 1   | 1   | 0   | InputG |
| 1   | 1   | 1   | InputH |

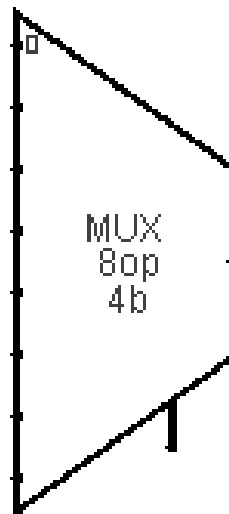
### Circuito:

Figura 30 — Circuito: Multiplexador 8 operações 4 bits



**Bloco Macro:**

Figura 31 — Macro: Multiplexador 8 operações 4 bits





### 3.1.5 — Shifter (Pós ULA)

O shifter é um circuito responsável por deslocar os bits do resultado para a esquerda ou para a direita, dependendo de suas linhas de controle de 2 bits. Quando configurado para Hold (00), ele simplesmente mantém o valor original, sem qualquer alteração — comportamento equivalente a uma operação nula. No modo SRL (01), o shifter desloca todos os bits para a direita, o que, em representação binária sem sinal, corresponde a dividir o valor por 2, descartando o bit menos significativo. Já o modo SLL (10) desloca os bits para a esquerda, o que equivale a multiplicar por 2, inserindo zero na posição menos significativa. Por fim, o modo 11 executa novamente um Hold, mantendo o valor inalterado.

Para evitar comportamentos redundantes e padronizar as seleções, foi empregado um pequeno macro auxiliar denominado **Helper\_4op\_to\_3op**, cuja função é justamente redirecionar a combinação 11 para a mesma ação que 00, prevenindo operações inválidas e simplificando o controle do shifter.

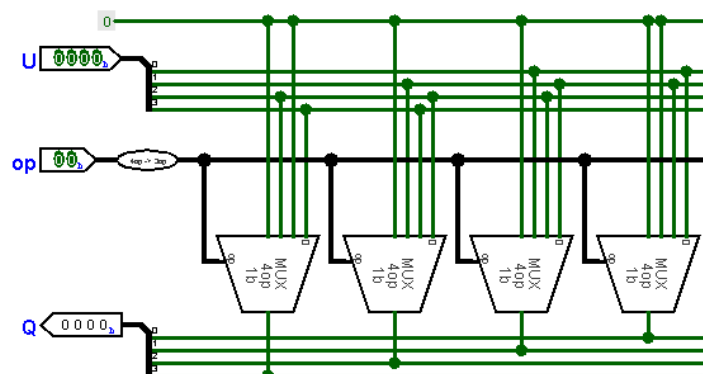
**Tabela:**

Tabela 19 — Shifter 4 bits

| Op1 | Op0 | Q          | Semelhança       |
|-----|-----|------------|------------------|
| 0   | 0   | Hold       | Não altera       |
| 0   | 1   | SRL        | Dividi por 2     |
| 1   | 0   | SLL        | Multiplica por 2 |
| 1   | 1   | N/A (Hold) | N/A (Não altera) |

**Circuito:**

Figura 32 — Circuito: Shifter 4 bits



**Bloco Macro:**

Figura 33 — Macro: Shifter 4 bits



### 3.1.6 — Helper 4op to 3op

Este macro auxiliar foi criado especificamente para este projeto com o objetivo de evitar combinações de controle inválidas ou desnecessárias. Ele atua em situações onde um conjunto de 2 bits de operação é enviado para circuitos como um decodificador 2x4, mas apenas três das quatro saídas (m0, m1 e m2) são realmente utilizadas.

Sem esse tratamento, a saída “m3” ficaria ativa em determinadas combinações, resultando em sinais flutuantes ou caminhos que nunca deveriam ser habilitados. Com o uso do Helper, qualquer tentativa de acionar a quarta saída é redirecionada para a primeira (m0).

Ele usa a teoria dos MaxTerms, para manter a saída iguais a da entrada, com exceção de quando for “11”, assim, ele transforma novamente em “00”.

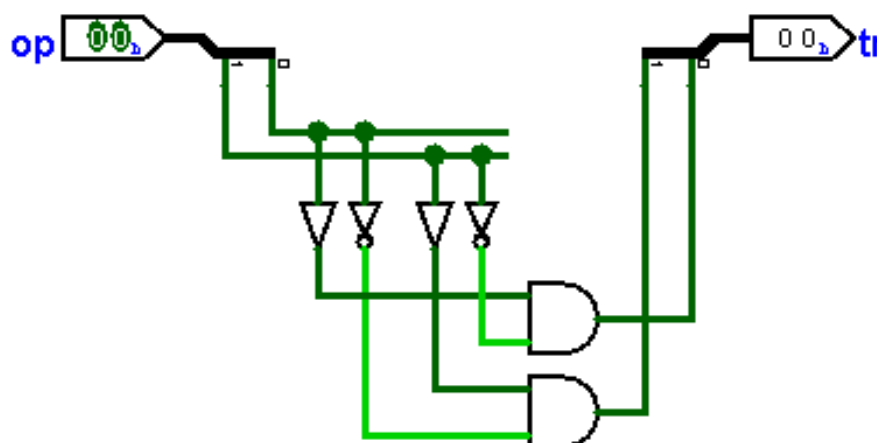
**Tabela:**

Tabela 20 — Helper 4op to 3op

| Op1 | Op0 | TR |
|-----|-----|----|
| 0   | 0   | 00 |
| 0   | 1   | 01 |
| 1   | 0   | 10 |
| 1   | 1   | 00 |

**Circuito:**

Figura 34 — Circuito: Helper 4op to 3op



## Bloco Macro:

Figura 35 — Macro: Helper 4op to 3op



### 3.2 — Módulos de Controle (Tri-States)

Os módulos de controle são responsáveis por regular o fluxo de dados dentro da CPU, permitindo que diferentes partes do circuito compartilhem o mesmo barramento sem gerar conflitos. Para isso, utilizam-se dispositivos Tri-State, capazes de assumir três estados distintos: nível lógico alto, nível lógico baixo e alta impedância (Z). O estado de alta impedância desconecta eletricamente a saída do componente do barramento, impedindo que ela interfira em outros sinais — princípio fundamental em arquiteturas que dependem de linhas compartilhadas.

Por trás desses controladores estão os Transmission Gates, elementos formados pela conexão paralela de um transistor NMOS e outro PMOS. Essa estrutura permite que o sinal seja conduzido apenas quando um controle de habilitação está ativo; quando desabilitado, a passagem do sinal é bloqueada, entrando justamente no estado de alta impedância. A partir dessa base, constroem-se Buffers e Inversores nas versões Tri-State, permitindo não apenas transmitir ou inverter um sinal, mas também controlar quando essa ação deve ocorrer.

Com isso, foram desenvolvidos 12 Tri-States no total:

- 6 Buffers Tri-State (High Enable e Low Enable nas versões de 1, 2 e 4 bits)
- 6 Inversores Tri-State (também nas versões High Enable e Low Enable para 1, 2 e 4 bits)

Os modelos de 2 e 4 bits foram obtidos pelo encadeamento de suas versões de 1 bit, seguindo a mesma lógica utilizada para expandir outros componentes modulares.

### 3.2.1 — Transmission Gate

O Transmission Gate (TG) é o elemento fundamental dos circuitos Tri-State utilizados no projeto. Ele é formado pela conexão paralela de um transistor NMOS e um PMOS, ambos controlados por sinais complementares. Quando habilitado, o TG permite a passagem livre do sinal entre entrada e saída, funcionando como um “switch” digital bidirecional. Quando desabilitado, ele se isola completamente, colocando sua saída em alta impedância (Z). Esse comportamento torna o TG ideal para controle de barramentos, onde vários dispositivos compartilham a mesma linha, mas apenas um deve conduzir o sinal por vez.

Figura 36 — Circuito: Transmission Gate

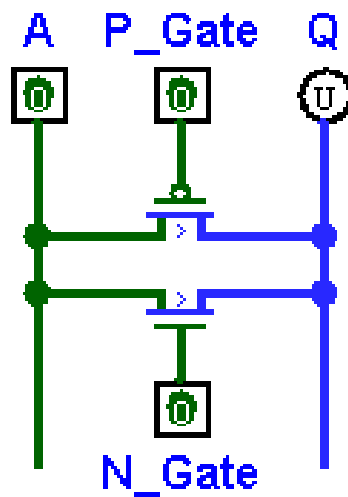
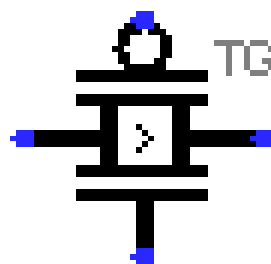


Figura 37 — Macro: Transmission Gate



### 3.2.2 — High & Low Enable

Os circuitos Tri-State podem ser controlados por dois estilos de habilitação: **High Enable** e **Low Enable**.

- Em um dispositivo **High Enable**, o sinal é conduzido apenas quando o pino de controle está em nível lógico alto.
- Em um **Low Enable**, a condução ocorre quando o pino de controle está em nível lógico baixo.

Ambas as versões são úteis em diferentes contextos de controle, pois permitem compatibilidade com sinais de ativação variados dentro da CPU e simplificam a lógica de interconexão. A partir desses esquemas de habilitação, definem-se claramente os momentos em que cada módulo pode enviar dados ao barramento ou deve permanecer isolado.

### 3.2.3 — Buffers Tri-States

Os **Buffers Tri-State** são módulos que reproduzem o nível lógico de entrada na saída, mas somente quando habilitados. Quando desabilitados, entram no estado de alta impedância, liberando o barramento para outros componentes. Eles são essenciais para o roteamento organizado de dados entre registradores, memória e ULA, garantindo que apenas um dispositivo escreva no barramento por vez. As versões de 1, 2 e 4 bits foram construídas a partir do encadeamento de buffers individuais, mantendo consistência com a lógica modular aplicada em todo o projeto.

#### Buffer Tri-State Active High Enable

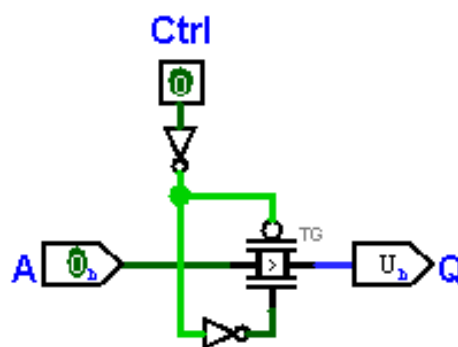
Tabela:

Tabela 21 — Buffer Tri-State Active High Enable

| Ctrl | A | Q    |
|------|---|------|
| 0    | 0 | Hi-Z |
| 0    | 1 | Hi-Z |
| 1    | 0 | 0    |
| 1    | 1 | 1    |

Circuito:

Figura 38 — Circuito: Buffer Tri-State Active High Enable





### Buffer Tri-State Active Low Enable

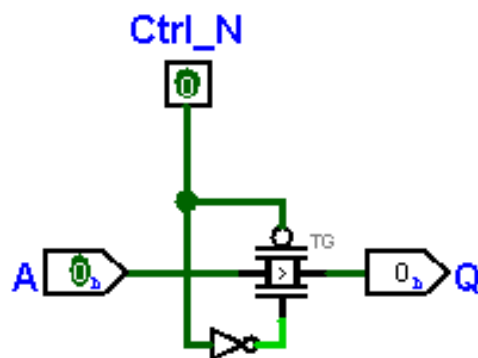
**Tabela:**

Tabela 22 — Buffer Tri-State Active Low Enable

| Ctrl | A | Q    |
|------|---|------|
| 0    | 0 | 0    |
| 0    | 1 | 1    |
| 1    | 0 | Hi-Z |
| 1    | 1 | Hi-Z |

**Circuito:**

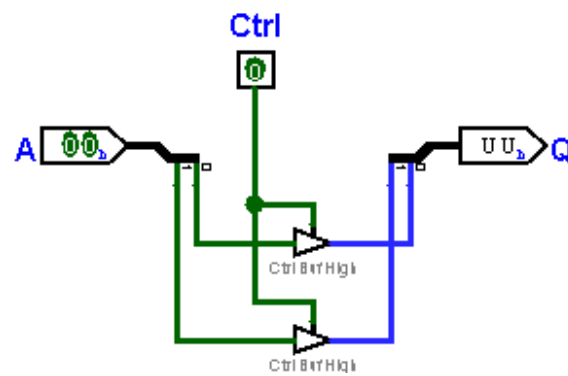
Figura 39 — Circuito: Buffer Tri-State Active Low Enable



## Buffer 2 bits Tri-State Active High Enable

Circuito:

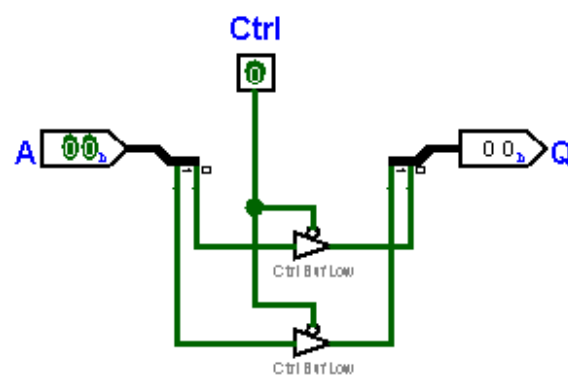
Figura 40 — Circuito: Buffer 2 bits Tri-State Active High Enable



## Buffer 2 bits Tri-State Active Low Enable

Circuito:

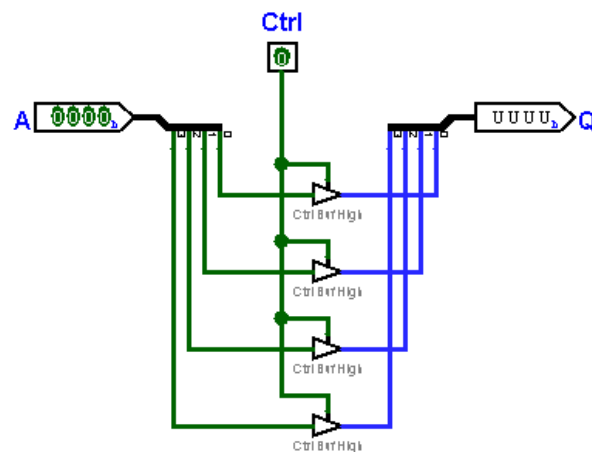
Figura 41 — Circuito: Buffer 2 bits Tri-State Active Low Enable



## Buffer 4 bits Tri-State Active High Enable

Circuito:

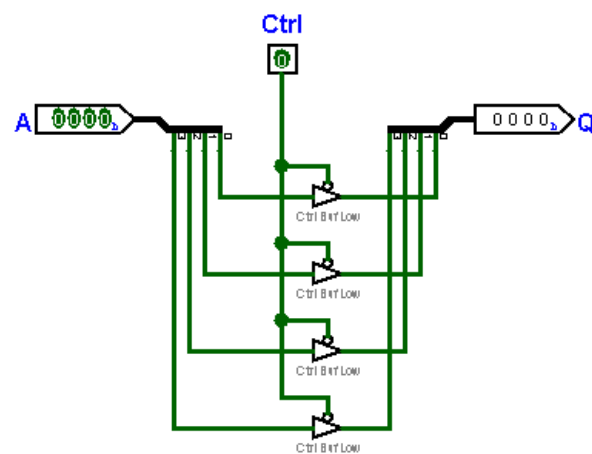
Figura 42 — Circuito: Buffer 4 bits Tri-State Active High Enable



## Buffer 4 bits Tri-State Active High Enable

Circuito:

Figura 43 — Circuito: Buffer 4 bits Tri-State Active Low Enable



### 3.2.4 — Inverters Tri-States

Os **Inversores Tri-State** combinam a função tradicional de inversão lógica com o mecanismo de controle de impedância. Assim como os buffers, eles permanecem ativos apenas quando habilitados e entram no estado “Z” quando desativados. A presença de inversores controláveis permite alterar sinais quando necessário, como rotinas de complemento, multiplexação avançada ou condicionamento de controle, sem interferir no restante do sistema. Versões de 1, 2 e 4 bits foram implementadas seguindo a mesma metodologia de expansão modular usada nos buffers.

#### Inverter Tri-State Active High Enable

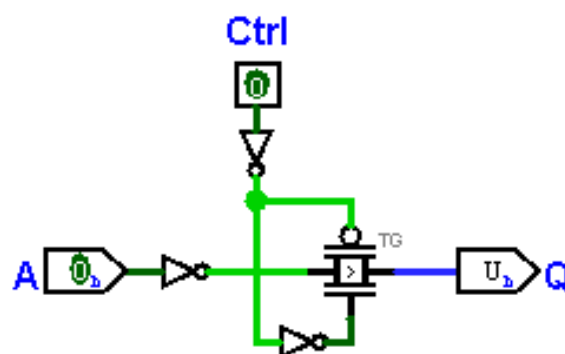
Tabela:

Tabela 23 — Inverter Tri-State Active High Enable

| Ctrl | A | Q    |
|------|---|------|
| 0    | 0 | Hi-Z |
| 0    | 1 | Hi-Z |
| 1    | 0 | 1    |
| 1    | 1 | 0    |

Circuito:

Figura 44 — Circuito: Inverter Tri-State Active High Enable



## Inverter Tri-State Active Low Enable

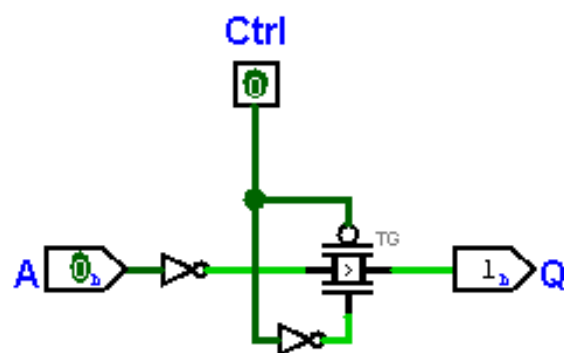
Tabela:

Tabela 24 — Inverter Tri-State Active Low Enable

| Ctrl | A | Q    |
|------|---|------|
| 0    | 0 | 1    |
| 0    | 1 | 0    |
| 1    | 0 | Hi-Z |
| 1    | 1 | Hi-Z |

Circuito:

Figura 45 — Circuito: Inverter Tri-State Active Low Enable



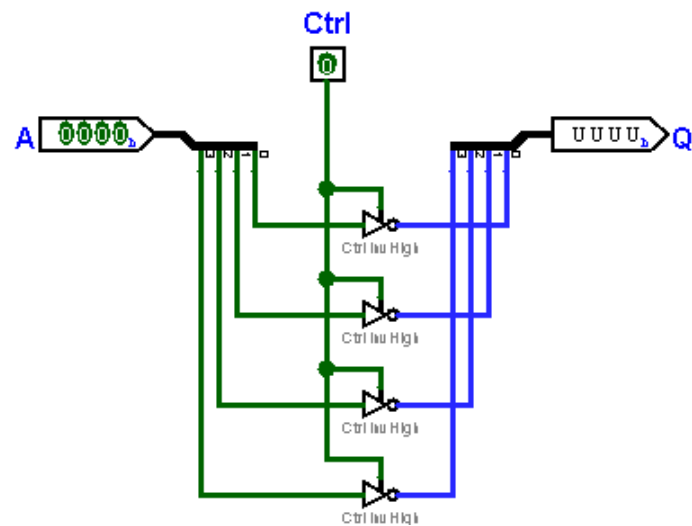
**Circuito:**

**Circuito:**

## Inverter 4 bits Tri-State Active High Enable

Circuito:

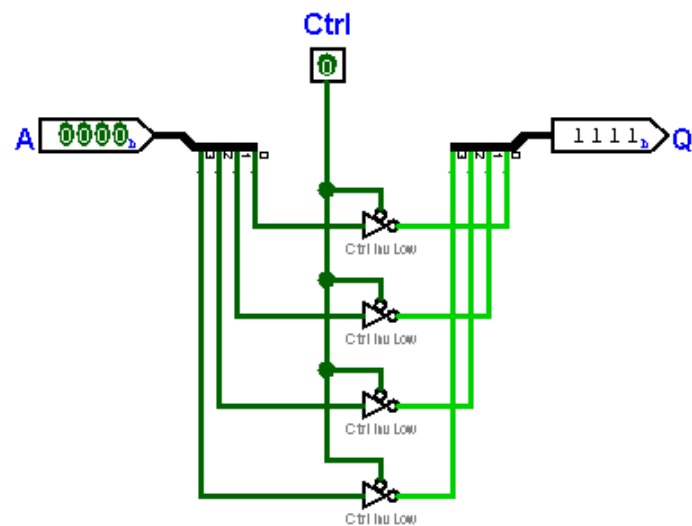
Figura 48 — Circuito: Inverter 4 bits Tri-State Active High Enable



## Inverter 4 bits Tri-State Active High Enable

Circuito:

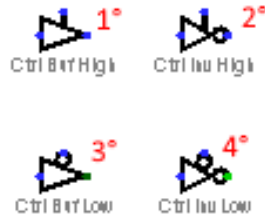
Figura 49 — Circuito: Inverter 4 bits Tri-State Active Low Enable



### 3.2.5 — Macros Tri-States

As imagens dos macros estão demonstradas em forma conjunta, e serão explicados o que são conforme apresentados.

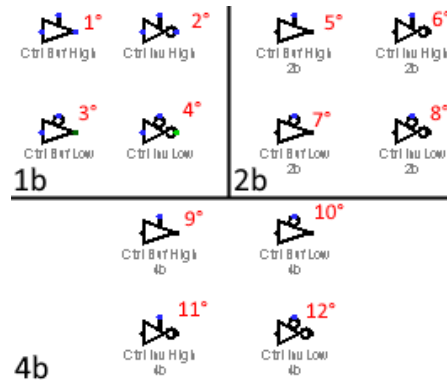
Figura 50 — Circuito: Macros Buffers & Inverters, High & Low Enable 1 bit



- 1° Buffer Tri-State High Enable 1 bit;
- 2° Buffer Tri-State Low Enable 1 bit;
- 3° Inverter Tri-State High Enable 1 bit;
- 4° Inverter Tri-State Low Enable 1 bit.

As imagens dos macros estão demonstradas em forma conjunta novamente.

Figura 51 — Circuitos: Macros Buffers & Inverters, High & Low Enable 1 bit, 2 bits & 4 bits



- 1°, 2°, 3°, 4° Mesmas das já citadas
- 5° Buffer Tri-State High Enable 2 bit;
- 6° Buffer Tri-State Low Enable 2 bit;
- 7° Inverter Tri-State High Enable 2 bit;
- 8° Inverter Tri-State Low Enable 2 bit;
- 9° Buffer Tri-State High Enable 4 bit;
- 10° Buffer Tri-State Low Enable 4 bit;
- 11° Inverter Tri-State High Enable 4 bit;
- 12° Inverter Tri-State Low Enable 4 bit.



## 3.3 — Módulos de Memória

### 3.3.1 — Latches Transparentes

#### 3.3.1.1 — Latches Transparentes

Os Latches são dispositivos de memória de nível lógico utilizados para armazenar temporariamente um bit de informação. Eles são sensíveis ao nível do sinal de controle (geralmente chamado Enable ou Clock), ou seja, escrevem o valor na saída enquanto o controle está ativo e o mantêm estável quando o controle está inativo. Por esse motivo, os latches são fundamentais para formar registradores, contadores e estruturas de memória mais complexas dentro da CPU.

Embora esse tipo de latch transparente não seja utilizado diretamente em todos os módulos do projeto, sua lógica de funcionamento é importante para compreender elementos posteriores. Em especial, os módulos **Latch A** e **Latch B**, utilizados no caminho de dados antes da ULA, seguem a mesma ideia de retenção temporária de valores — embora sejam implementados com flip-flops D da própria ferramenta. Esses dois módulos serão detalhados mais adiante, em sua seção específica.

Para este projeto, foram implementados Latches de 1 bit e de 4 bits, sendo estes últimos construídos pelo encadeamento de quatro unidades de 1 bit, seguindo o padrão modular adotado em todo o circuito.

Nas Tabelas, Circuitos e Macros apresentados, as letras: E ou Clk, D, Q e Q' ou Qn, indicam: Nível do Clock, Dados recebidos, saída e saída invertida, respectivamente.

**Tabela:**

Tabela 25 — Latch

| E | D | Q     | Q'    |
|---|---|-------|-------|
| 0 | 0 | Hold  | Hold' |
| 0 | 1 | Hold  | Hold' |
| 1 | 0 | Reset | Set   |
| 1 | 1 | Set   | Reset |

### 3.3.1.2 — Latches Master

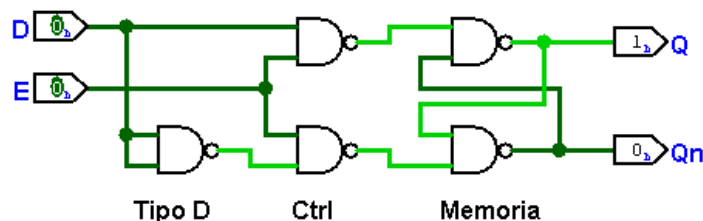
O Latch Master é a primeira etapa de um par de latches utilizados em conjunto para evitar problemas de transparência (ou seja, a atualização contínua do valor durante o sinal ativo). O Master captura o valor de entrada enquanto o sinal de controle está em nível ativo — geralmente nível alto — e o mantém congelado quando o controle muda para o nível inativo.

Seu papel é garantir que a informação seja absorvida apenas em momentos específicos, preparando-a para ser armazenada de forma estável pelo estágio seguinte, o Slave.

#### Latch Master 1 bit

**Circuito:**

Figura 52 — Circuito: Latch Master 1 bit



**Bloco Macro:**

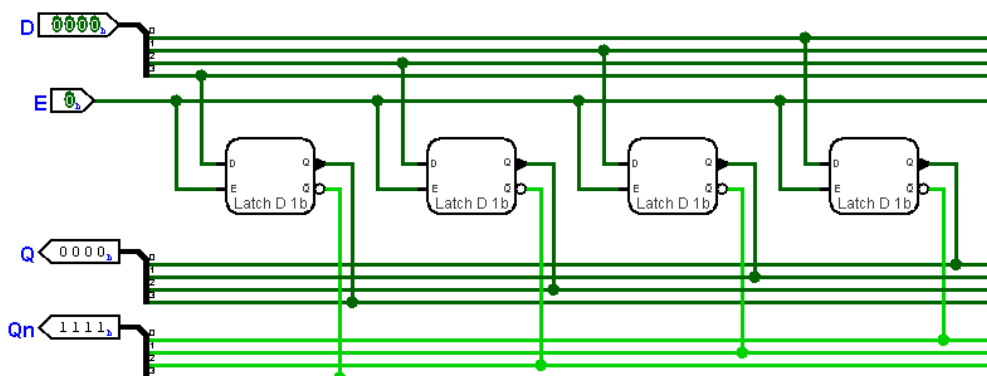
Figura 53 — Macro: Latch Master 1 bit



#### Latch Master 4 bit

**Circuito:**

Figura 54 — Circuito: Latch Master 4 bit



## Bloco Macro:

Figura 55 — Macro: Latch Master 4 bit



### 3.3.1.3 — Latches Slave

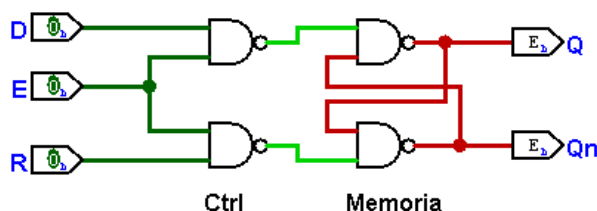
O Latch Slave é a segunda etapa do conjunto Master–Slave. Ele recebe o valor previamente capturado pelo Master e o transfere para sua saída quando o sinal de controle assume o nível oposto. Dessa forma, enquanto o Master escreve com o clock em nível alto, o Slave escreve apenas quando o clock está em nível baixo (ou vice-versa, dependendo do projeto).

Essa combinação elimina a transparência simultânea entre entrada e saída, resultando em um comportamento semelhante ao de um Flip-Flop, armazenando o dado de forma estável a cada ciclo completo do sinal de controle.

#### Latch Slave 1 bit

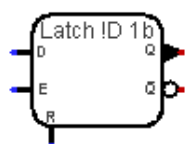
**Circuito:**

Figura 56 — Circuito: Latch Slave 1 bit



**Bloco Macro:**

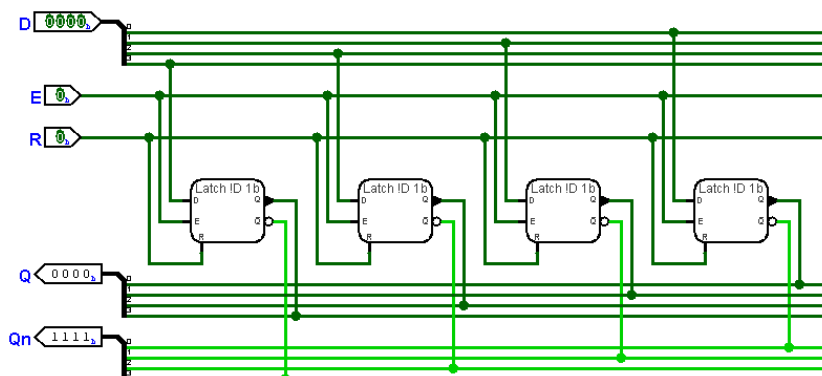
Figura 57 — Macro: Latch Slave 1 bit



#### Latch Slave 4 bit

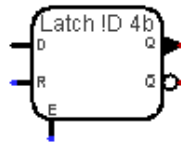
**Circuito:**

Figura 58 — Circuito: Latch Slave 4 bit



## Bloco Macro:

Figura 59 — Macro: Latch Slave 4 bit



### 3.3.2 — Flip Flop D

O Flip-Flop do tipo D (Data ou Delay) é um elemento sequencial sincronizado por borda, capaz de armazenar um bit de informação de forma estável enquanto o clock permanece em nível inativo. Diferente dos latches, que são dispositivos sensíveis ao nível do sinal de controle, o Flip-Flop D é sensível à borda do clock, armazenando o valor da entrada D exatamente no momento da transição (geralmente a borda de subida).

Sua principal vantagem é a previsibilidade: por armazenar dados apenas na borda do clock, o Flip-Flop D evita atualizações indesejadas que normalmente aparecem em latches simples, tornando-o ideal para sistemas síncronos e pipelines.

#### 3.3.2.1 — Implementações construídas

Para fins didáticos e como parte da evolução natural da construção dos elementos de memória deste projeto, também foram desenvolvidos Flip-Flops D de 1 bit e de 4 bits. Apesar de suportarem múltiplos bits, as versões de 4 bits não foram efetivamente utilizadas na CPU, estando presentes principalmente como exercício de entendimento e padronização modular.

#### 3.3.2.2 — Observações importantes

Os Flip-Flops D foram inicialmente construídos de forma “manual”, a partir da composição de latches Master-Slave, permitindo visualizar sua montagem interna e compreender seu funcionamento em um nível mais fundamental.

Entretanto, tal implementação artesanal naturalmente apresenta algumas limitações, como:

- risco de glitches;
- violação de setup ou hold time;
- sensibilidade a transições curtas de clock;
- inconsistências em simulações com sinais não idealizados.

Corrigir completamente esses aspectos demandaria um esforço considerável e desviaria do foco principal do projeto.

#### 3.3.2.3 — Uso dos Flip-Flops nativos da ferramenta

Por esse motivo, após a etapa de estudo e construção conceitual, optou-se por utilizar os Flip-Flops D fornecidos pela própria ferramenta, que já incluem:

- tratamento interno de temporização;
- proteção contra glitches;
- comportamento consistente para borda de clock;
- confiabilidade para uso em circuitos maiores.

Nas Tabelas, Circuitos e Macros apresentados, as letras: E ou Clk, D, Q e Q' ou Qn, indicam: Nível do Clock, Dados recebidos, saída e saída invertida, respectivamente.

**Tabela:**

Tabela 26 — Flip Flop D

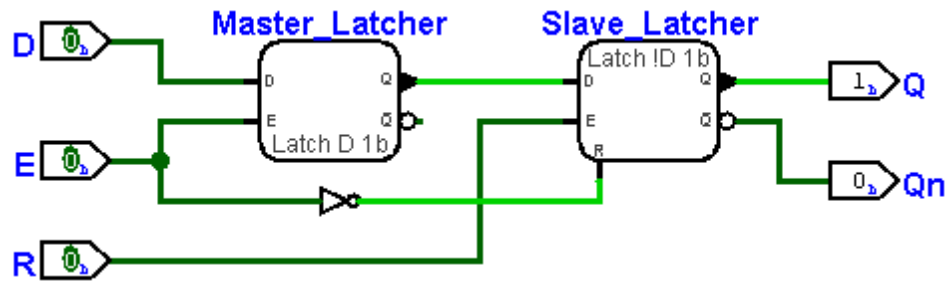
| Clk | D | Q | Q' |
|-----|---|---|----|
| ↓   | 0 | X | X  |
| ↓   | 1 | X | X  |
| ↑   | 0 | 0 | 1  |
| ↑   | 1 | 1 | 0  |

- ↓: Clock baixo (borda baixa);
- ↑: Clock de subida;
- X: Indiferença.

### 3.3.2.4 — Flip Flop D 1 bit

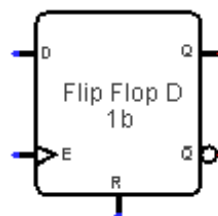
Circuito:

Figura 60 — Circuito: Flip Flop D 1 bit



Bloco Macro:

Figura 61 — Macro: Flip Flop D bit

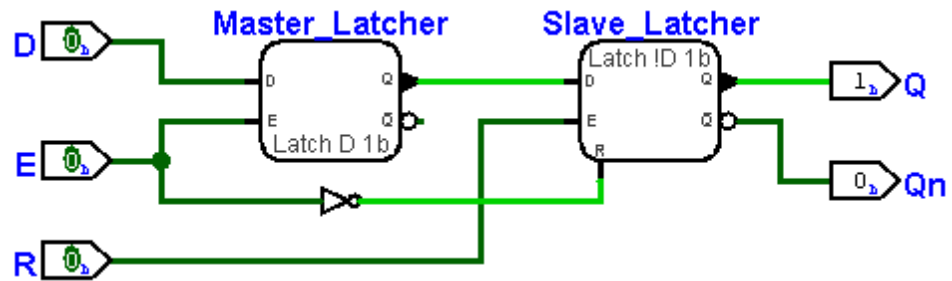




### 3.3.2.5 — Flip Flop D 4 bit

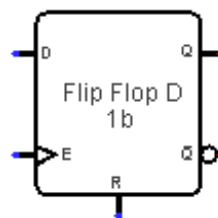
Circuito:

Figura 62 — Circuito: Flip Flop D 4 bits



Bloco Macro:

Figura 63 — Macro: Flip Flop D 4 bits



### 3.3.3 — Latch AB (Latch A e Latch B)

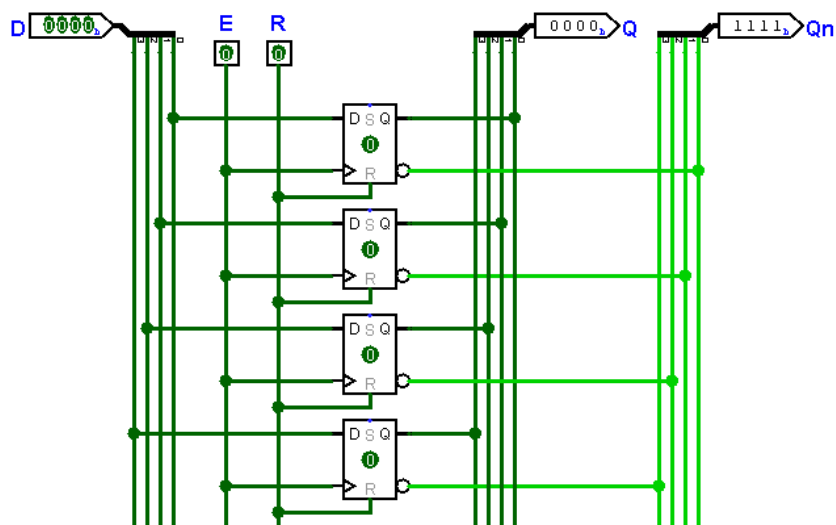
Os módulos Latch A e Latch B atuam como pequenos registradores de 4 bits posicionados entre o Banco de Registradores e a ULA. Apesar do nome, eles não são latches transparentes, mas sim blocos implementados com flip-flops D, utilizados para manter os operandos estáveis durante o ciclo de execução.

O Latch A recebe um valor vindo da saída A do banco de registradores e é conectado ao AMUX, permitindo que sua saída seja combinada com o valor do MBR quando necessário. Já o Latch B recebe valores da saída B do banco de registradores e segue diretamente para a entrada B da ULA, atuando como segundo operando em operações binárias.

Essa separação possibilita que a entrada A tenha maior flexibilidade (permitindo inclusive operações unitárias, como NOT), enquanto a entrada B mantém um operando fixo para operações que exigem dois valores.

**Circuito:**

Figura 64 — Circuito: Latch AB (Latch A e Latch B)



**Bloco Macro:**

Figura 65 — Macro: Latch AB (Latch A e Latch B)



### 3.3.4 — MAR (Memory Address Register)

O MAR (Memory Address Register) é o registrador responsável por armazenar o endereço que será enviado à memória durante operações de leitura ou escrita. Sua função principal é receber um valor de endereço oriundo do barramento e mantê-lo estável durante todo o ciclo de memória. Assim como os demais registradores do projeto, o MAR atua como um ponto de sincronização entre o barramento de dados e o subsistema de memória.

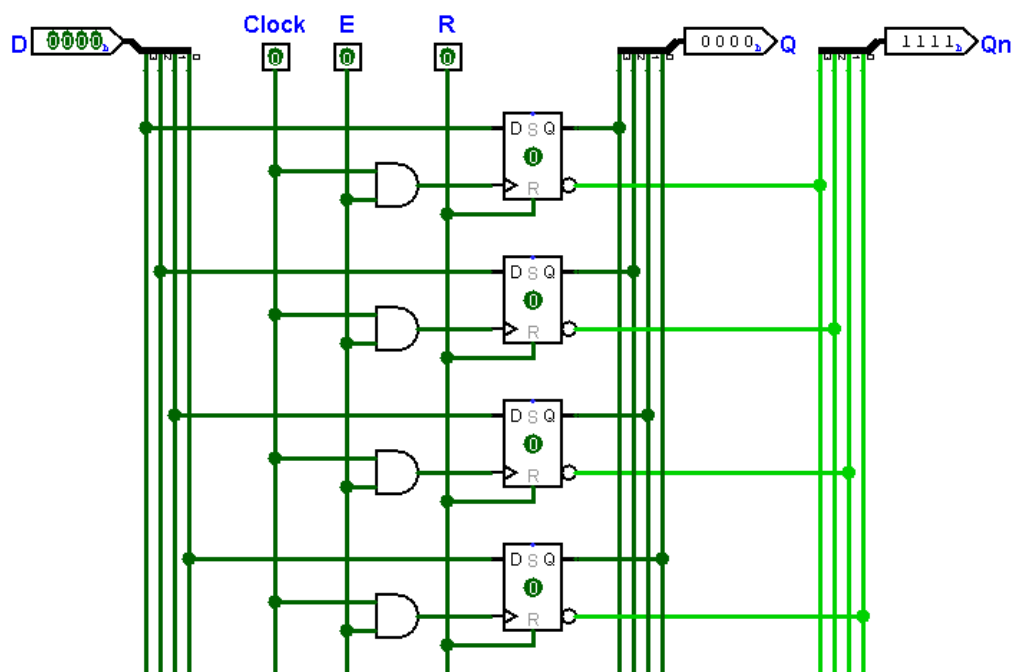
Para garantir esse funcionamento, o MAR foi implementado utilizando Flip-Flops D, oferecendo armazenamento síncrono e atualização baseada na borda de clock. Diferentemente dos Latches construídos manualmente no projeto, os Flip-Flops utilizados no MAR são os componentes fornecidos pela própria ferramenta de simulação, que já incluem a correção dos atrasos, condições de corrida e outros fenômenos que poderiam comprometer a operação de um registrador real.

Com isso, o MAR se torna um módulo totalmente sincronizado, capaz de:

- Capturar o endereço do barramento no momento adequado do ciclo de clock.
- Manter o valor estável até o término da operação de memória.
- Evitar condições de leitura incorreta causadas por propagação ou transparência de nível.

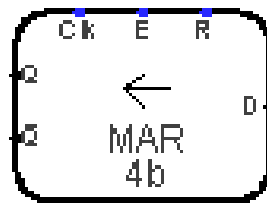
**Circuito:**

Figura 66 — Circuito: MAR 4 bits



### Bloco Macro:

Figura 67 — Macro: MAR 4 bits



Embora o projeto tenha abordado a construção manual de Latches e Flip-Flops para aprofundar o entendimento dos mecanismos internos de armazenamento, o MAR faz uso dos Flip-Flops internos da ferramenta exclusivamente por robustez, precisão temporal e fidelidade ao comportamento esperado em arquiteturas reais.

### 3.3.5 — MBR (Memory Buffer Register)

O MBR (Memory Buffer Register) é o registrador responsável por intermediar a transferência de dados entre a CPU e a memória, funcionando como um ponto de passagem entre o barramento interno (RxTx) e o módulo de memória propriamente dito. Seu papel é armazenar temporariamente os dados que estão sendo lidos da memória (Read) ou que serão escritos nela (Write), garantindo sincronização e integridade durante essas operações.

Em diagramas tradicionais e nos modelos apresentados em aula, o MBR aparece conectado ao barramento por um único fio bidirecional, pois em circuitos reais essa linha pode tanto receber quanto enviar dados. Entretanto, a ferramenta utilizada neste projeto — o Logisim — não oferece suporte nativo a pinos bidirecionais. Por esse motivo, o MBR foi implementado com duas portas separadas, uma de entrada e outra de saída, ambas conectadas ao barramento RxTx. Essas portas são controladas por sinais Tri-State de forma equivalente ao comportamento bidirecional original, garantindo que o registrador escreva no barramento apenas quando autorizado e permaneça em alta impedância quando não estiver ativo.

Outro aspecto importante observado durante o desenvolvimento é que o mecanismo de controle interno do MBR — o circuito que decide o que vai fazer com os dados — precisa ser replicado para cada bit do registrador. Para evitar duplicação manual extensa e melhorar a organização do projeto, foi criado um módulo auxiliar chamado “Handler MBR”, que encapsula exatamente essa lógica de controle, permitindo replicação estruturada e consistente entre todos os bits.

Além disso, por conta do MBR construído fazer apenas 3 operações, novamente será feito o uso do Helper “4 op to 3 op” na conexão da entrada de operações.

**Tabela:**

Tabela 27 — MBR

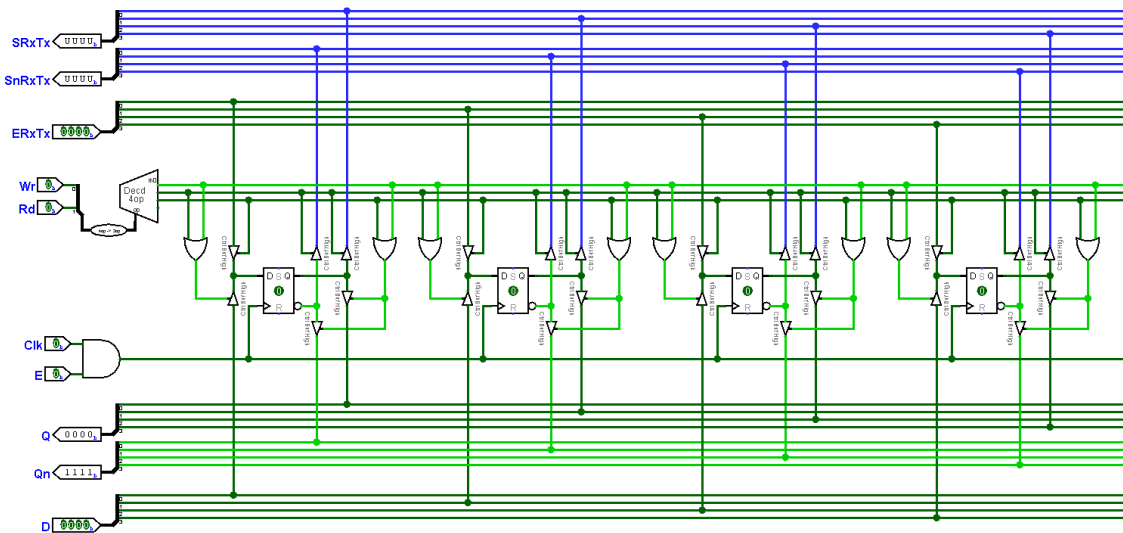
| Op1 | Op0 | Ação |
|-----|-----|------|
| 0   | 0   | N/A  |
| 0   | 1   | Wr   |
| 1   | 0   | Rd   |
| 1   | 1   | N/A  |

### 3.3.5.1 — Pure MBR (MBR Puro)

Este é o circuito apresentado de forma pura, ou seja, sem o uso do “Handler MBR”, assim sendo possível ver a poluição visual e repetição do circuito.

**Circuito:**

Figura 68 — Circuito: Pure MBR 4 bits

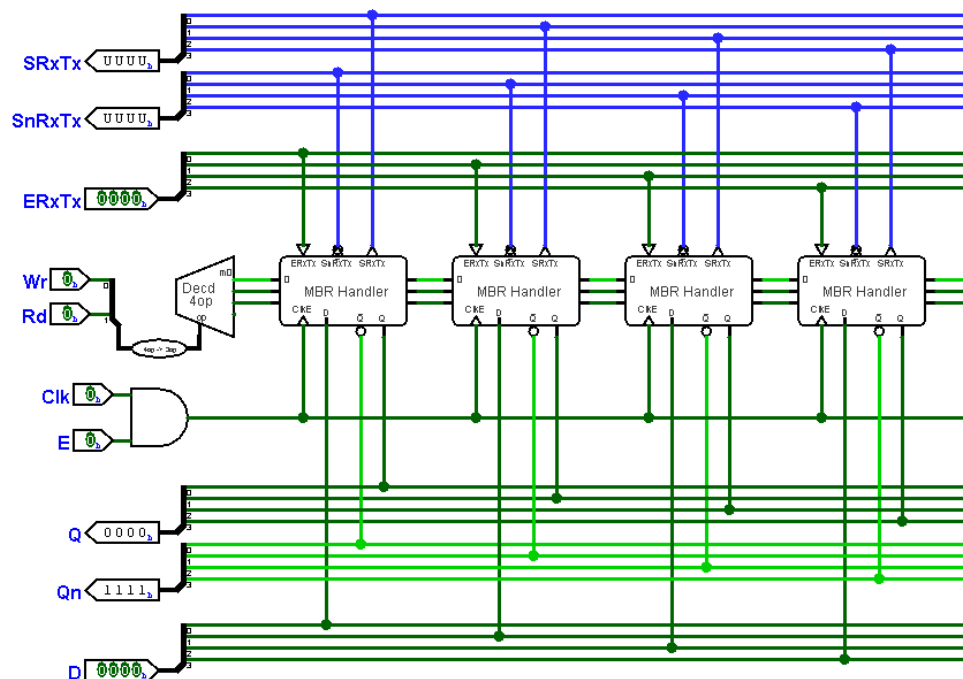


### 3.3.5.2 — MBR

Este é o circuito apresentado e usado no projeto, ou seja, com o uso do “Handler MBR”, ele oferece uma melhor organização, e permite uma melhor expansibilidade.

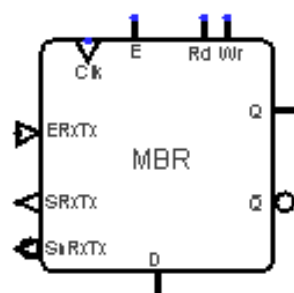
**Circuito:**

Figura 69 — Circuito: MBR 4 bits



**Bloco Macro:**

Figura 70 — Macro: MBR 4 bits

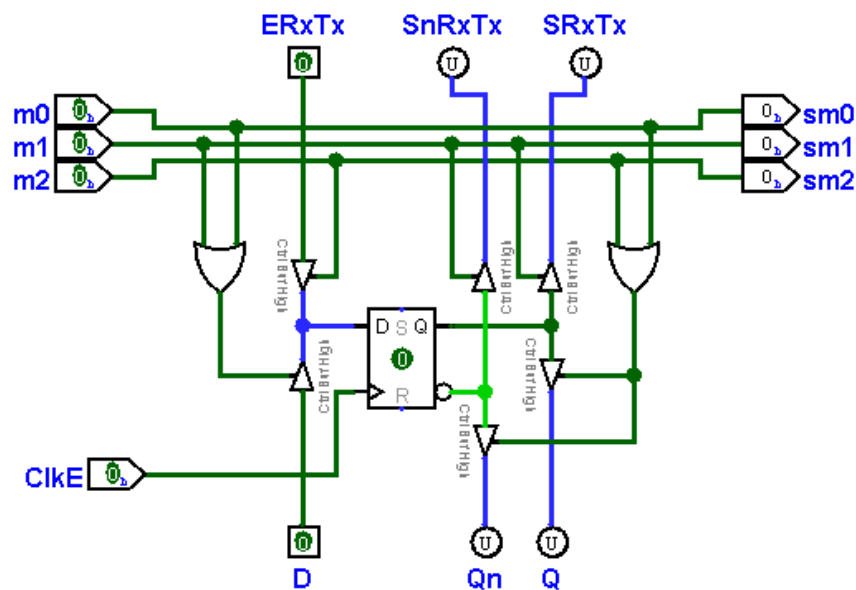


### 3.3.5.3 — Handler MBR

O Handler MBR, é exatamente o mesmo circuito que realiza o controle de ação, porém, encapsulado. Para construir de forma mais dinâmica, até as saídas usadas do decodificar são embutidas dentro do macro. As saídas e entradas estão na mesma direção do circuito puro, assim mantendo uma facilidade no entendimento.

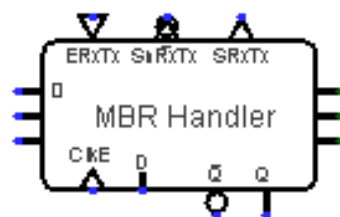
**Circuito:**

Figura 71 — Circuito: Handler MBR



**Bloco Macro**

Figura 72 — Macro: Handler MBR





### 3.3.6 — Registradores (Registers)

Os registradores são elementos fundamentais da CPU, responsáveis por armazenar temporariamente valores utilizados durante a execução das instruções. Eles funcionam como pequenas memórias de acesso imediato, permitindo que a unidade de controle e a ULA operem de forma rápida e sincronizada. Assim como o MAR e o MBR, os registradores utilizam Flip-Flops para manter seus dados de maneira estável, respondendo apenas aos sinais de carga definidos pelo controle.

Cada registrador do projeto possui duas possíveis saídas independentes, chamadas tradicionalmente de Saída A e Saída B. Essa divisão é comum em arquiteturas didáticas, pois permite que diferentes registradores forneçam simultaneamente valores distintos para a ULA, reduzindo conflitos e simplificando o gerenciamento dos barramentos internos.

Entretanto, assim como ocorre em outros módulos, cada saída só pode ser habilitada quando realmente se deve enviar dados ao barramento. Para garantir esse comportamento, há um circuito que centraliza o controle de liberação das saídas. Esse circuito recebe os sinais que determinam se o registrador deve disponibilizar seu conteúdo pela Saída A ou pela Saída B, ativando apenas o caminho correto e mantendo a outra linha em estado de alta impedância (Hi-Z).

Assim como o MBR, também há um “Handler Register”, que encapsula esta lógica de controle, mantendo o circuito mais organizado.

**Tabela:**

Tabela 28 — Register

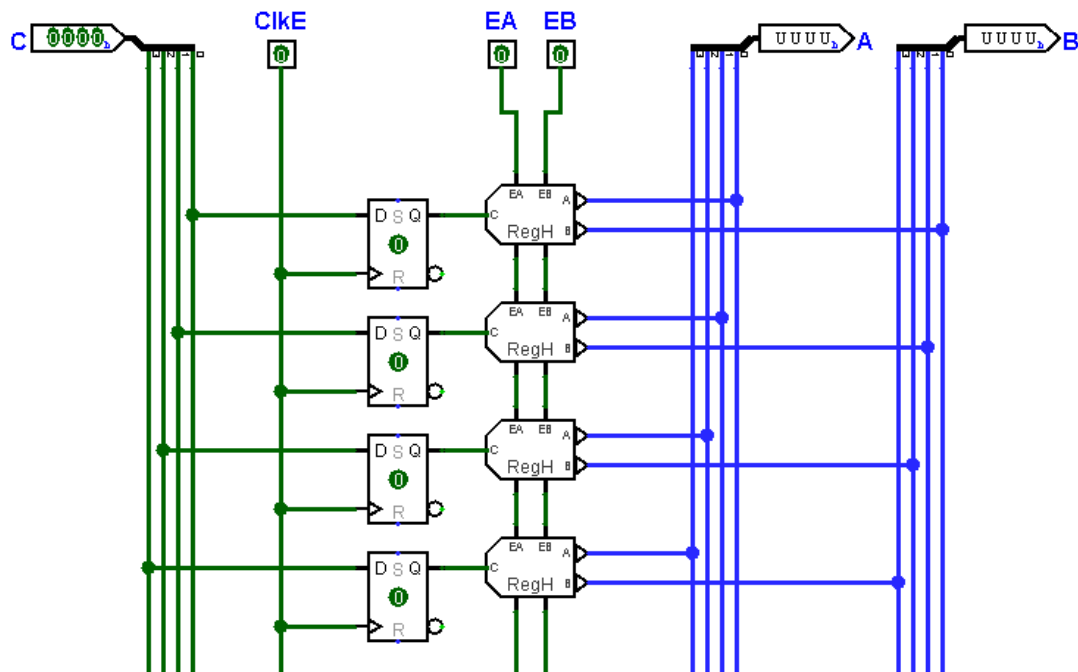
| EA | EB | InputA | InputB |
|----|----|--------|--------|
| 0  | 0  | Hi-Z   | Hi-Z   |
| 0  | 1  | Hi-Z   | InputC |
| 1  | 0  | InputC | Hi-Z   |
| 1  | 1  | InputC | InputC |

### 3.3.6.1 — Register

Este é o circuito apresentado e usado no projeto, ou seja, com o uso do “Handler Register”, ele oferece uma melhor organização, e permite uma melhor expansibilidade.

**Circuito:**

Figura 73 — Circuito: Register 4 bits



**Bloco Macro**

Figura 74 — Macro: Register 4 bits

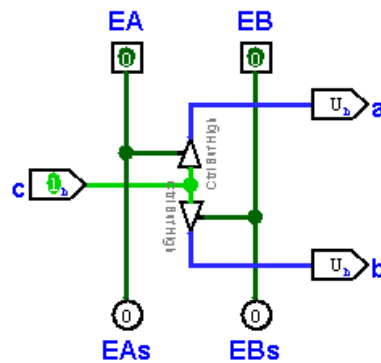


### 3.3.6.2 — Handlers Register

O Handler Register, é o circuito que realiza o controle de qual saída será direcionado, porém, encapsulado. As saídas e entradas estão na mesma direção do circuito puro, assim mantendo uma facilidade no entendimento.

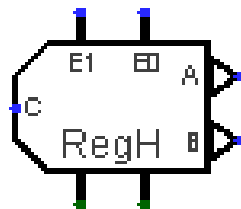
**Circuito:**

Figura 75 — Circuito: Handler Register



## Bloco Macro

Figura 76 — Macro: Handler Register



### 3.3.7 — Banco de Registradores (Registers Bank)

O Banco de Registradores é o módulo responsável por organizar e gerenciar um conjunto de registradores acessíveis simultaneamente pela CPU. No projeto, foram implementados oito registradores gerais, nomeados de R0 a R7, permitindo que diferentes valores sejam armazenados, lidos e manipulados de maneira independente durante a execução das instruções.

Para controlar qual registrador deve receber um novo valor, utiliza-se um decodificador 3x8, já apresentado em seções anteriores. Esse decodificador converte os três bits de seleção (DecodersC) em exatamente um sinal de habilitação, garantindo que somente um dos oito registradores seja escrito por vez.

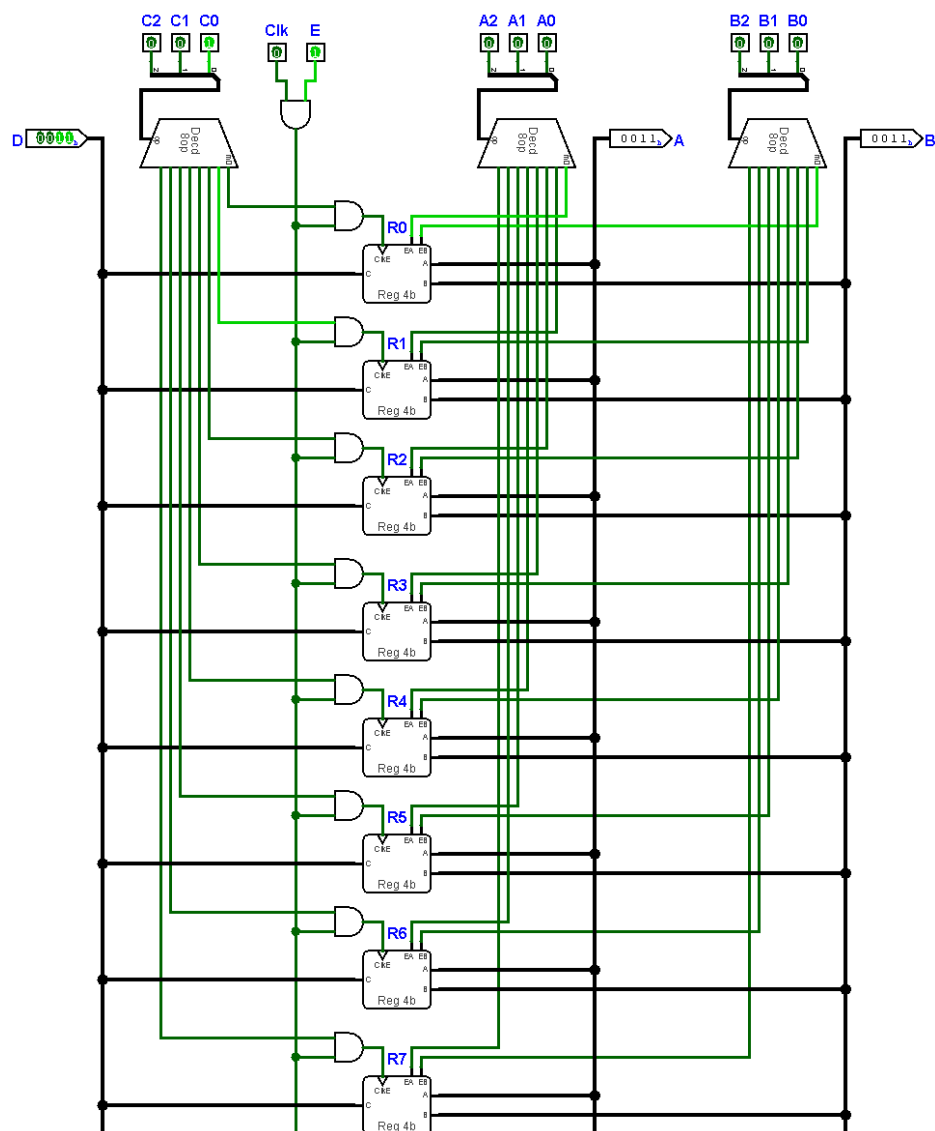
Além da seleção de escrita, o Banco de Registradores também oferece duas portas de leitura independentes, correspondentes às saídas A e B da CPU. Cada uma dessas saídas possui seu próprio circuito de controle, também baseado em decodificadores 3x8, que determinam de qual registrador cada porta deverá ler o valor armazenado. Assim, a CPU pode, por exemplo, enviar o conteúdo de R3 para a Saída A e simultaneamente o conteúdo de R7 para a Saída B, permitindo operações aritméticas e lógicas sem conflitos de barramento.

Cada registrador, assim como descrito anteriormente, utiliza Tri-States em suas saídas, ativando apenas o caminho selecionado enquanto mantém todos os demais registradores em estado de alta impedância (Hi-Z). Isso evita colisões no barramento e mantém o banco compacto, organizado e escalável.

Essa arquitetura segue o padrão clássico de um **Register File** de duas portas de leitura e uma porta de escrita, amplamente utilizado em processadores didáticos e pipelines simples.

## Circuito:

Figura 77 — Circuito: Banco de Registradores (Registers Bank)



## Bloco Macro

Figura 78 — Macro: Banco de Registradores (Registers Bank)



## 4 — DESENVOLVIMENTO DA ULA

### 4.1 — NAND

Um circuito NAND de 4 bits tem o objetivo de aplicar a operação lógica NAND bit a bit entre duas entradas, cada uma contendo 4 linhas de dados.

Para construir esse circuito de forma modular, utiliza-se o encadeamento de quatro portas NAND de 1 bit. Essa mesma lógica de encadeamento pode ser aplicada para expandir outras portas lógicas de 1 bit, como NOT, AND, OR, NOR, XOR e XNOR.

**Expressão:**

$$S3 = \overline{A3 \cdot B3}$$

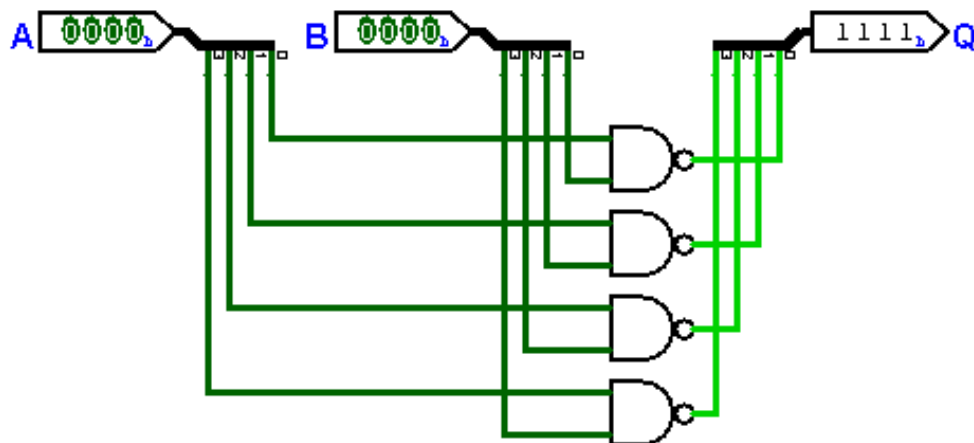
$$S2 = \overline{A2 \cdot B2}$$

$$S1 = \overline{A1 \cdot B1}$$

$$S0 = \overline{A0 \cdot B0}$$

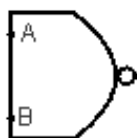
**Circuito:**

Figura 79 — Circuito: Nand 4 bits



**Bloco Macro:**

Figura 80 — Macro: Nand 4 bits



## 4.2 — NOR

Um circuito NOR de 4 bits tem o objetivo de aplicar a operação lógica NOR bit a bit entre duas entradas, cada uma contendo 4 linhas de dados.

**Expressão:**

$$S3 = \overline{A3 + B3}$$

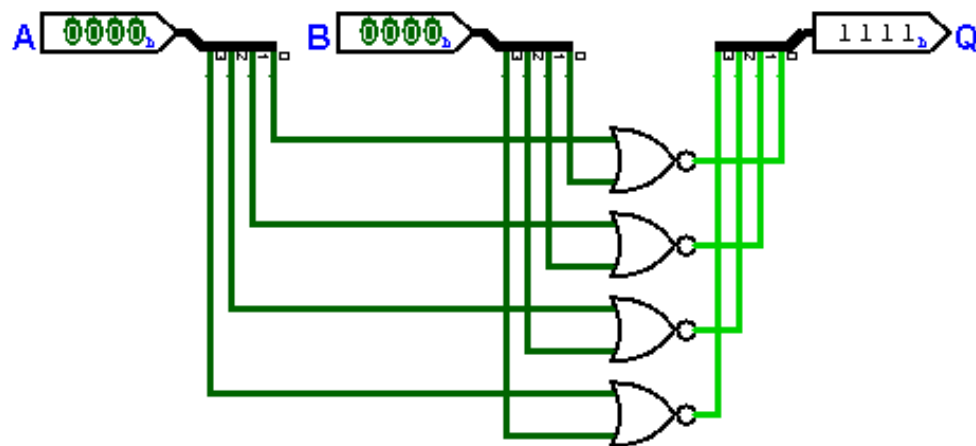
$$S2 = \overline{A2 + B2}$$

$$S1 = \overline{A1 + B1}$$

$$S0 = \overline{A0 + B0}$$

**Circuito:**

Figura 81 — Circuito: Nor 4 bits



**Bloco Macro:**

Figura 82 — Macro: Nor 4 bits



### 4.3 — XOR

Um circuito NOR de 4 bits tem o objetivo de aplicar a operação lógica NOR bit a bit entre duas entradas, cada uma contendo 4 linhas de dados.

**Expressão:**

$$S3 = A3 \oplus B2$$

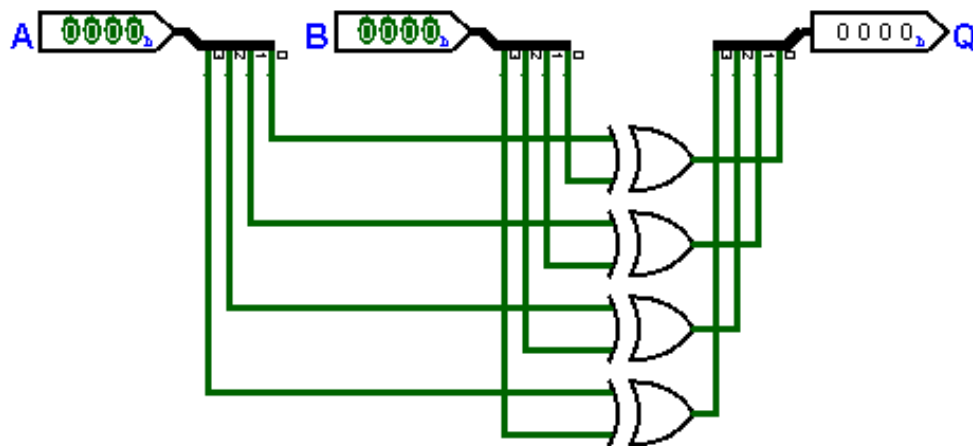
$$S2 = A2 \oplus B2$$

$$S1 = A1 \oplus B1$$

$$S0 = A0 \oplus B0$$

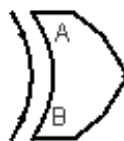
**Circuito:**

Figura 83 — Circuito: Xor 4 bits



**Bloco Macro:**

Figura 84 — Macro: Xor 4 bits





## 4.4 — XNOR

Um circuito NOR de 4 bits tem o objetivo de aplicar a operação lógica NOR bit a bit entre duas entradas, cada uma contendo 4 linhas de dados.

**Expressão:**

$$S3 = \overline{A3 \oplus B3}$$

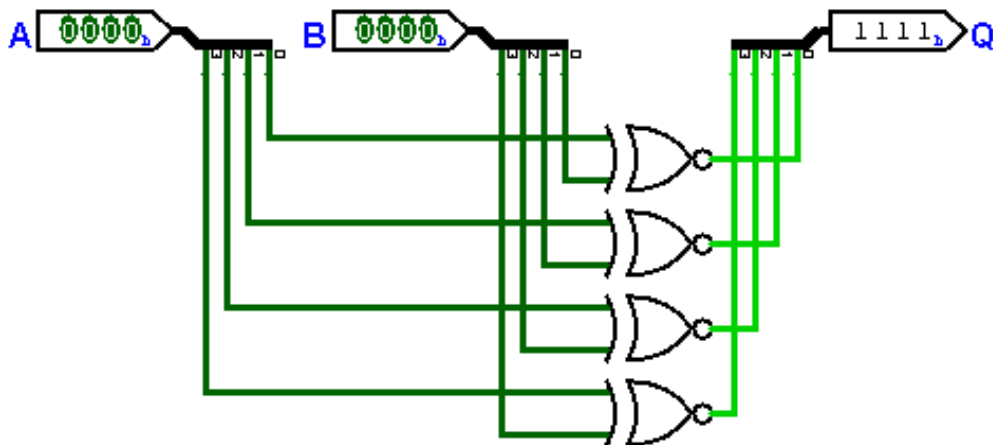
$$S2 = \overline{A2 \oplus B2}$$

$$S1 = \overline{A1 \oplus B1}$$

$$S0 = \overline{A0 \oplus B0}$$

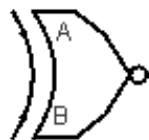
**Circuito:**

Figura 85 — Circuito: Xnor 4 bits



**Bloco Macro:**

Figura 86 — Macro: Xnor 4 bits



## 4.5 — Somador (Adder)

A soma na lógica digital é construída a partir de operações básicas entre bits. Como cada bit só pode assumir o valor 0 ou 1, o processo de adicionar dois bits funciona de forma semelhante à adição na matemática convencional, mas utilizando portas lógicas para determinar tanto o bit de resultado quanto o transporte para a próxima posição. O comportamento dessa soma começa de forma simples, considerando apenas dois bits, e depois é expandido para lidar com entradas maiores.

O primeiro passo para entender essa construção é observar o funcionamento do somador mais simples, conhecido como Half Adder. Esse circuito recebe dois bits e produz dois resultados independentes: o bit de soma e o bit de transporte. O bit de soma indica o resultado direto da adição dos dois valores, enquanto o transporte representa o “vai-um” que ocorre quando ambos os bits são 1. Esse circuito não recebe nenhum transporte, e por possuir uma XOR e uma AND em seu circuito, também serve como otimização para a expressão de um somador completo.

Para lidar com situações onde existe um transporte anterior, é usado o somador completo, também referenciado como Full Adder. Esse novo circuito considera três entradas: os dois bits que estão sendo somados e o transporte vindo da posição anterior. Com isso, ele consegue gerar tanto o bit de resultado quanto um novo transporte, que será encaminhado para a próxima posição da soma.

Com o Full Adder definido, torna-se possível construir somadores maiores simplesmente encadeando vários desses blocos. Esse método é chamado de ripple carry, pois o transporte se propaga de um estágio para o outro, passando pelo circuito inteiro. Cada Full Adder é responsável por somar um par de bits correspondentes, levando em conta o transporte que recebeu. Após calcular seu próprio resultado, ele gera um novo transporte, que “rippleia” (se espalha) para o próximo somador na cadeia.

Essa estrutura modular permite a construção de um somador de 4 bits utilizando quatro somadores completos conectados em série. O primeiro estágio recebe o transporte inicial, normalmente definido como zero, e calcula a soma do bit menos significativo. Seu transporte passa para o próximo somador, que utiliza essa informação para realizar seu próprio cálculo. Esse processo continua de forma sequencial até o último estágio, que produz o transporte final da operação, caso exista. Assim como as portas lógicas de 1 bit podem ser encadeadas, o somador também segue essa lógica de expansão natural.

Essa mesma lógica de controle, pode ser usada para montar um subtrator, porém, há um método mais eficiente, que usa o complemento da segunda entrada.

### 4.5.1 — Meio Somador (Adder Half)

Expressão:

$$S = A \oplus B$$

$$Co = A \cdot B$$

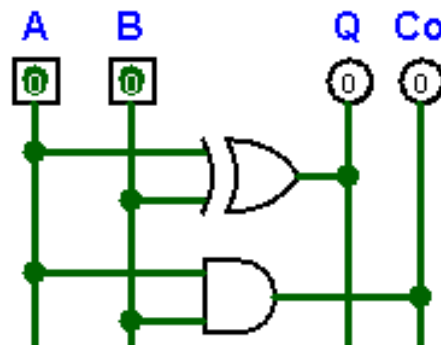
Tabela:

Tabela 29 — Meio Somador (Adder Half)

| A | B | S | Co |
|---|---|---|----|
| 0 | 0 | 0 | 0  |
| 0 | 1 | 1 | 0  |
| 1 | 0 | 1 | 0  |
| 1 | 1 | 0 | 1  |

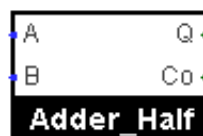
Circuito:

Figura 87 — Circuito: Meio Somador (Adder Half)



Bloco Macro:

Figura 88 — Macro: Meio Somador (Adder Half)



#### 4.5.2 — Somador Inteiro ou 1 bit (Full Adder)

Expressão:

$$S = A \oplus B \oplus Ci$$

$$Co = (A \cdot B) + (Ci \cdot (A \oplus B))$$

Essas expressões podem ser feitas de uma forma otimizada. Forma essa, que pode ser usada como base para montar um circuito mais eficiente:

$$T1 = A \oplus B$$

$$T2 = A \cdot B$$

$$T3 = Ci \cdot T1$$

$$S = T1 \oplus Ci$$

$$Co = T2 + T3$$

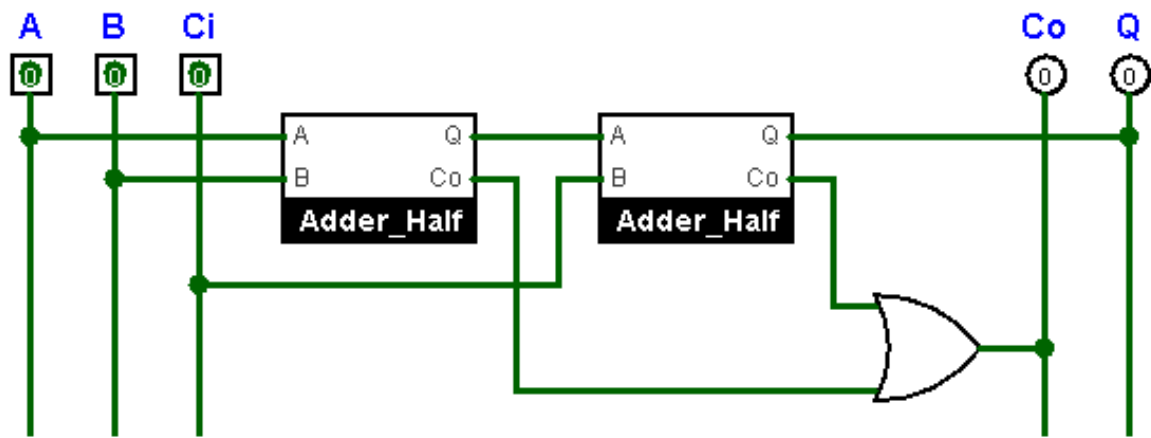
Tabela:

Tabela 30 — Meio Somador (Adder Half)

| A | B | Ci | S | Co |
|---|---|----|---|----|
| 0 | 0 | 0  | 0 | 0  |
| 0 | 0 | 1  | 1 | 0  |
| 0 | 1 | 0  | 1 | 0  |
| 0 | 1 | 1  | 0 | 1  |
| 1 | 0 | 0  | 1 | 0  |
| 1 | 0 | 1  | 0 | 1  |
| 1 | 1 | 0  | 0 | 1  |
| 1 | 1 | 1  | 1 | 1  |

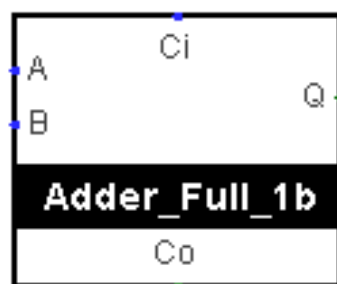
### Circuito:

Figura 89 — Circuito: Meio Somador (Adder Half)



### Bloco Macro:

Figura 90 — Macro: Meio Somador (Adder Half)



### 4.5.3 — Somador 4 bits usando Ripple-Carry (Adder)

#### Expressão:

O índice **i** representa a posição do bit, indo do menos significativo ( $i = 0$ ) ao mais significativo ( $i = 3$ ). Assim, **A<sub>i</sub>** e **B<sub>i</sub>** são os bits correspondentes das entradas A e B.

O transporte entre estágios é representado por **C**, onde **C<sub>i</sub>** é o carry que chega ao bit  $i$ , e **C<sub>i+1</sub>** é o carry que será enviado para o próximo estágio. Portanto, cada carry depende sempre do carry produzido no bit anterior.

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = (A_i \cdot B_i) + (C_i \cdot (A_i \oplus B_i))$$

E para saída da soma completa e do carry final:

$$S = (S_3 \cdot S_2 \cdot S_1 \cdot S_0)$$

$$Co = C_4$$

#### Tabela:

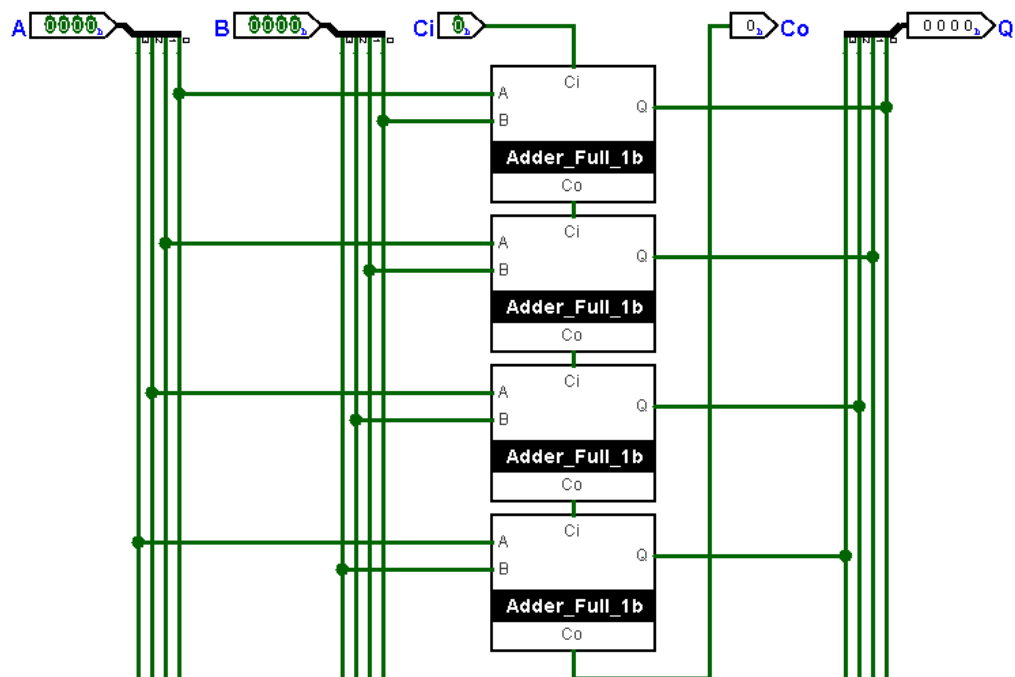
A tabela dessas operações geram variações gigantes, então apresenta-se um exemplo da tabela funcional, onde as linhas são os valores e os resultados.

Tabela 31 — Somador 4 bits (Exemplificativa)

| Entradas |      |                | Saídas |    |
|----------|------|----------------|--------|----|
| A        | B    | C <sub>i</sub> | S      | Co |
| 0101     | 0011 | 0              | 1000   | 0  |
| 1111     | 0001 | 0              | 0000   | 1  |
| 1010     | 0111 | 1              | 0010   | 1  |

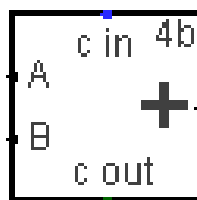
## Circuito:

Figura 91 — Circuito: Somador 4 bits



## Bloco Macro:

Figura 92 — Macro: Somador 4 bits



## 4.6 — Subtrator (Subtractor)

### 4.6.1 — Subtrator 4 bits usando Complemento (Subtractor)

A subtração em circuitos digitais segue um princípio semelhante ao da soma, porém utilizando uma transformação da entrada B. Como citado, o método adotado consiste em aplicar o complemento de dois em B, permitindo que a operação de subtração seja realizada como uma soma modificada. Para isso, a primeira etapa consiste em inverter todos os bits de B, algo facilmente obtido utilizando o circuito NAND de 4 bits já construído, configurado de modo a atuar como inversor para cada linha de dados.

Após a inversão, soma-se o valor 1 por meio do transporte inicial, completando a formação do complemento de dois. Com B transformado, o circuito passa a calcular  $A - B$  utilizando o mesmo somador empregado na operação de adição, sem a necessidade de componentes dedicados exclusivamente à subtração. Essa abordagem se mostra mais eficiente e modular, depois de expandir a soma, basta reajustar os valores no subtrator, tratando dois circuitos de uma vez.

Assim como na soma de múltiplos bits, o transporte se propaga entre os estágios conforme o método ripple carry. A diferença está no fato de que o primeiro carry-in não é zero, mas sim 1, garantindo a finalização do processo de complemento. Esse valor propagado, em subtratores, é chamado de borrow in e borrow out.

O resultado produzido representa o valor subtraído, enquanto o carry final pode atuar como indicador de empréstimo (borrow) ou ser ignorado, também pode ser utilizado como sinal de saída, que ao ser tratado, pode servir como um indicador de um número negativo.

#### **Expressão:**

A expressão é baseada no método utilizado, usando o complemento de B.

$$S = A - B = A + (\overline{B}) + 1$$



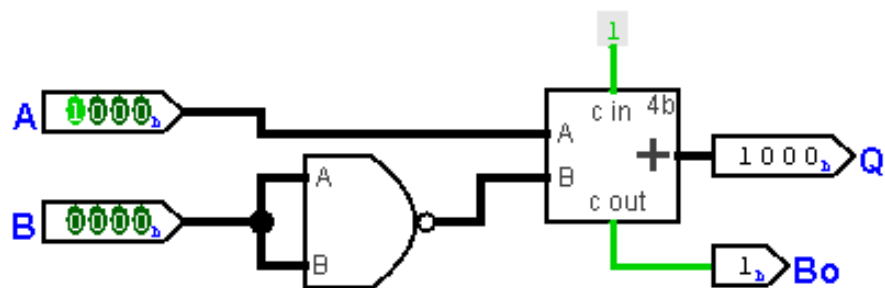
**Tabela:**

Tabela 32 — Subtrator 4 bits (Exemplificativa)

| Entradas |      |    | Saídas |    |
|----------|------|----|--------|----|
| A        | B    | Bi | S      | Bo |
| 1001     | 0011 | 0  | 0110   | 0  |
| 0101     | 1010 | 0  | 1011   | 1  |
| 0110     | 0110 | 0  | 0000   | 0  |

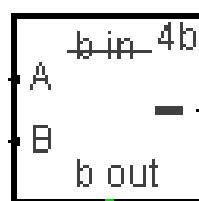
**Circuito:**

Figura 93 — Circuito: Subtrator 4 bits



**Bloco Macro:**

Figura 94 — Macro: Subtrator 4 bits



## 4.7 — Multiplicador (Multiplier)

### 4.7.1 — Multiplicador Combinacional 2 bits (Multiplier 2 bits)

O multiplicador de 2 bits recebe dois vetores  $A = (a_1 a_0)$  e  $B = (b_1 b_0)$ . Gera quatro produtos parciais por portas AND:

- $p_{00} = a_0 \cdot b_0$ ;
- $p_{01} = a_0 \cdot b_1$ ;
- $p_{10} = a_1 \cdot b_0$ ;
- $p_{11} = a_1 \cdot b_1$ .

Esses produtos são somados com deslocamentos apropriados para formar o produto  $P$  de 4 bits ( $P_3 P_2 P_1 P_0$ ). A lógica de combinação é implementada com somas de meio / somadores completos construídos a partir de portas lógicas (XOR/AND), conforme o circuito da imagem.

Outro ponto relevante a citar, é que o circuito faz uso de 4 portas Nand, para garantir que não seja multiplicado por 0, e após esse tratamento é possível ver a combinação de outras 2 portas And e 2 portas Xor, isso torna possível a utilização do Half Adder, uma vez que “S” do meio somador é um XOR e o “Co” é uma And, assim podendo reutilizar os macros do meio somador nesta situação.

Equações booleanas (saídas do  $2 \times 2$ ):

- $p_0 = p_{00}$ ;
- $S_1 = p_{01} \oplus p_{10}$  (soma na posição 1);
- $C_1 = p_{01} \cdot p_{10}$  (carry gerado ao somar  $p_{01}$  e  $p_{10}$ );
- $p_1 = S_1$ ;
- $S_2 = p_{11} \oplus C_1$  (soma na posição 2);
- $C_2 = p_{11} \cdot C_1$ ;
- $P_2 = S_2$ ;
- $P_3 = C_2$ .

**Expressão Final:**

$$P_0 = a_0 \cdot b_0$$

$$P_1 = (a_0 \cdot b_1) \oplus (a_1 \cdot b_0)$$

$$P_2 = (a_1 \cdot b_1) \oplus ((a_0 \cdot b_1) \cdot (a_1 \cdot b_0))$$

$$P_3 = (a_1 \cdot b_1) \cdot ((a_0 \cdot b_1) \cdot (a_1 \cdot b_0))$$

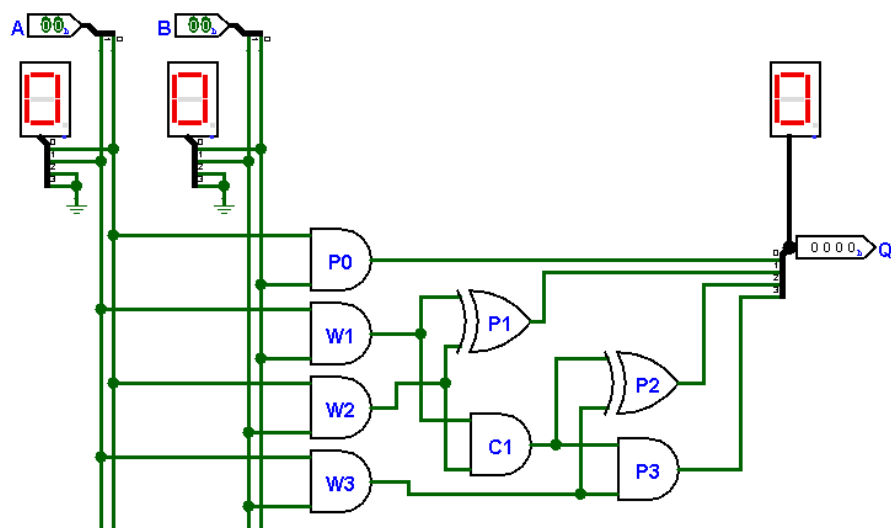
**Tabela:**

Tabela 33 — Multiplicador Combinacional 2 bits (Exemplificativa)

| A | B | S | Co |
|---|---|---|----|
| 0 | 0 | 0 | 0  |
| 0 | 1 | 1 | 0  |
| 1 | 0 | 1 | 0  |
| 1 | 1 | 0 | 1  |

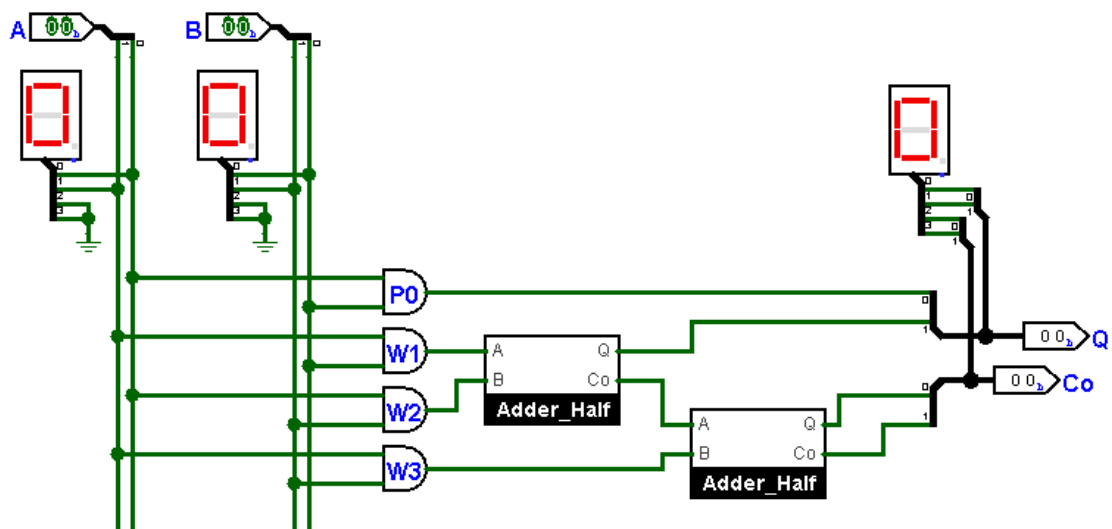
**Circuito Puro (e com saída de 4 bits):**

Figura 95 — Circuito Puro (e com saída de 4 bits): Multiplicador Combinacional 2 bits



## Circuito usando Half Adder:

Figura 96 — Circuito usando Half Adder: Multiplicador Combinacional 2 bits



## Bloco Macro:

Figura 97 — Macro: Multiplicador Combinacional 2 bits



#### **4.7.2 — Multiplicador 4 bits usando o Multiplicador de 2 bits**

A ideia para ampliar um multiplicador é repetir o mesmo princípio usado no caso multiplicador de 2 bits, gerar produtos parciais e somá-los de forma organizada.

Cada bit do multiplicando é multiplicado por cada bit do multiplicador por meio de ANDs, e esses resultados são alinhados conforme sua posição (deslocamento). Depois, eles são combinados usando Half Adder's e Full Adder's, exatamente como já mostrado no multiplicador de 2 bits.

Este conceito usa alguns princípios de deslocamento de bits usando Shifters, algo que para o projeto preferiu-se não se adotar, então foi procurado um outro método para incluir a multiplicação de 4 bits, usando o método do "Array Multiplier".

### 4.7.3 — Multiplicador 4 bits usando Array Multiplier (Multiplier 4 bits)

A multiplicação binária em circuitos digitais pode ser implementada de forma inteiramente combinacional por meio do método dos produtos parciais. Esse método consiste em decompor a multiplicação em várias operações elementares de AND e adição, o que permite construir o circuito a partir de blocos já desenvolvidos anteriormente no projeto, como o somador completo de 1 bit e o somador de 4 bits utilizado na ULA. O funcionamento básico do multiplicador parte do princípio de que cada bit do operando B é associado a uma linha de produtos parciais, obtidos pela operação AND entre esse bit e todos os bits do operando A. Dessa forma, formam-se quatro conjuntos de produtos parciais, cada um representando a multiplicação de A por um único bit de B.

Após a geração dos produtos parciais, cada linha é deslocada logicamente para a esquerda conforme sua posição na palavra binária de B, de modo equivalente ao procedimento manual de multiplicação. Esses valores deslocados são então somados entre si, operação realizada por meio de uma rede de somadores completos conectados em cascata. Essa rede utiliza diretamente o somador já construído anteriormente no projeto, garantindo uniformidade entre os módulos da CPU. A combinação entre portas AND e esses somadores permite acumular os resultados de cada linha de produtos parciais, resultando em um valor final de 8 bits correspondente ao produto entre as duas entradas de 4 bits. Todo o processo ocorre de forma puramente combinacional, sem necessidade de armazenamento intermediário, utilizando exclusivamente lógica previamente desenvolvida.

#### Expressão:

A expressão usa as regras matemáticas para apresentação.

$$P = A \times B = \sum_{j=0}^3 (A \cdot B_j) \ll j$$

- **P**: Representa o produto final da multiplicação entre os dois operandos A e B. É o resultado de 8 bits produzido pelo multiplicador.
- **$\Sigma$  (Sigma)**: Indica somatório. Significa que a expressão está somando vários termos de forma sequencial, dentro de um intervalo definido para o índice “j”.
- **j**: É o índice do somatório. Nesse caso, “j” representa a posição do bit do operando B que está sendo utilizado para gerar uma linha de produtos parciais. Assim, “j” varia de 0 a 3 para um operando de 4 bits.
- **$\ll$  (deslocamento à esquerda)**: O operador “ $\ll j$ ” significa deslocar o valor para a esquerda “j” posições. Equivale a multiplicar por  $2^j$ . No contexto do multiplicador, isso representa o deslocamento das linhas de produtos parciais, do mesmo modo que ocorre na multiplicação manual.
- **$B_i$  ou  $B_j$** : Representa o bit específico do operando B.

### Tabela:

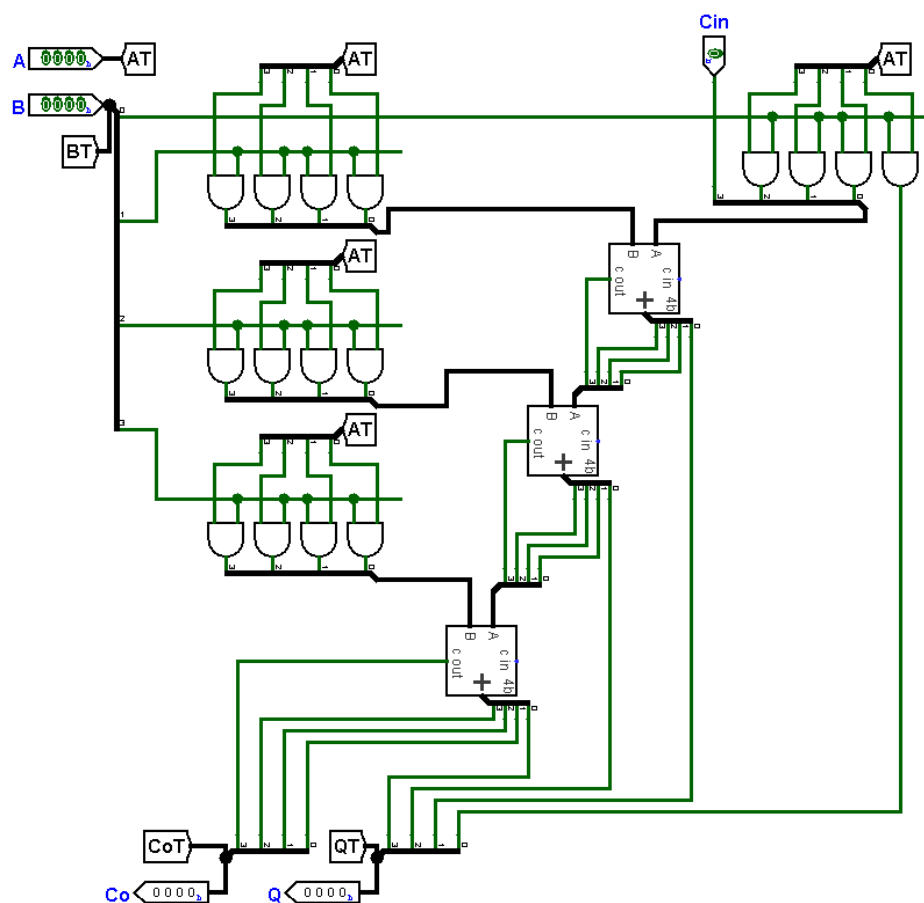
O circuito funciona recebendo 2 entradas de 4 bits, conseguindo devolver um resultado de até 8 bits, a saída “S” são os primeiros 4 bits (bit 0 até bit 3) do resultado final, e o “Co” é os restantes dos bits (bit 4 até bit 7).

Tabela 34 — Multiplicador 4 bits (Exemplificativa)

| Entradas |      |    | Saídas |      |
|----------|------|----|--------|------|
| A        | B    | Ci | S      | Co   |
| 0101     | 0011 | 0  | 1111   | 0000 |
| 1111     | 0010 | 0  | 1110   | 0001 |
| 1001     | 1001 | 0  | 0001   | 0100 |

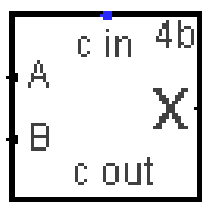
### Circuito:

Figura 98 — Circuito: Multiplicador 4 bits



## Bloco Macro:

Figura 99 — Macro: Multiplicador 4 bits





## 4.8 — Divisor (Divider)

Devido ao caráter didático do projeto e ao fato de a divisão não constituir o foco principal da arquitetura desenvolvida, optou-se por não implementar manualmente os métodos de divisão na CPU. Em vez disso, foram apenas estudados os principais métodos utilizados para sua realização, priorizando a compreensão conceitual sem a necessidade de construir toda a lógica correspondente.

### 4.8.1 — Método 1: Divisor teórico

Um divisor aritmético pode ser implementado de forma puramente combinacional e sequencial através do processo tradicional de divisão binária, semelhante ao método de “divisão por subtrações sucessivas” ou da divisão longa binária. A ideia central é comparar o dividendo com o divisor, subtrair quando possível e deslocar bits até que todo o dividendo tenha sido processado. A cada etapa, determina-se se o divisor “cabe” no trecho atual do dividendo; quando cabe, registra-se um 1 no quociente e subtrai-se o divisor, quando não cabe, registra-se um 0. Esse processo segue até esgotar todos os bits analisáveis, produzindo o quociente final e o resto associado. Esse método é teoricamente simples, porém exige diversas operações internas, como comparações, deslocamentos e subtrações.

### 4.8.2 — Método 2: Divisor com Flip-Flops / circuitos sequencial

Uma implementação mais prática em hardware usa uma arquitetura sequencial com registradores e flip-flops. Nesse arranjo, um registrador armazena o dividendo (ou parte dele), outro mantém o divisor, e cada passo da divisão é controlado por um circuito que desloca, compara e subtrai de forma iterativa. Flip-flops são usados para armazenar estados intermediários, permitindo que o circuito execute a divisão em múltiplos ciclos de clock. A lógica verifica continuamente se a subtração entre o conteúdo parcial e o divisor é possível; quando é, o resultado parcial é atualizado e o bit correspondente do quociente é armazenado. Esse método é mais eficiente em termos de área do circuito e espelha o comportamento de divisores reais utilizados em CPUs simples.

### 4.8.3 — Método 3: Divisor do Logisim (utilizado na ULA)

No projeto da ULA, optou-se por utilizar o componente de divisão nativo do Logisim. Diferentemente dos módulos anteriores, que foram construídos manualmente, o divisor da ferramenta já encapsula toda a lógica interna e apresenta uma interface simples: duas entradas, A (dividendo) e B (divisor), e três saídas principais.

A saída padrão, geralmente identificada apenas como “/” ou  $\div$ , corresponde ao quociente da divisão inteira entre A e B. Além dela, o componente oferece a saída REM, que representa o resto da operação, e a saída Upper, que corresponde aos bits mais altos do produto intermediário utilizado na operação interna do divisor. Na prática, o Upper raramente é necessário em CPUs básicas e pode ser ignorado em

arquiteturas que não dependem dos detalhes da divisão parcial; já o REM é útil quando se deseja implementar operações modulares.

**Tabela:**

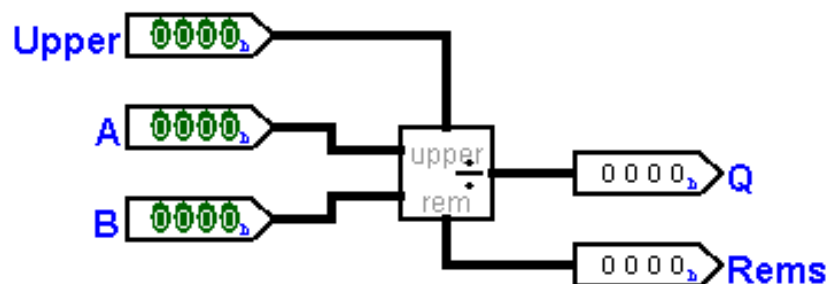
Tabela 35 — Divisor 4 bits (Exemplificativa)

| Entradas |      | Saídas |      |
|----------|------|--------|------|
| A        | B    | S      | Rem  |
| 1100     | 0011 | 0100   | 0000 |
| 1110     | 0101 | 0010   | 0100 |
| 1001     | 0010 | 0100   | 0001 |

**Circuito (usando o do Logisim):**

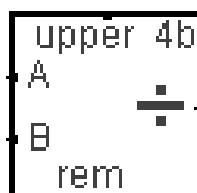
(A saída do resto “Rem” teve que ser nomeado como “Rems”, pois a ferramenta não permite nomear como “Rem”)

Figura 100 — Circuito: Multiplicador 4 bits



**Bloco Macro:**

Figura 101 — Macro: Multiplicador 4 bits



## 4.9 — Junção na ULA

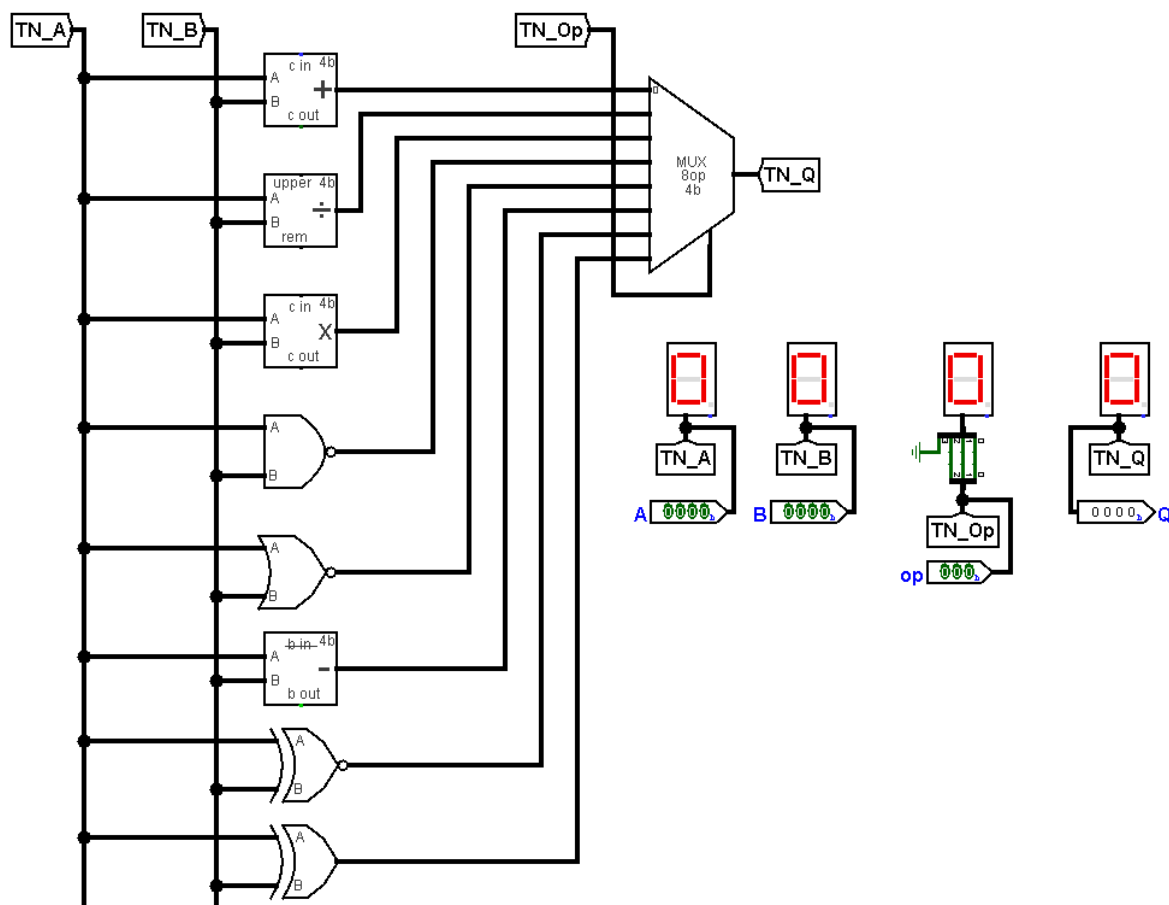
Finalmente, para concluir a construção da ULA de 4 bits e 8 operações, basta realizar as conexões entre os Barramentos de entrada “A” e “B”, internamente, conectá-los da forma esperada em cada um das operações, a saída das operações, conectá-las nas entradas do Mux de 4 bits e 8 operações criado, respeitando a ordem já mencionada, em seguida, utilizar a entrada de 3 bits para escolher as operações conectando-as no Mux, e finalizar com ligando a saída do multiplexador em pinos de saídas de dados.

Nesta construção, não foi explorado o uso de sinais de saídas, algo que pode ser implementado, graças à estrutura criada para os circuitos.

### Circuito:

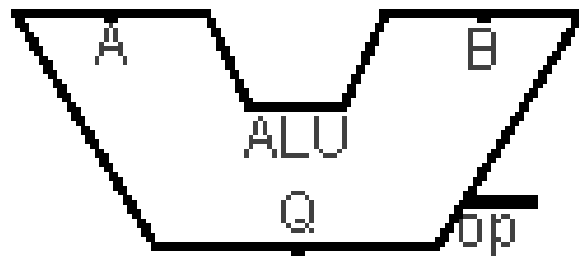
É possível ver o uso dos Tunnel 's para uma visualização mais intuitiva do resultado.

Figura 102 — Circuito: ULA Final



**Bloco Macro:**

Figura 103 — Macro: ULA Final



## 5 — DESENVOLVIMENTO DOS MÓDULOS PARA A CPU

### 5.1 — MIR (Microinstruction Register)

O MIR (Microinstruction Register) é o registrador responsável por armazenar a microinstrução atualmente em execução pela CPU. Cada microinstrução corresponde a um conjunto de sinais de controle que definem o comportamento completo do ciclo em andamento — como seleção de registradores, operação da ULA, habilitação do MBR, manipulação do MAR, acionamento do shifter, escrita na memória, leitura, entre outros. Assim, o MIR funciona como o "núcleo de controle imediato" da arquitetura, convertendo a instrução macro em ações específicas no nível de hardware.

No projeto, o MIR foi implementado como um registrador de 24 bits, contendo os sinais de controle necessários para o funcionamento completo da CPU. Entretanto, apenas 20 desses bits são efetivamente utilizados, visto que quatro deles correspondem a saídas sem função na versão final da arquitetura. Esses quatro bits permanecem fisicamente presentes (devido ao padrão estrutural do registrador), mas não estão conectados a nenhum módulo interno, mantendo-se isolados e sem impacto no funcionamento.

O MIR foi implementado utilizando Flip-Flops D fornecidos pela ferramenta.

#### 5.1.1 — Lógica de organização e ordem dos bits

A organização dos 24 bits do MIR não é arbitrária: ela segue uma ordem lógica baseada na funcionalidade e no agrupamento natural dos sinais de controle. Assim, a definição final da sequência  $Q0 \rightarrow Q23$  leva em conta:

- **Agrupamento por função** (ex.: todas as linhas do Decoder C juntas, todas do Decoder A juntas, etc.)
- **Simplicidade de cabeamento** dentro do projeto, reduzindo cruzamentos e facilitando o roteamento interno
- **Proximidade física** com os blocos controlados
- **Hierarquia de impacto** (bits mais críticos ou mais recorrentes aparecem mais próximos do início da palavra)

#### 5.1.2 — Ordem dos Bits

A lista final dos bits segue a convenção:

- **Q0** = bit menos significativo (LSB);
- **Q23** = bit mais significativo (MSB).

Somente **Q0** a **Q19** possuem função.

Os bits **Q20** a **Q23** existem fisicamente, mas não são conectados.

**Ordem dos bits do MIR:**

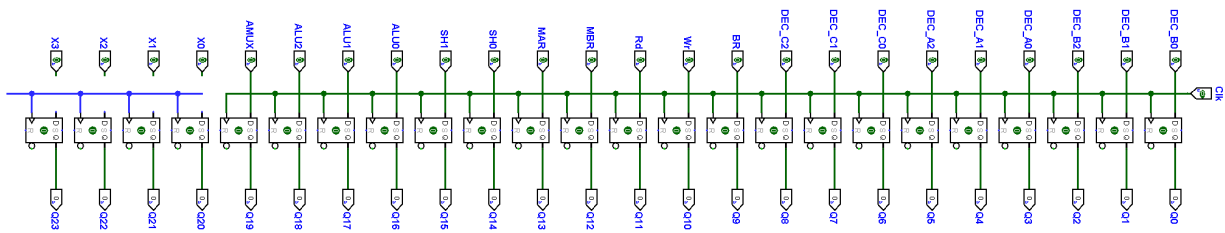
Tabela 36 — Ordem dos Bits

| Bit | Sinal   | Função                                          |
|-----|---------|-------------------------------------------------|
| Q0  | DECD_B0 | Seleção da Saída B (bit 0)                      |
| Q1  | DECD_B1 | Seleção da Saída B (bit 1)                      |
| Q2  | DECD_B2 | Seleção da Saída B (bit 2)                      |
| Q3  | DECD_A0 | Seleção da Saída A (bit 0)                      |
| Q4  | DECD_A1 | Seleção da Saída A (bit 1)                      |
| Q5  | DECD_A2 | Seleção da Saída A (bit 2)                      |
| Q6  | DECD_C0 | Seleção de escrita no Banco Registrador (bit 0) |
| Q7  | DECD_C1 | Seleção de escrita no Banco Registrador (bit 1) |
| Q8  | DECD_C2 | Seleção de escrita no Banco Registrador (bit 2) |
| Q9  | BR      | Habilita escrita no Banco de Registradores      |
| Q10 | Wr      | Solicitar escrita na memória externa            |
| Q11 | Rd      | Solicitar leitura da memória externa            |
| Q12 | MBR     | Habilitação do MBR                              |
| Q13 | MAR     | Habilitação de escrita no MAR                   |
| Q14 | SH0     | Shifter — linha de controle 0                   |
| Q15 | SH1     | Shifter — linha de controle 1                   |
| Q16 | ULA0    | Controle da ULA – bit 0                         |
| Q17 | ULA1    | Controle da ULA – bit 1                         |
| Q18 | ULA2    | Controle da ULA – bit 2                         |
| Q19 | AMUX    | Seleciona qual será a entrada A da ULA          |

|     |    |               |
|-----|----|---------------|
| Q20 | X1 | Bit 0 sem uso |
| Q21 | X1 | Bit 1 sem uso |
| Q22 | X2 | Bit 2 sem uso |
| Q23 | X3 | Bit 3 sem uso |

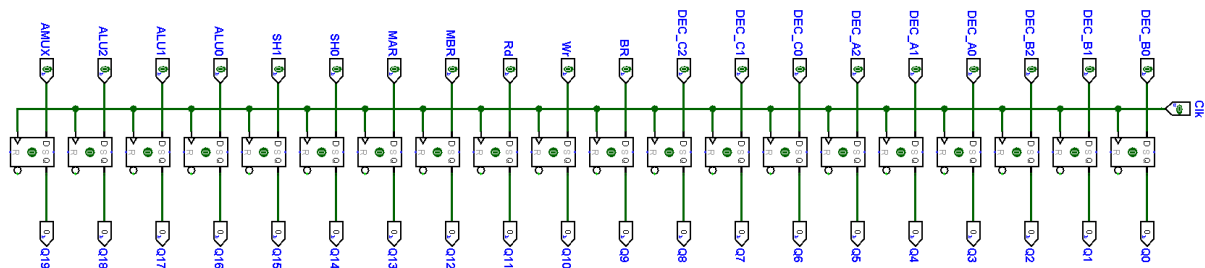
Circuito Completo:

Figura 104 — Circuito Completo: MIR



Circuito Recordado:

Figura 105 — Circuito Recordado: MIR



Bloco Macro

Figura 106 — Macro: MIR



## 5.2 — Banco MIR (Microinstruction ROM)

Para alimentar o MIR com as microinstruções necessárias ao funcionamento da CPU, foi utilizado um banco de microinstruções baseado na ROM nativa do Logisim. Esse banco funciona como o armazenamento permanente das microinstruções, garantindo que cada instrução macro do processador seja decomposta em uma sequência definida de operações de controle.

No projeto, utilizou-se uma **ROM de 32 palavras por 24 bits (32×24)**, dimensão suficiente para armazenar todas as microinstruções requeridas pela arquitetura. Seu tamanho foi definido considerando o conjunto total de instruções suportadas e os estados internos necessários para cada uma delas. A seleção da microinstrução é feita por meio de um endereço de 5 bits, permitindo acessar diretamente qualquer uma das 32 posições disponíveis.

Cada palavra da ROM corresponde a uma microinstrução completa, com a mesma largura do MIR — ou seja, 24 bits contendo todos os sinais de controle. Quando o endereço é fornecido, a ROM disponibiliza imediatamente essa microinstrução na saída, que é então carregada no MIR através dos Flip-Flops D utilizados na implementação. Dessa forma, o banco MIR funciona como o "microprograma" da CPU, armazenando a lógica de controle que descreve o comportamento interno de cada instrução.

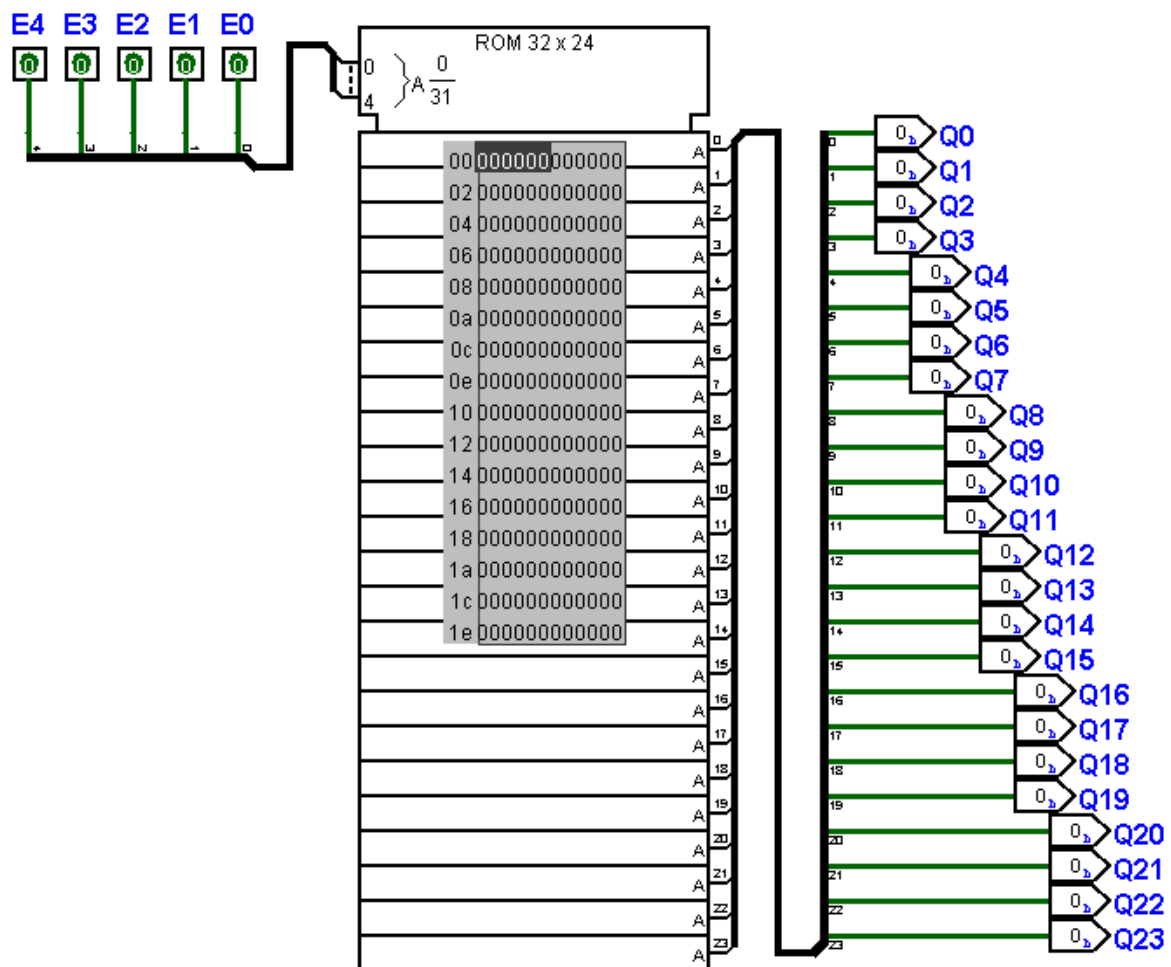
Esse arranjo garante previsibilidade e padronização: a ROM fornece as instruções previamente definidas, e o MIR mantém em execução a microinstrução corrente. Com isso, separa-se claramente o papel do armazenamento (ROM) e o papel da execução imediata (MIR), organizando o fluxo de controle de forma modular e simples de interpretar.

Este micro programa pode também pode ser nomeado como "PROM" em alguns lugares, mas para este relatório sempre será referenciado diretamente como banco MIR.



## Circuito:

Figura 107 — Circuito: Banco MIR



## Bloco Macro

Figura 108 — Macro: Banco MIR



## 5.1.1 — Mapeamento

### 5.1.1.1 — Mascara Hex (Hex Mask)

Essas são as máscaras obtidas para realizar instruções específicas:

- Q0 DECD\_B0 = 0x000001
- Q1 DECD\_B1 = 0x000002
- Q2 DECD\_B2 = 0x000004
- Q3 DECD\_A0 = 0x000008
- Q4 DECD\_A1 = 0x000010
- Q5 DECD\_A2 = 0x000020
- Q6 DECD\_C0 = 0x000040
- Q7 DECD\_C1 = 0x000080
- Q8 DECD\_C2 = 0x000100
- Q9 BR = 0x000200
- Q10 Wr = 0x000400
- Q11 Rd = 0x000800
- Q12 MBR = 0x001000
- Q13 MAR = 0x002000
- Q14 SH0 = 0x004000
- Q15 SH1 = 0x008000
- Q16 ULA0 = 0x010000
- Q17 ULA1 = 0x020000
- Q18 ULA2 = 0x040000
- Q19 AMUX = 0x080000
- Q20 X0 = 0x100000 (no use)
- Q21 X1 = 0x200000 (no use)
- Q22 X2 = 0x400000 (no use)
- Q23 X3 = 0x800000 (no use)

### 5.1.1.2 — Palavras de endereço na formatação (ROM WordAddress)

Essas são as máscaras obtidas para realizar instruções específicas:

v3.0 hex words addressed

00: 000a01 000a41 000a81 000ac1 000b01 000b41 000b81 000bc1

08: 010201 020201 040201 080201 100201 200201 400201 000201

10: 000a00 000a02 000a04 000a08 000a10 000a20 000a40 000200

18: 001200 001a80 001a40 001a00 001400 000200 000200 000200

## 5.1.2 — Instruções Configuradas

### Instruções Humanas

#### Comandos Aritméticos

- 00 (00000) Add R0,R1
- 01 (00001) Div R0,R1
- 02 (00010) Mul R0,R1
- 03 (00011) Nand R0,R1
- 04 (00100) Nor R0,R1
- 05 (00101) Sub R0,R1
- 06 (00110) Xnor R0,R1
- 07 (00111) Xor R0,R1

#### Comandos de R0 para Registradores

- 08 (01000) Mov R0,R1
- 09 (01001) Mov R0,R2
- 0A (01010) Mov R0,R3
- 0B (01011) Mov R0,R4
- 0C (01100) Mov R0,R5
- 0D (01101) Mov R0,R6
- 0E (01110) Mov R0,R7
- 0F (01111) NOP / Mov R0,R0

#### Comandos de Registradores para R0

- 10 (10000) Mov R1,R0
- 11 (10001) Mov R2,R0
- 12 (10010) Mov R3,R0
- 13 (10011) Mov R4,R0
- 14 (10100) Mov R5,R0
- 15 (10101) Mov R6,R0
- 16 (10110) Mov R7,R0

17 (10111) NOP / Mov R0,R0

### **Comandos de Comunicação**

18 (11000) Mov R2,MBR

19 (11001) Mov R1,MBR

1A (11010) Mov R0,MBR

1B (11011) In

1C (11100) Out

1D (11101) NOP

1E (11110) NOP

1F (11111) NOP

### **5.1.3 — Aviso de teste**

Nem todas as operações foram possíveis de configurar ou testar, e podem apresentar problemas.

Para isso, foi incluído no projeto um circuito da CPU sem o uso do Banco MIR para testes.

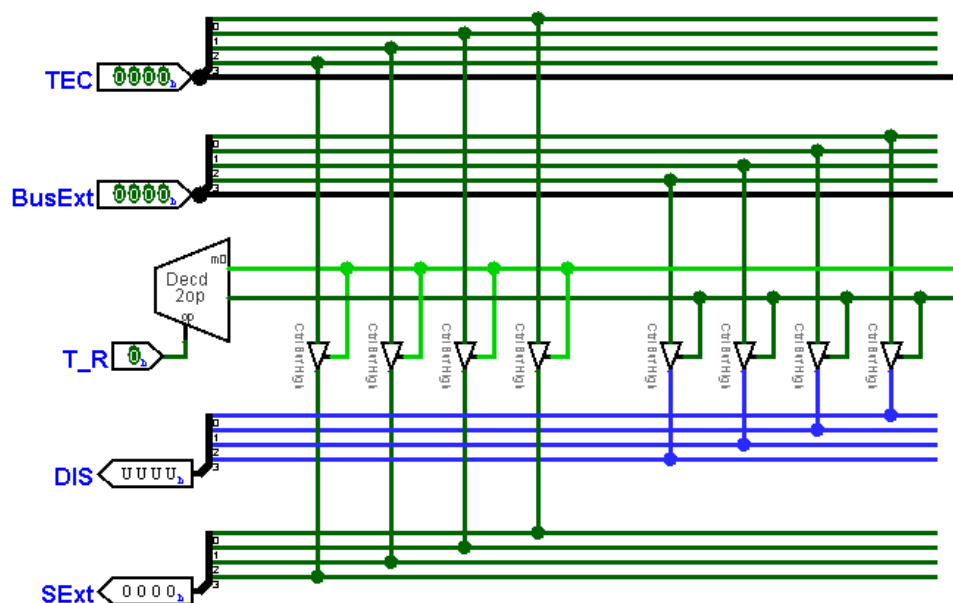
### 5.3 — RxTx (Receiver/Transceiver)

O módulo RxTx é responsável por controlar o fluxo de dados entre a CPU e a memória, definindo quando o sistema está realizando uma operação de leitura (read) ou de escrita (write). Para isso, o circuito utiliza um decoder 1×2, que recebe um único sinal de controle e o converte em duas linhas mutuamente exclusivas: uma habilitando a leitura e outra habilitando a escrita. Dessa forma, garante-se que apenas uma das operações esteja ativa por vez, evitando conflitos no barramento de dados.

Como já comentado, em diagramas — geralmente — os fios que enviam e recebem dados do MBR são os mesmos, porém, por conta de limitações da ferramenta, o circuito foi adaptado e separado como entrada e saída. A lógica para liberação continua igual, mesmo que não haja conflito a lógica foi mantida para manter a fidelidade e simular corretamente o circuito.

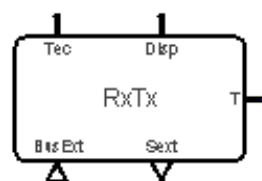
**Circuito:**

Figura 109 — Circuito: RxTx



**Bloco Macro**

Figura 110 — Macro: RxTx



### 5.3 — Clock ou Sub-ciclos

A CPU desenvolvida utiliza um sistema de clock dividido em quatro fases internas, todas derivadas do mesmo sinal base, mas defasadas entre si. Cada fase corresponde a um subciclo do clock e é responsável por ativar um conjunto específico de componentes, garantindo que as operações ocorram em ordem e evitando conflitos no barramento. Essa estratégia permite sequenciar corretamente as ações internas da microinstrução executada pelo MIR.

As fases são executadas sempre na mesma ordem, permitindo que cada parte da CPU receba o pulso de clock apenas no instante adequado. No projeto, a sequência operacional segue esta ordem lógica:

1. **Fase do MIR** — mantém o clock do MIR ativo após o reset, garantindo que cada microinstrução seja carregada no momento correto.
2. **Fase do Latch A/B** — atualiza os valores utilizados nas operações internas da ULA.
3. **Fase do MAR** — grava o endereço que será utilizado em uma operação de leitura ou escrita.
4. **Fase do MBR e dos Registradores** — captura os dados vindo do RxTx ou envia os dados de volta ou armazena no banco registrador.

Essa organização garante que os dados fluam de forma controlada pelo sistema, evitando que diferentes módulos tentem alterar o mesmo barramento simultaneamente.

#### 5.3.1 — Configuração dos Clocks no Logisim

Tabela 37 — Ordem dos Bits

| Nome         | Fase | High Duration | Low Duration | Phase Offset |
|--------------|------|---------------|--------------|--------------|
| MIR          | 1°   | 1             | 3            | 3            |
| LATCH        | 2°   | 1             | 3            | 2            |
| MAR          | 3°   | 1             | 3            | 1            |
| MBR/REGISTER | 4°   | 1             | 3            | 0            |

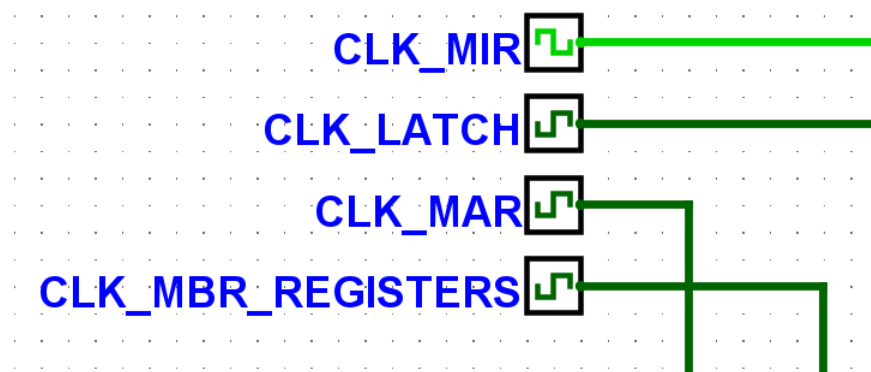
Essas configurações podem ser explicadas da seguinte forma:

- **High Duration (duração em nível alto):** Define por quantas unidades de tempo o clock permanece em nível lógico '1'. No projeto, cada fase fica ativa por 1 unidade.

- **Low Duration (duração em nível baixo):** Define por quantas unidades de tempo o clock permanece em nível '0'. O valor 3 garante que cada clock só volte a ficar alto após as demais fases já terem sido executadas.
- **Phase Offset (deslocamento de fase):** Determina o atraso inicial do clock em relação ao ciclo base. Um offset maior significa que aquele clock "espera" mais antes de gerar seu primeiro pulso alto. Assim, os quatro clocks são distribuídos uniformemente dentro do ciclo completo (total = 4 unidades), criando quatro sub-ciclos sequenciais perfeitamente alinhados.

**Circuito:**

Figura 111 — Circuito: Clock



O Clock em si não possui um macro, e suas ligações são feitas diretamente e internamente no circuito da CPU.

### 5.3 — Input e Output de Dados

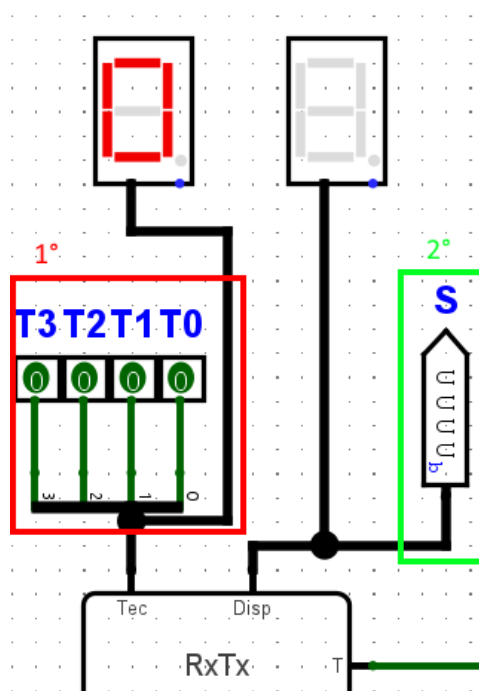
O “input” é a área responsável por inserir os dados que serão trabalhados dentro do processador, ou seja, desde de números para operações aritméticas, lógicas, uso do shifter, guardar algo no banco registrador e outras. Enquanto o “output”, é o que o processador devolve para saída de dados. O input será um teclado, ou seja, 4 pinos de 1 bit cada, que são juntados e enviados para o RxTx, e o output é um pino de saída de 4 bits, ambos possuem um display hex para melhor visualização.

#### Circuito:

A área destacada em vermelho, e legendada como “1°” é a parte em si de entrada das informações, o “Teclado”, onde é possível ver sua conexão com o RxTx, e a área destacada em verde, e legendada como “2°” é o pino de saída de dados.

O logisim permite inserir um label nos pinos, “S” se refere a saída e “T3, T2, T1, T0” aos pinos de entrada, onde T3 é o bit de maior peso e T0 o de menor peso.

Figura 112 — Circuito: Input e Output de Dados



A entrada e saída de dados não possuem um macro, e suas ligações são feitas diretamente e internamente no circuito da CPU.



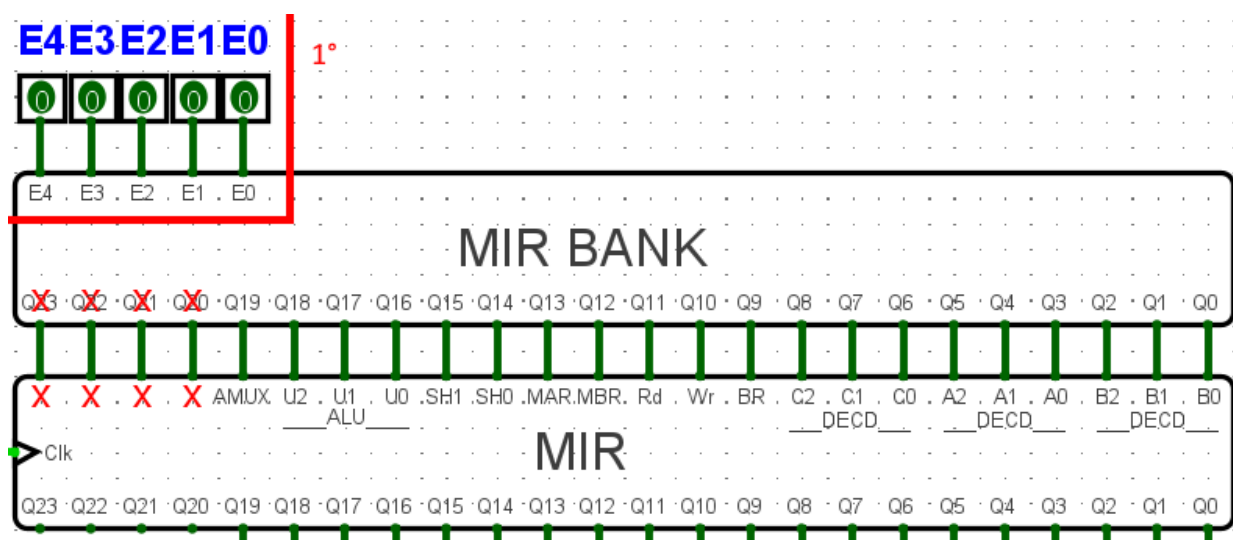
### 5.3 — Entrada de Instruções

A entrada de instruções é a parte da CPU onde é inserido as instruções de qual operação será feita pelo processador, não operações referente a ULA, e sim quaisquer tipos de operações configuradas, como receber informações (In), devolver informações (Out), escrever dados em um dos registradores, mover dados de um registrador para outro, e também realizar alguma operação da ULA ou do Shifter.

#### Circuito:

A área destacada em vermelho, e legendada como “1°” é a parte em si de entrada de instruções, onde é possível ver sua conexão com o banco MIR.

Figura 113 — Circuito: Entrada de Instruções



A entrada das instruções não possuem um macro, e suas ligações são feitas diretamente e internamente no circuito da CPU.

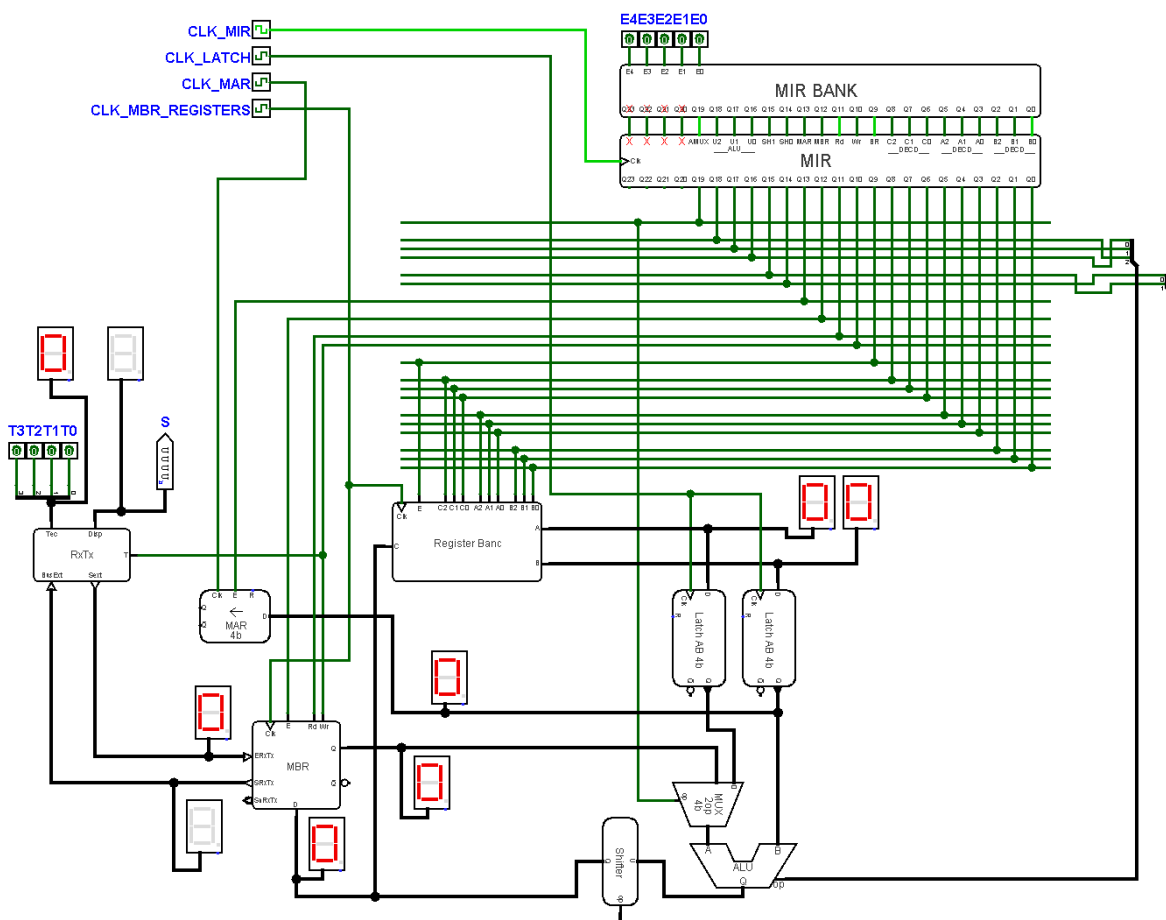
## 6 — RESULTADO DA CPU

Após conectar os módulos construídos, obtemos o resultado final da construção da CPU.

Foram implementados 2 circuitos, com o Banco MIR incluído na construção assim possuído 5 bits de operação, e outro sem o Banco MIR, apenas com os 20 bits de operações.

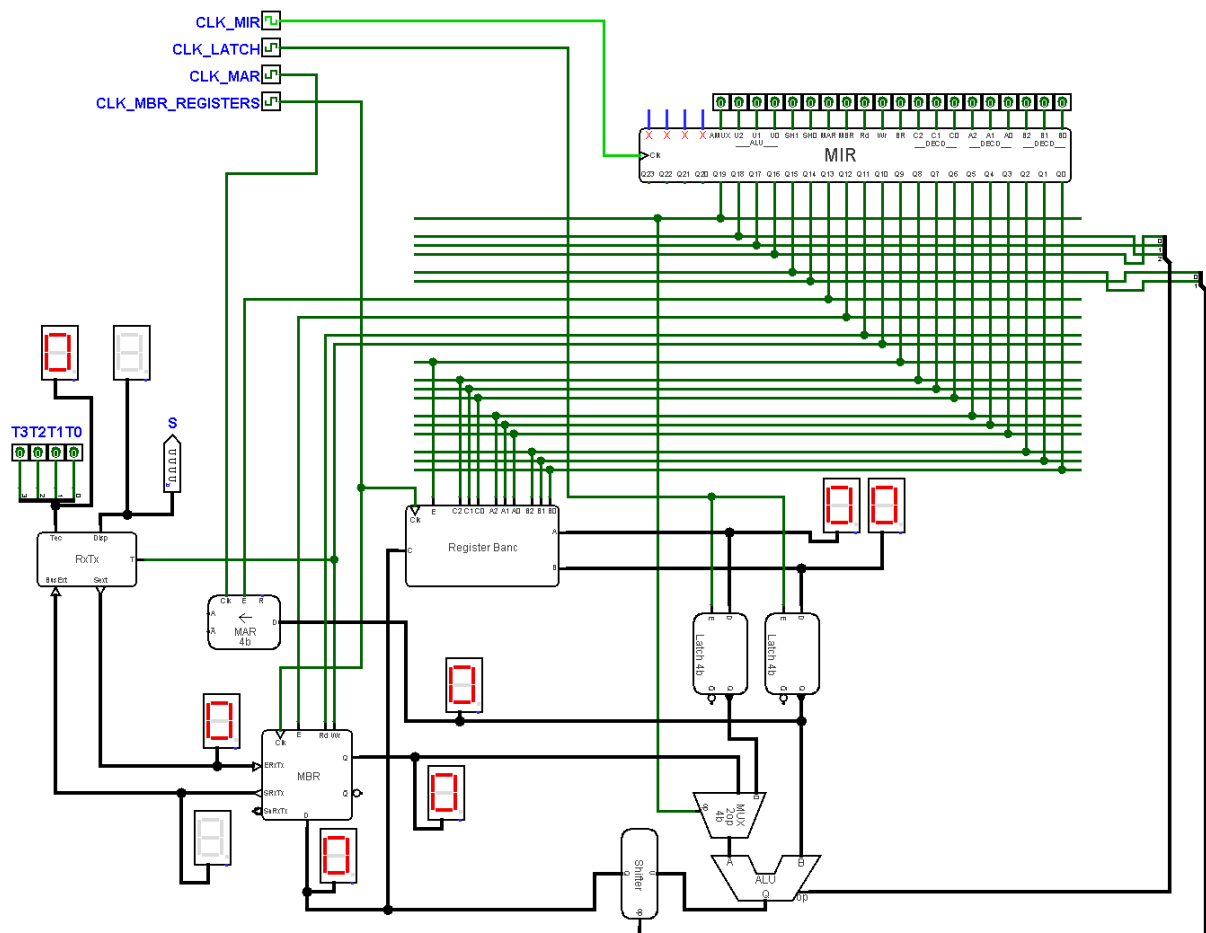
### Circuito com Banco MIR:

Figura 114 — Circuito: CPU com Banco MIR



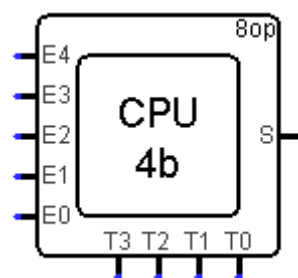
## Circuito sem Banco MIR:

Figura 115 — Circuito: CPU sem Banco MIR



## Bloco Macro

Figura 116 — Macro: CPU



## CONCLUSÃO

A construção da CPU no Logisim representou um exercício completo de integração de módulos digitais, permitindo aplicar de forma prática e progressiva conceitos fundamentais de arquitetura de computadores. Desde os blocos mais simples — como decoders, multiplexadores, registradores e unidades combinacionais — até componentes centrais como a ULA, o MIR e o banco de microinstruções, cada elemento foi projetado, testado e validado individualmente antes de ser incorporado à arquitetura final. Essa abordagem sistemática garantiu não apenas o correto funcionamento da CPU, mas também uma compreensão detalhada de como cada módulo contribui para o fluxo de controle e dados.

A implementação da ULA, em particular, se mostrou um dos pontos mais desafiadores e enriquecedores do desenvolvimento. A ampliação do conjunto de operações — incluindo operadores lógicos, aritméticos e o uso do divisor nativo do Logisim — exigiu um planejamento cuidadoso, bem como a construção de módulos claros, padronizados e integráveis ao restante da arquitetura.

Como consequência direta, o mapeamento do banco MIR tornou-se significativamente mais compreensível. A necessidade de coordenar corretamente cada operação da ULA, seus sinais habilitadores e as interações com componentes como MBR, MAR, registradores A e B e o shifter deixou explícita a importância da microprogramação. A elaboração do microcódigo passou a fazer sentido prático, conectando teoria e implementação ao demonstrar como uma instrução de máquina é decomposta em micro-ações coerentes.

Por fim, a arquitetura desenvolvida mantém um alto grau de modularidade e clareza estrutural. Essa característica facilita expansões futuras — seja na largura de dados, no número de operações suportadas pela ULA ou no conjunto de instruções implementados. Além disso, a organização cuidadosa dos blocos torna o projeto facilmente reutilizável dentro do Logisim, podendo ser incorporado como uma biblioteca funcional e bem estruturada para evoluções posteriores.

## Referências Bibliográficas

O entendimento sobre conversões digitais, expressões lógicas, tabela verdade, circuito e portas lógicas, lista de execução, sinais de saída, mintermos/maxtermos, portas básicas e avançadas, unidades lógicas aritméticas, decodificadores, multiplexadores, fundamentos da construção de circuitos digitais, foi possível a partir dos ensinamentos do professor José A. A. Viana (informação verbal, disciplina de Arquitetura de Organização de Computadores, FATEC Rio Preto, 2025).

O entendimento sobre módulos avançados, como flip-flops, latches, mar, mbr, registradores, banco de registradores, memória de instrução, sinais de clock, controle sequencial, shifters, foi desenvolvido a partir dos materiais complementares disponibilizados pelo professor José A. A. Viana (informação verbal, disciplina de Arquitetura de Organização de Computadores, FATEC Rio Preto, 2025).

Parte das decisões de projeto, organização estrutural e escolha de abordagem modular foram inspiradas por relatórios e materiais técnicos produzidos por grupos de alunos de turmas anteriores da mesma disciplina. Esses documentos serviram como base comparativa para compreender diferentes estilos de implementação, aprofundar o entendimento sobre o uso da ferramenta de simulação e permitir a expansão do projeto para áreas não exploradas nos trabalhos originais (materiais de consulta, Arquitetura e Organização de Computadores, FATEC Rio Preto, 2025).

Além dos materiais desenvolvidos especificamente para este projeto, diversos conceitos fundamentais sobre portas lógicas, circuitos sequenciais, registradores, unidades lógicas e aritméticas, datapaths, mecanismos de controle e princípios gerais de arquitetura foram complementados a partir de obras clássicas da área de computação e eletrônica digital. Entre os principais referenciais teóricos utilizados como base conceitual, destacam-se:

- **Tanenbaum**, A. S.; Austin, T. Structured Computer Organization. Pearson.
- **Mano**, M. Morris; Ciletti, M. Digital Design. Pearson.
- **Patterson**, D.; Hennessy, J. Computer Organization and Design: The Hardware/Software Interface. Morgan Kaufmann.
- **Tocci**, R.; Widmer, N.; Moss, G. Digital Systems: Principles and Applications. Prentice Hall.
- **Wakerly**, J. Digital Design: Principles and Practices. Prentice Hall.

O entendimento sobre o uso da ferramenta Logisim-Evolution foi feita de forma independente, explorando e testando a ferramenta para praticar, e realizando pesquisas de como utilizar alguns componentes nativos do software.

**VOO, Vanessa; DELGADO, Reniê; ALEXANDRO, Daniel.** *Sistemas Digitais – Circuitos Aritméticos*. Monitoria SD 2011.2, Centro de Informática – UFPE.

Disponível em: <https://www.cin.ufpe.br/~voo/sd/Aula5.pdf>. Acesso em: 11 de

novembro de 2025. Material utilizado para compreensão dos princípios de somadores, subtratores e fundamentos da aritmética digital, servindo como base para estudos posteriores sobre ULAs e operações combinacionais.

**ALLDATASHEET.** Biblioteca online de datasheets de componentes eletrônicos.

Disponível em: <https://www.alldatasheet.com>. Acesso em: 15 de novembro de 2025.

Consultado para obtenção de informações técnicas sobre circuitos integrados utilizados nos exemplos ou estudos relacionados.

**EMBARCADOS.** Portas Lógicas — fundamentos, exemplos e aplicações.

Disponível em: <https://embarcados.com.br/portas-logicas>. Acesso em: 15 de novembro de 2025. Referência utilizada para visualizar diagramas de portas lógicas em melhor qualidade e complementar a parte conceitual sobre elementos básicos de circuitos digitais.

**EL-GHONIMY, Osama.** Tri-state Buffer. YouTube, 05 de outubro de 2019.

Disponível em: <https://www.youtube.com/watch?v=ZcS7lzy1slg>. Acesso em: 30 nov. 2025.

**EngMicroLectures.** Tristate Buffers. YouTube, 16 de setembro de 2011. Disponível

em: <https://www.youtube.com/watch?v=X2HPjppH7Rs>. Acesso em: 30 nov. 2025.

**STALLINGS, William.** *Arquitetura e Organização de Computadores.*

**TANENBAUM, Andrew S.; AUSTIN, Todd.** *Organização Estruturada de Computadores.*

**MANO, Morris; CILETTI, Michael.** *Circuitos Digitais e Design de Sistemas.*

**PATTERSON, David; HENNESSY, John.** *Organização e Projeto de Computadores – A Interface Hardware/Software.*